

การพัฒนาเกมโดยประยุกต์ใช้เทคนิค Finite State Machine **ในการควบคุมตัวละครและปัญญาประดิษฐ์**

Development of a Game using Finite State Machine Techniques for Character Control and Artificial Intelligence

ชื่อผู้จัดทำ

นายจิรายุ สงบวาจา

นางสาวอรไพลิน สอาดเอี่ยม

นางสาววนิดา คล้ายสกุล

โครงงานวิจัยระดับปริญญาตรีนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิทยาศาสตรบัณฑิต
สาขาวิชาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์และเทคโนโลยี
มหาวิทยาลัยเทคโนโลยีราชมงคลสุวรรณภูมิ พ.ศ. 2569

การพัฒนาเกมโดยประยุกต์ใช้เทคนิค Finite State Machine **ในการควบคุมตัวละครและปัญญาประดิษฐ์**

Development of a Game using Finite State Machine Techniques for Character Control and Artificial Intelligence

ชื่อผู้จัดทำ

นายจิรายุ สงบวาจา

นางสาวอรไพลิน สอาดเอี่ยม

นางสาววนิดา คล้ายสกุล

โครงงานวิจัยระดับปริญญาตรีนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิทยาศาสตรบัณฑิต
สาขาวิชาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์และเทคโนโลยี
มหาวิทยาลัยเทคโนโลยีราชมงคลสุวรรณภูมิ พ.ศ. 2569

หัวข้อโครงงาน	การพัฒนาเกมโดยประยุกต์ใช้เทคนิค Finite State Machine ในการควบคุมตัวละครและปัญญาประดิษฐ์ Development of a Game using Finite State Machine Techniques for Character Control and Artificial Intelligence
ผู้จัดทำ	นายจิรายุ สงบวาจา นางสาวอรไพลิน สอาดเอี่ยม นางสาววนิดา คล้ายสกุล
อาจารย์ที่ปรึกษา	อาจารย์ธนวัฒน์ ถาวรกูล ผศ.วราภรณ์ มั่นทุ่ง
ระดับการศึกษา	ปริญญาตรี
สาขาวิชา	วิทยาการคอมพิวเตอร์
ปีการศึกษา	2569

บทคัดย่อ

โครงงานนี้มีวัตถุประสงค์เพื่อพัฒนาเกมสองมิติแนวเอาชีวิตรอดและลอบเร้นด้วย Unity และภาษา C# โดยใช้เครื่องสถานะจำกัด (Finite State Machine: FSM) ควบคุมตัวละครผู้เล่นและศัตรู Killer ระบบผู้เล่นประกอบด้วย Normal, Hidden, Frozen, Hurt, UsingItem, Transitioning และ Dead ส่วนศัตรูใช้สถานะสำหรับลาดตระเวน รับรู้เสียง ตรวจสอบ ค้นหา ไล่ล่า โจมตี และกรีดร้อง การตรวจจับใช้ Raycasting ร่วมกับเหตุการณ์เสียง
ผู้เล่นต้องเก็บกุญแจให้ครบและปลดล็อกประตูเพื่อผ่านด่านทั้งสาม ระบบ Animator เชื่อมพฤติกรรมกับภาพเคลื่อนไหวผ่านพารามิเตอร์ Bool และ Trigger ผลจากการตรวจโค้ดและสาธิตใน Play Mode พบโครงสร้างระบบและหน้าจอแสดงผลตามสถานะที่กำหนด อย่างไรก็ตาม การสาธิตมีการควบคุมตำแหน่งและเรียกสถานะผ่านสคริปต์
จึงยังไม่ยืนยันการเล่นครบวงจรหรือความถูกต้องทุกกรณี
ยังไม่ได้เก็บแบบสอบถามจากผู้ทดลองเล่นและไม่สามารถสรุประดับความพึงพอใจจริงได้ รายงานแสดงแผนทดสอบ ข้อจำกัด
และตัวอย่างข้อมูลสมมติสำหรับอธิบายวิธีวิเคราะห์คะแนนโดยแยกจากผลจริงอย่างชัดเจน

กิตติกรรมประกาศ

โครงงานวิจัยฉบับนี้สำเร็จลุล่วงไปได้ด้วยดีเนื่องด้วยความกรุณาจาก อาจารย์ธนวัฒน์ ถาวรกูล และ ผศ.วราภรณ์ มั่นทุ่ง อาจารย์ที่ปรึกษาโครงงาน ที่ได้ให้คำปรึกษา แนะนำแนวทาง และแก้ไขข้อบกพร่องต่าง ๆ ในการพัฒนาเกมสยองขวัญ 2 มิติด้วยภาษา C# ตลอดจนถ่ายทอดความรู้และเทคนิคต่าง ๆ ในการใช้โปรแกรม Unity จนโครงงานเล่มนี้เสร็จสมบูรณ์

ขอกราบขอบพระคุณบิดา มารดา และครอบครัว ที่สนับสนุนการศึกษาและการจัดทำโครงงาน รวมถึงผู้ที่ให้คำแนะนำและความช่วยเหลือในการพัฒนาระบบ ทั้งนี้การเก็บข้อมูลประเมินความพึงพอใจจากผู้ทดลองเล่นยังเป็นขั้นตอนที่จะดำเนินการต่อไป

คุณค่าและประโยชน์ของโครงงานวิจัยฉบับนี้ ผู้จัดทำขอมอบเป็นเครื่องบูชาพระคุณบิดา มารดา ครูบาอาจารย์ และผู้มีพระคุณทุกท่านที่มีส่วนช่วยให้ผู้จัดทำประสบความสำเร็จในการศึกษา

ชื่อผู้จัดทำ

จิรายุ สงบวาจา

อรไพลิน สอาดเอี่ยม

วนิดา คล้ายสกุล

สารบัญ

บทที่ 1 บทนำ 10

1.1 ความเป็นมาและความสำคัญของปัญหา 10

1.2 วัตถุประสงค์ของโครงงาน 11

1.3 ขอบเขตของโครงงาน 11

1.4 เครื่องมือที่เกี่ยวข้องกับโครงงาน 14

1.5 แผนการดำเนินโครงงาน 14

1.6 1.6 ประโยชน์ที่คาดว่าจะได้รับ 15

บทที่ 2 ทฤษฎีและงานวิจัยที่เกี่ยวข้อง 16

2.1 แนวคิดเกี่ยวกับการพัฒนาเกม (Game Development) 16

2.2 การพัฒนาเกม 2 มิติ 16

2.3 เกม Side-scrolling game หรือ Side-view game 17

2.4 เกมประเภท Hide-and-Seek Horror Game 17

2.5 แนวคิดเกี่ยวกับ Finite State Machine (FSM) 17

2.6 การพัฒนาปัญญาประดิษฐ์ในเกม 18

2.7 ภาษาและเครื่องมือที่ใช้พัฒนา 18

2.8 อัลกอริทึมการค้นหาเส้นทาง (Pathfinding Algorithm) 19

2.9 2.9 การออกแบบ UX และ UI ให้เหมาะสม 20

2.10 งานวิจัยและเอกสารที่เกี่ยวข้อง 20

บทที่ 3 วิธีการดำเนินงาน 22

3.1 การกำหนดปัญหา 22

3.2 การรวบรวมข้อมูล 23

3.3 การวิเคราะห์ระบบ 24

3.4 การออกแบบระบบ 26

3.5 การพัฒนาระบบ 38

3.6 การทดสอบระบบ 47

3.7 การบำรุงรักษาระบบ 49
บทที่ 4 ผลการดำเนินงาน 51
4.1 ผลการพัฒนาระบบควบคุมตัวละครผู้เล่น 51
4.2 ผลการพัฒนา Finite State Machine ของผู้เล่น 53
4.3 ผลการพัฒนาระบบปัญญาประดิษฐ์ของศัตรู 54
4.4 ผลการพัฒนาระบบตรวจจับผู้เล่น 56
4.5 ผลการพัฒนาระบบไอเท็มและประตูทางออก 58
4.6 ผลการพัฒนาระบบส่วนติดต่อผู้ใช้งาน 59
4.7 ผลการพัฒนาด่านภายในเกม 61
4.8 ผลการพัฒนาระบบความตึงเครียด 63
4.9 ผลการพัฒนาระบบแอนิเมชันของตัวละคร 64
4.10 ผลการทดสอบการทำงานของระบบ 67
4.11 ผลการประเมินโดยผู้ใช้งาน 70
4.12 สรุปผลการดำเนินงาน 71
บทที่ 5 สรุปผลโครงงานและข้อเสนอแนะ 72
5.1 สรุปผลโครงงาน 72
5.2 ปัญหาที่พบในโครงงาน 75
5.3 ข้อเสนอแนะ 77
บรรณานุกรม 79
ภาคผนวก ก หลักฐานและวิธีการสาธิต 80
ภาคผนวก ข ตัวอย่างคะแนนสมมติและแบบประเมิน 81

สารบัญภาพ

ภาพที่ 3-1 หน้าจอเมนูหลัก (ภาพสาธิตใน Play Mode) 27
ภาพที่ 3-2 หน้าจอตั้งค่าเสียง (ภาพสาธิตใน Play Mode)..... 28
ภาพที่ 3-3 หน้าจอระหว่างเล่น (ภาพสาธิตใน Play Mode) 28
ภาพที่ 3-4 การวิ่งและระบบ Stamina (ภาพสาธิตใน Play Mode) 29
ภาพที่ 3-5 สาธิตตำแหน่งกุญแจและระบบเก็บไอเท็ม 29
ภาพที่ 3-6 หน้าจอคลังไอเท็ม (ภาพสาธิตใน Play Mode) 30
ภาพที่ 3-7 ระบบซ่อนตัว (ภาพสาธิตใน Play Mode) 30
ภาพที่ 3-8 ระบบไฟฉายและแบตเตอรี่ (ภาพสาธิตใน Play Mode) 31
ภาพที่ 3-9 สาธิต Killer รับเหตุการณ์เสียงประเภท Rock 31
ภาพที่ 3-10 การตรวจพบและไล่ล่าผู้เล่น (ภาพสาธิตใน Play Mode)..... 32
ภาพที่ 3-11 การได้รับความเสียหายและ Game Over (ภาพสาธิตใน Play Mode)..... 33
ภาพที่ 3-12 สาธิตข้อความและความคืบหน้าการปลดล็อกประตู 34
ภาพที่ 3-13 หน้าจอผ่านด่านและจบเกม (ภาพสาธิตใน Play Mode) 35
ภาพที่ 3-14 ระบบคำสาปใน Level 2 (ภาพสาธิตใน Play Mode)..... 36
ภาพที่ 3-15 สภาพแวดล้อมสวนสนุกและพื้นที่ป่าใน Level 3 (ภาพสาธิตใน Play Mode)..... 36
ภาพที่ 3-16 สาธิตตำแหน่งกับดักเสียงใน Level 3 37
ภาพที่ 3-17 ระบบแจ้งเตือนเมื่อศัตรูอยู่ใกล้ (ภาพสาธิตใน Play Mode)..... 37
ภาพที่ 3-18 แผนภาพสถาปัตยกรรมของระบบ..... 39
ภาพที่ 3-19 แผนภาพ Player FSM..... 40
ภาพที่ 3-20 แผนภาพ Enemy FSM 41
ภาพที่ 3-21 แผนภาพสถานะความตึงเครียดและการส่งข้อมูลเสียง..... 42
ภาพที่ 3-22 แผนผัง Animator ของตัวละครผู้เล่นจาก Unity..... 43
ภาพที่ 3-23 แผนผัง Animator และ Parameters ของ Killer จาก Unity..... 44
ภาพที่ 4-1 ผลการทำงานของระบบควบคุมตัวละครผู้เล่น (ภาพสาธิตใน Play Mode)..... 52
ภาพที่ 4-2 ผลการทำงานของระบบไฟฉาย (ภาพสาธิตใน Play Mode) 53

ภาพที่ 4-3 สาธิตหน้าจอ Normal และ Hidden ของผู้เล่น 54
ภาพที่ 4-4 Killer ในสถานะ Patrol ที่จัดไว้เพื่อสาธิต 55
ภาพที่ 4-5 สาธิตการรับเสียง Rock และสถานะ Listening 55
ภาพที่ 4-6 สาธิตสถานะ Chase และข้อความ DANGER 56
ภาพที่ 4-7 แผนภาพเงื่อนไขการตรวจแนวสายตาจาก KillerAI 57
ภาพที่ 4-8 สาธิตพื้นที่กับดักเสียงของ Level 3 57
ภาพที่ 4-9 ผลการทำงานของระบบเก็บกุญแจ (ภาพสาธิตใน Play Mode) 58
ภาพที่ 4-10 ผลการทำงานของระบบประตูทางออก (ภาพสาธิตใน Play Mode) 59
ภาพที่ 4-11 ผลการแสดงข้อมูลบนหน้าจอระหว่างเล่น (ภาพสาธิตใน Play Mode) 60
ภาพที่ 4-12 ผลการแสดงหน้าจอสรุปสถานะเกม (ภาพสาธิตใน Play Mode) 61
ภาพที่ 4-13 ผลการพัฒนาด่านภายในเกม (ภาพสาธิตใน Play Mode) 62
ภาพที่ 4-14 สวนสนุกและพื้นที่ป่าใน Level 3 (ภาพสาธิตใน Play Mode) 63
ภาพที่ 4-15 เปรียบเทียบหน้าจอปกติกับคำเตือนเมื่อเรียก Chase (ภาพสาธิตใน Play Mode) 64
ภาพที่ 4-16 ตัวอย่างสี่เฟรมแรกของแอนิเมชัน Idle ของผู้เล่น 65
ภาพที่ 4-17 ตัวอย่างสี่เฟรมแรกของแอนิเมชัน Walk ของผู้เล่น 65
ภาพที่ 4-16 และ 4-17 แสดงท่ายืนและท่าเดินที่นำไปใช้กับ PlayerIdle.anim และ PlayerWalk.anim ตามลำดับ ทั้งสองคลิปกำหนด Sample Rate เท่ากับ 12 และเปิด Loop Time เพื่อเล่นวนขณะตัวละครคงอยู่ในสถานะเดิม โดยคลิป Idle มีช่วงเวลา 0.75 วินาที ส่วน Walk มีช่วงเวลา 1 วินาที 65
ภาพที่ 4-18 ตัวอย่างสี่เฟรมแรกของแอนิเมชัน Walk ของ Killer 66
ภาพที่ 4-19 ตัวอย่างสี่เฟรมแรกของแอนิเมชัน Run ของ Killer 66
ภาพที่ 4-20 ตัวอย่างสี่เฟรมแรกของแอนิเมชัน Attack1 ของ Killer 66
ภาพที่ 4-21 ตัวอย่างสี่เฟรมแรกของแอนิเมชัน Scream ของ Killer 66

สารบัญตาราง

ตารางที่ 3-1 คำสั่งควบคุมเกม 24
ตารางที่ 3-2 สถานะของ Player FSM 24
ตารางที่ 3-3 สถานะของ Enemy FSM 25
ตารางที่ 3-4 พารามิเตอร์ของแต่ละด่าน 26
ตารางที่ 3-5 แผนทดสอบการควบคุมพื้นฐาน 47
ตารางที่ 3-6 แผนทดสอบ Player FSM 47
ตารางที่ 3-7 แผนทดสอบ Enemy FSM 48
ตารางที่ 3-8 แผนทดสอบไอเท็มและ Level 3 49
ตารางที่ 4-1 ผลการทดสอบระบบควบคุมพื้นฐาน 68
ตารางที่ 4-2 ผลการทดสอบ Player FSM 68
ตารางที่ 4-3 ผลการทดสอบ Enemy FSM 69
ตารางที่ 4-4 ผลการทดสอบระบบเกมและ Level 3 70
ตารางที่ 4-5 สถานะข้อมูลการประเมินจากผู้ใช้งานจริง 70
ตารางที่ ก-1 ตัวอย่างสถานะจากบันทึกการสาธิต 80
ตารางที่ ข-1 คะแนนสมมติสำหรับตัวอย่างการคำนวณเท่านั้น 81
ตารางที่ ข-2 ผลคำนวณจากคะแนนสมมติเท่านั้น 81
ตารางที่ ข-3 แบบประเมินสำหรับผู้ทดลองเล่นจริง 82

บทที่ 1 บทนำ

1.1 ความเป็นมาและความสำคัญของปัญหา

เกมแนวลอบเร้นต้องควบคุมการเคลื่อนที่ การซ่อนตัว การใช้ไอเท็ม และการตอบสนองต่ออันตรายให้สัมพันธ์กัน หากตรวจเงื่อนไขทั้งหมดโดยไม่มีโครงสร้างสถานะ อาจเกิดคำสั่งที่ขัดแย้ง เช่น การเคลื่อนที่ขณะซ่อนหรือเสียชีวิต โครงงานจึงใช้ FSM แยกพฤติกรรมและเงื่อนไขการรับคำสั่งของผู้เล่นและศัตรูให้ตรวจสอบได้ชัดเจน

นอกจากนี้ การสร้างศัตรูคอมพิวเตอร์ (AI Agent) ให้มีความฉลาดและตอบสนองต่อพฤติกรรมของผู้เล่นอย่างสมจริง จำเป็นต้องประยุกต์ใช้อัลกอริทึมขั้นสูง เช่น Finite State Machine (FSM) ในการจำแนกสถานะ และการคำนวณทางคณิตศาสตร์เวกเตอร์ (Vector Mathematics) เพื่อตรวจสอบการมองเห็น (Line of Sight) และการได้ยินเสียงที่แปรผันตามท่าทางของผู้เล่น

ผู้จัดทำจึงมีแนวคิดพัฒนาโครงงาน "การพัฒนาเกมโดยประยุกต์ใช้เทคนิค Finite State Machine ในการควบคุมตัวละครและปัญญาประดิษฐ์" โดยใช้ Unity Game Engine และภาษา C# โครงงานนี้จะมุ่งเน้นการเขียนโปรแกรมเพื่อจัดการกลไกการเคลื่อนไหวที่ซับซ้อน (Advanced Movement Mechanics) และระบบ AI ที่ไล่ล่าผู้เล่นอย่างเป็นระบบ เพื่อเป็นกรณีศึกษาการนำความรู้วิทยาการคอมพิวเตอร์มาประยุกต์ใช้ในการสร้างซอฟต์แวร์ที่มีการโต้ตอบสูง

1.2 วัตถุประสงค์ของโครงงาน

1.2.1 เพื่อพัฒนาเกมคอมพิวเตอร์ 2 มิติ ด้วยภาษา **C#** โดยเน้นการวางโครงสร้างโค้ดแบบเชิงวัตถุ (**Object-Oriented Programming)**

1.2.2 เพื่อประยุกต์ใช้ Finite State Machine ในการจัดการสถานะการควบคุมตัวละครผู้เล่น

1.2.3 เพื่อพัฒนาปัญญาประดิษฐ์ของศัตรูที่ตอบสนองต่อการมองเห็น เสียง และสถานะการซ่อนของผู้เล่น

1.2.4 เพื่อสร้างระบบการเล่น (**Gameplay Loop)** ที่สมบูรณ์ ทั้งการเก็บไอเทม การแก้ไขปริศนา และการเปลี่ยนฉากเมื่อผ่านเงื่อนไข

1.2.5 เพื่อประเมินความพึงพอใจของผู้เล่นต่อเกมที่พัฒนาขึ้น เพื่อนำไปปรับปรุงและพัฒนาระบบให้มีประสิทธิภาพยิ่งขึ้น

1.3 ขอบเขตของโครงงาน

1.3.1 ส่วนพัฒนาระบบควบคุมตัวละคร (Player Controller System)

พัฒนาระบบควบคุมตัวละครผู้เล่นให้สามารถเคลื่อนไหวได้หลากหลายรูปแบบ และส่งผลต่อการเล่นในเชิงกลยุทธ์ เช่น การลดเสียงเพื่อหลบศัตรู

1.3.1.1 State Management

ใช้ FSM ควบคุมสถานะ Normal, Hidden, Frozen, Hurt, UsingItem, Transitioning และ Dead โดยแยกจากสถานะภาพเคลื่อนไหว Idle และ Walk ใน Animator

1.3.1.2 การควบคุมการชนและการซ่อนตัว

ใช้ Rigidbody2D และ Collider2D ควบคุมการเคลื่อนที่ การตรวจพื้น และการชน ส่วน HidingSystem ควบคุมการเข้าและออกจากจุดซ่อน
รุ่นที่ตรวจสอบยังไม่มีการนั่งย่อและหมอบคลานเป็นสถานะผู้เล่น

1.3.1.3 Variable Parameters

กำหนดค่าความเร็วและระดับเสียงในแต่ละสถานะ เพื่อใช้เป็นข้อมูลในการตรวจจับของศัตรู

1.3.2 ส่วนพัฒนาระบบปัญญาประดิษฐ์ของศัตรู (**Enemy AI System)**

พัฒนาระบบ AI ที่สามารถตอบสนองต่อพฤติกรรมของผู้เล่น
โดยเน้นการตรวจจับและการตัดสินใจแบบเรียลไทม์

1.3.2.1 Behavior System (FSM)

กำหนดสถานะของศัตรู เช่น Patrol, Investigate, Chaseและ Attack เพื่อควบคุมพฤติกรรม

1.3.2.2 Sensory System

พัฒนาระบบการมองเห็นด้วย Raycasting
และระบบการได้ยินโดยใช้การคำนวณระยะทางร่วมกับระดับเสียงของผู้เล่น

1.3.2.3 การเคลื่อนที่ตามเป้าหมาย

ศัตรูเคลื่อนที่แนวนอนด้วย Rigidbody2D ไปยังจุดลาดตระเวน ตำแหน่งเสียง
หรือตำแหน่งผู้เล่น พร้อมตรวจพื้นและสิ่งกีดขวาง รุ่นปัจจุบันยังไม่ได้ใช้ NavMesh หรือ A*
สำหรับคำนวณเส้นทาง

1.3.3 ส่วนพัฒนาระบบเกม (Game Logic System)

พัฒนากลไกหลักของเกมเพื่อกำหนดเป้าหมายและลำดับการเล่น

1.3.3.1 Inventory และ Mission Logic

ผู้เล่นต้องเก็บกุญแจ 5 ดอกตาม LevelConfig ของแต่ละด่าน
แล้วปลดล็อกประตูทางออก

1.3.3.2 Scene Management

ใช้ GameManager และ SceneManager จัดการเมนู ด่าน การแพ้และการชนะ
โดยการบันทึกความคืบหน้าข้ามการเปิดเกมเป็นข้อเสนอสำหรับพัฒนาต่อ

1.3.4 ส่วนออกแบบส่วนติดต่อผู้ใช้งาน (UI/UX)

เป็นการออกแบบส่วนติดต่อผู้ใช้เพื่อแสดงข้อมูลสำคัญ เช่น สถานะตัวละคร ระดับเสียง
และจำนวนไอเทม เพื่อช่วยให้ผู้เล่นสามารถวางแผนการเล่นและตัดสินใจได้อย่างมีประสิทธิภาพ

1.3.5 ขอบเขตด้านเนื้อหาเกม (**Game Content Scope)**

1.3.5.1 เกมสองมิติแนวลอบเร้น (Stealth Game) โดยผู้เล่นหลบศัตรูและวางแผนการเคลื่อนที่

1.3.5.2 มีฉากการเล่น 3 ด่าน พร้อมสิ่งกีดขวาง จุดซ่อน ไอเท็ม และประตูทางออก

1.3.5.3 มีตัวละครศัตรูอย่างน้อย 1 ประเภท ที่สามารถลาดตระเวน ตรวจจับเสียง และไล่ล่าผู้เล่นได้

1.3.5.4 มีระบบการเก็บไอเทม โดยผู้เล่นต้องรวบรวมไอเทมให้ครบตามจำนวนที่กำหนด เพื่อผ่านด่าน

1.3.5.5 มีเงื่อนไขการชนะ (Win Condition) เมื่อผู้เล่นเก็บไอเทมครบและไปถึงทางออกของฉาก

1.3.5.6 ผู้เล่นแพ้เมื่อพลังชีวิตหมด และระบบแสดงหน้าจอ Game Over

1.3.6 ขอบเขตด้านเทคโนโลยี (**Technology Scope)**

1.3.6.1 ใช้ Unity Game Engine ในการพัฒนาเกม

1.3.6.2 ใช้ภาษา C# ในการเขียนโปรแกรมควบคุมระบบต่าง ๆ ภายในเกม

1.3.6.3 ใช้แนวคิดการเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming OOP) ในการออกแบบโครงสร้างระบบ

1.3.6.4 ใช้ Finite State Machine ควบคุมสถานะของตัวละครผู้เล่นและศัตรู

1.3.6.5 ใช้ Raycasting ในการพัฒนาระบบการมองเห็นของศัตรู และใช้การคำนวณระยะทางสำหรับระบบตรวจจับเสียง

1.3.6.6 ใช้การเคลื่อนที่ด้วย Rigidbody2D และการตรวจพื้นกับสิ่งกีดขวาง โดยศึกษาการค้นหาเส้นทางเป็นแนวทางสำหรับพัฒนาต่อ

1.4 เครื่องมือที่เกี่ยวข้องกับโครงงาน

1.4.1 **Unity Game Engine** เครื่องมือหลักในการพัฒนา

1.4.2 ภาษา **C#** สำหรับเขียนสคริปต์ควบคุมการทำงาน

1.4.3 **Visual Studio: IDE** สำหรับเขียนโค้ดและ **Debug**

1.4.4 โปรแกรมสร้างกราฟิก 2**D:** สำหรับสร้าง **Sprite** ตัวละครและฉาก

1.5 แผนการดำเนินโครงงาน

การดำเนินโครงงานวิจัย	ระยะเวลาการดำเนินโครงงานวิจัย																							
	พ.ย.		ธ.ค.		ม.ค.		ก.พ.		มี.ค.		เม.ย.		พ.ค.		มิ.ย.		ก.ค.		ส.ค.		ก.ย.	
1. ศึกษาปัญหาและความต้องการ																						
2. กำหนดขอบเขต																						
3. ศึกษาการใช้งาน Unity																						
4. ออกแบบตัวละคร																						
5. ออกแบบระบบ																						
6. พัฒนาระบบ																						
7. ติดตั้งโปรแกรมและทดสอบ																						
8. ปรับปรุงและแก้ไขระบบ																						
10. ทำเอกสารงานวิจัย																						
11. พบอาจารย์ที่ปรึกษา																						

แผนงานแบ่งเป็นการศึกษาข้อมูล วิเคราะห์และออกแบบ พัฒนาโมดูล ทดสอบระบบ และจัดทำรายงาน ตารางกำหนดเวลาในต้นฉบับใช้เป็นแผนการดำเนินงาน ไม่ใช้ยืนยันวันที่เสร็จจริงของแต่ละกิจกรรม

สถานะปัจจุบันคือมีต้นแบบเกมและรายงานประกอบ การตรวจกรณีทดสอบครบวงจรและการเก็บข้อมูลผู้ทดลองเล่นยังต้องดำเนินการต่อ

1.6 1.6 ประโยชน์ที่คาดว่าจะได้รับ

1.6.1 ได้รับความรู้ความเข้าใจเชิงลึกในการเขียนโปรแกรมจัดการสถานะ (State Management) ซึ่งเป็นพื้นฐานสำคัญของการพัฒนาซอฟต์แวร์

1.6.2 สามารถประยุกต์ใช้คณิตศาสตร์และอัลกอริทึม (Physics & Algorithms) ในการสร้างระบบการตรวจจับ (Detection System) ภายในเกมได้

1.6.3 ได้ต้นแบบซอฟต์แวร์เกมที่มีระบบการควบคุมตัวละคร (Character Controller) และ AI ที่ทำงานสัมพันธ์กันอย่างเป็นระบบ

1.6.4 มีทักษะในการออกแบบ UX/UI ที่สื่อสารข้อมูลเชิงเทคนิค (เช่น ระดับเสียง) ให้ผู้ใช้งานเข้าใจได้ง่าย

1.6.5 ได้แนวทางเก็บข้อมูลความพึงพอใจเพื่อใช้ปรับการควบคุม ความยาก และการสื่อสารข้อมูลของเกม

บทที่ 2 ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

ในการพัฒนาเกมคอมพิวเตอร์ 2 มิติ แนว Side-scrolling Hide-and-Seek Horror Game ด้วย C# ผู้จัดทำได้ศึกษาหลักการทฤษฎีและเทคโนโลยีที่ เกี่ยวข้อง เพื่อใช้เป็นแนวทางในการดำเนินงานวิจัย ประกอบไปด้วยหัวข้อหลัก ๆ ดังต่อไปนี้

2.1 แนวคิดเกี่ยวกับการพัฒนาเกม (Game Development)

2.2 การพัฒนาเกม 2 มิติ

2.3 เกม Side-scrolling game หรือ Side-view game

2.4 เกมประเภท Hide-and-Seek Horror Game

2.5 แนวคิดเกี่ยวกับ Finite State Machine (FSM)

2.6 การพัฒนาปัญญาประดิษฐ์ในเกม

2.7 ภาษาและเครื่องมือที่ใช้พัฒนา

2.8 อัลกอริทึมการค้นหาเส้นทาง (Pathfinding Algorithm)

2.9 การออกแบบ ux ui ให้เหมาะสม

2.10 งานวิจัยที่เกี่ยวข้อง

2.1 แนวคิดเกี่ยวกับการพัฒนาเกม (Game Development)

การพัฒนาเกมเป็นกระบวนการสร้างซอฟต์แวร์เชิงโต้ตอบ ประกอบด้วยการวิเคราะห์ความต้องการ ออกแบบกลไก พัฒนาโปรแกรมและสื่อ ทดสอบ และประเมินผล องค์ประกอบหลัก ได้แก่ การควบคุม กราฟิก แอนิเมชัน เสียง ปัญญาประดิษฐ์ และส่วนติดต่อผู้ใช้ การแยกองค์ประกอบเป็นโมดูลช่วยให้ตรวจสอบปัญหาและปรับระบบได้สะดวก โดยควรใช้ผลการทดสอบจริงยืนยันว่าระบบที่พัฒนาตอบสนองต่อเป้าหมายของเกม

2.2 การพัฒนาเกม 2 มิติ

เกมสองมิติใช้พิกัดแกน X และ Y ในการกำหนดตำแหน่ง โดย Unity รองรับ Sprite, Tilemap, Physics 2D และ Animator การออกแบบต้องพิจารณาระดับพื้น สิ่งกีดขวาง การชน และขอบเขตกล้อง เพื่อให้ภาพกับพื้นที่ที่ผู้เล่นเคลื่อนที่ได้สอดคล้องกัน ประสิทธิภาพขึ้นอยู่กับจำนวนวัตถุ เอฟเฟกต์ การตรวจฟิสิกส์ และวิธีจัดการทรัพยากร จึงต้องวัดบนอุปกรณ์เป้าหมาย

2.3 เกม Side-scrolling game หรือ Side-view game

เกม Side-scrolling game หรือ Side-view game คือเกมที่มุมกล้อง ถูกกำหนดให้มองจากด้านข้างของตัวละคร ทำให้ผู้เล่นเห็นตัวละครในแนวตั้งและสามารถ เคลื่อนที่ไปทางซ้ายหรือขวาตามแกน X เป็นหลัก มุมมองแบบนี้เป็นที่นิยมอย่างกว้างขวางในเกมยุคแรก ๆ และยังคงได้รับความนิยมอย่างต่อเนื่องในเกมยุคปัจจุบัน เนื่องจาก สามารถสร้างประสบการณ์การเล่นที่ชัดเจนและเข้าใจง่าย คุณลักษณะสำคัญของเกม Side-scrolling ได้แก่ การเคลื่อนที่ของตัวละครใน แนวแกน X เป็นหลัก กล้องจะเลื่อนตาม (Follow Camera) ไปพร้อมกับตัวละคร การออกแบบฉากในแนวนอนที่มีความต่อเนื่อง รวมถึงการใช้ระบบชั้น (Layering) เพื่อสร้างความลึกในภาพแบบ 2 มิติ เช่น ฉากหลังเลื่อนช้ากว่าฉากหน้า (Parallax Scrolling) ซึ่งช่วยเพิ่มความสมจริงให้กับเกม ในการพัฒนาเกม Side-scrolling Horror Game นั้น มุมมองด้านข้างช่วย สร้างบรรยากาศความหวาดกลัวได้อย่างมีประสิทธิภาพ เนื่องจากผู้เล่นจะมองเห็นสิ่งที่อยู่ ข้างหน้าและข้างหลังได้อย่างชัดเจน แต่ไม่สามารถมองเห็นสิ่งที่อยู่ด้านบนหรือด้านล่างได้ ครบถ้วน ทำให้เกิดความตื่นเต้นและความไม่แน่นอนขณะเล่น

2.4 เกมประเภท Hide-and-Seek Horror Game

เกมแนวซ่อนตัวและสยองขวัญให้ผู้เล่นหลบผู้ไล่ล่า รวบรวมสิ่งของ และหาทางออก โดยใช้การมองเห็น เสียง และทรัพยากรจำกัดสร้างความเสี่ยง ในโครงงานนี้ผู้เล่นเลือกเดินหรือวิ่ง ใช้ไฟฉาย ซ่อนตัว และใช้ก้อนหินสร้างเสียงล่อ
การปรับพฤติกรรมศัตรูและการจัดวางฉากมีผลต่อความยาก
จึงควรประเมินจากทั้งการทดสอบทางเทคนิคและผู้ทดลองเล่น

2.5 แนวคิดเกี่ยวกับ Finite State Machine (FSM)

Finite State Machine (FSM) หรือเครื่องสถานะจำกัด เป็นแบบจำลอง และการ การคำนวณที่ใช้กำหนดพฤติกรรมของระบบผ่านชุดของสถานะ (States) เปลี่ยนผ่านสถานะ (Transitions) ตามเงื่อนไขที่กำหนด โดยในช่วงเวลาใดเวลาหนึ่ง ระบบจะอยู่ได้เพียงสถานะเดียวเท่านั้น FSM เป็นที่นิยมอย่างกว้างขวางในการพัฒนาเกม เนื่องจากเข้าใจง่าย ใช้ทรัพยากรประมวลผลต่ำ และสามารถปรับแก้พฤติกรรมได้โดยไม่ กระทบโครงสร้างทั้งหมด

องค์ประกอบของ FSM ได้แก่ สถานะเริ่มต้น สถานะที่เป็นไปได้ เหตุการณ์ เงื่อนไข และการเปลี่ยนสถานะ โครงงานใช้ FSM ในสคริปต์เพื่อกำหนดสิทธิ์การควบคุมผู้เล่นและพฤติกรรมศัตรู ส่วน Animator ใช้เครื่องสถานะอีกชุดหนึ่งสำหรับเลือกภาพเคลื่อนไหว
ทั้งสองส่วนเชื่อมกันผ่านพารามิเตอร์ จึงไม่จำเป็นต้องมีชื่อหรือจำนวนสถานะเท่ากัน เช่น Normal รองรับทั้งการยืนและเดิน ส่วน Chase ของ Killer ส่งค่าให้เล่น Run

2.6 การพัฒนาปัญญาประดิษฐ์ในเกม

Game Al (Game Artificial Intelligence) หมายถึง ระบบที่ควบคุม พฤติกรรมของตัวละครในเกมให้สามารถโต้ตอบกับผู้เล่นได้อย่างสมเหตุสมผล เป้าหมาย หลักของ Game Al ไม่ใช่การเรียนรู้ชั้นสูงแบบ Machine Learning แต่เป็นการ สร้าง "ภาพลวงตาของความฉลาด" เพื่อเพิ่มความสนุกและความท้าทายในการเล่น เทคนิคที่นิยมใช้ใน Game Al ได้แก่ Rule-based System, Finite State Machine (FSM), Behavior Tree และ Pathfinding Algorithm ในเกม 2 มิติแนว Hide-and-Seek Horror Game ระบบ Al ของศัตรูจำเป็นต้อง ผสมผสานเทคนิคหลายอย่างเข้าด้วยกัน ดังนี้

2.6.1 Finite State Machine (FSM) สำหรับศัตรู ใช้ FSM ในการจัดการสถานะพฤติกรรมของศัตรู (Ghost/Killer Al) โดยแบ่ง ออกเป็นสถานะ Patrol, Investigate, Chase และ Attack โดยแต่ละ สถานะจะเปลี่ยนไปตามเงื่อนไขที่ตรวจจับได้จากสภาพแวดล้อม เช่น ระยะสายตา และระดับเสียงของผู้เล่น

2.6.2 Sensory System (ระบบการรับรู้) พัฒนาระบบรับรู้ของ Al โดยใช้ Raycasting ในการจำลองสายตา (Line of Sight / Vision Cone) เพื่อตรวจสอบว่าผู้เล่นอยู่ในแนวสายตาของศัตรูหรือไม่ และใช้การคำนวณระยะทาง (Distance Check) เปรียบเทียบกับ Noise Radius ที่ผู้เล่นสร้างขึ้นตามท่าทางการเคลื่อนไหว

2.6.3 การเคลื่อนที่และการนำทาง ระบบนำทางอาจใช้กราฟ กริด หรือข้อมูลพื้นผิวร่วมกับอัลกอริทึมค้นหาเส้นทาง ในต้นแบบนี้ KillerAI กำหนดความเร็วแนวนอนตามตำแหน่งเป้าหมายและตรวจสิ่งกีดขวาง การนำ A* มาใช้เป็นแนวทางพัฒนาต่อสำหรับฉากที่ซับซ้อนกว่าเดิม

2.7 ภาษาและเครื่องมือที่ใช้พัฒนา

2.7.1 Visual Studio Code

เป็นโปรแกรมแก้ไขโค้ด (Code Editor) ที่พัฒนาโดย Microsoft มีความ ยืดหยุ่นสูง รองรับภาษาโปรแกรมต่างๆ ด้วยส่วนขยาย (Extensions) ที่หลาก หลาย สามารถเชื่อมต่อกับภาษา C# และ Unity Game Engine ได้อย่าง สะดวก มีระบบ IntelliSense ช่วยแนะนำโค้ดและแก้ไขข้อผิดพลาดได้อย่างมี ประสิทธิภาพ

2.7.2 เอ็นจิ้นเกม Unity

เป็นแอปพลิเคชันสำหรับการพัฒนาเกม มีความยืดหยุ่นสูง รองรับการพัฒนาเกมทั้ง 2 มิติ และ 3 มิติ โดยสามารถส่งออกเกมไปยัง Windows, Android และ iOS ได้อย่างสะดวก Unity มีเครื่องมือสำคัญดังนี้

2.7.2.1 Scene Editor ใช้ในการออกแบบฉากและจัดวางวัตถุใน เกม รวมถึงการสร้าง Tilemap สำหรับเกม 2 มิติแนว Side-scrolling

2.7.2.2 Animator ใช้ควบคุมและแสดงผลแอนิเมชันของตัวละครใน สถานะต่าง ๆ เช่น Walk, Crouch, Prone

2.7.2.3 Physics Engine 2D จัดการระบบฟิสิกส์ การชน (Collision) และการเคลื่อนที่ของวัตถุในเกม 2 มิติ

2.7.3 ภาษาโปรแกรม C#

เป็นภาษาหลักที่ใช้ในการพัฒนาเกมใน Unity มีโครงสร้างเชิงวัตถุ (OOP) ที่ ชัดเจน รองรับการสร้างระบบที่ซับซ้อน เช่น ระบบ FSM ระบบ AI ระบบ Inventory และการจัดการสถานะตัวละคร โดยในโครงงานนี้ใช้ C# เขียน สคริปต์ควบคุมการทำงานทั้งหมด

2.7.3.1 Game Logic Scripting สำหรับเขียนตรรกะการทำงาน ของระบบต่าง ๆ เช่น การเปลี่ยนสถานะ FSM และการตรวจจับ

2.7.3.2 Event Handling ใช้จัดการเหตุการณ์ต่าง ๆ เช่น Input ของผู้เล่น การชน และการเปลี่ยนฉาก

2.7.3.3 Data Management ใช้จัดการข้อมูลไอเทม สถานะผู้เล่น และการส่งต่อข้อมูลข้ามฉาก (Data Persistence)

2.7.4 JSON (JavaScript Object Notation)

เป็นรูปแบบไฟล์ข้อมูลที่มีโครงสร้างแบบคู่ระหว่างชื่อ (Key) และค่า (Value) ซึ่งอ่านและแก้ไขได้ง่ายในรูปแบบ Human-Readable Format สามารถใช้ ร่วมกับภาษา C# ใน Unity เพื่อบันทึกและโหลดข้อมูลเกม เช่น ข้อมูลตัวละคร และความคืบหน้าในการเล่น

2.8 อัลกอริทึมการค้นหาเส้นทาง (Pathfinding Algorithm)

การค้นหาเส้นทางเป็นการเลือกเส้นทางจากจุดเริ่มต้นไปยังเป้าหมายบนข้อมูลกราฟหรือกริด A* จัดลำดับโหนดด้วย $f(n) = g(n) + h(n)$ โดย $g(n)$ คือค่าใช้จ่ายที่สะสมและ $h(n)$ คือค่าประมาณที่เหลือ คุณสมบัติการหาเส้นทางเหมาะสมขึ้นอยู่กับเงื่อนไขของ heuristic และวิธีใช้งาน

หัวข้อนี้เป็นความรู้สำหรับพัฒนาต่อ รุ่นปัจจุบันใช้การเคลื่อนที่แนวนอนและตรวจพื้น ไม่ได้เรียก A* หรือ NavMesh จึงยังไม่มีผลเปรียบเทียบประสิทธิภาพของอัลกอริทึมในโครงงาน

2.9 2.9 การออกแบบ UX และ UI ให้เหมาะสม

UI ต้องสื่อสารทรัพยากรและอันตรายโดยไม่บดบังพื้นที่เล่น โครงงานใช้ HUD แสดง Health, Stamina, Battery, Keys และจำนวนก้อนหิน พร้อมข้อความ HIDDEN และ DANGER และ Prompt สำหรับวัตถุที่โต้ตอบได้ สีและตำแหน่งข้อความควรอ่านง่าย
ส่วนเสียงและขอบจอช่วยแจ้งอันตรายโดยไม่ต้องมองเห็นศัตรูตลอดเวลา
ความชัดเจนของการสื่อสารต้องตรวจด้วยผู้ทดลองเล่นเพิ่มเติม

2.10 งานวิจัยและเอกสารที่เกี่ยวข้อง

2.10.1 Finite State Machine in Game Development

Jagdale (2021) อธิบายการจัดพฤติกรรม NPC ด้วยสถานะและเงื่อนไขการเปลี่ยนสถานะ รวมถึงแนวคิด FSM แบบลำดับชั้น แนวคิดนี้ใช้เป็นพื้นฐานในการแยกการลาดตระเวน การตรวจสอบ และการโจมตีของ Killer
โดยรายงานนี้ไม่ใช้บทความดังกล่าวเป็นหลักฐานว่าต้นแบบมีเฟรมเรตหรือความพึงพอใจสูงขึ้น

2.10.2 Implementation of Inventory Checking System Using Finite State Machine in Role Playing Game

Kusuma และ Handayani (2024) ใช้ FSM ร่วมกับระบบตรวจสอบไอเท็มและภารกิจใน RPG Maker MV โดยพัฒนาตามวงจร GDLC
แนวคิดที่นำมาประยุกต์คือการตรวจเงื่อนไขไอเท็มก่อนเปลี่ยนสถานะภารกิจ
ซึ่งสัมพันธ์กับการตรวจจำนวนกุญแจก่อนเปลี่ยน ExitDoor จาก Locked เป็น Ready
แม้เกมเอนจินและประเภทเกมต่างกัน

2.10.3 An Analysis of Finite State Machine Based Enemy Artificial Intelligence in Kirby Nightmare in Dream Land

Thyrun (2026)
วิเคราะห์แบบจำลองพฤติกรรมศัตรูและผลของเวลาคูลดาวน์ต่อความยากด้วยการสังเกตและการจำลอง
บทความเป็น preprint ที่ยังไม่ผ่านการประเมินโดยผู้ทรงคุณวุฒิ
ใช้ประกอบแนวคิดการแบ่งสถานะโจมตีและพักหลังโจมตี ไม่ใช้เป็นผลทดสอบของ Killer ในโครงงานนี้

2.10.4 Unity Animator Controller

เอกสาร Unity Technologies อธิบายการใช้ Animator Controller เป็นแผนผังสถานะและ Transition เพื่อเลือกคลิปตามพารามิเตอร์ ใช้เป็นแนวทางสร้าง Idle และ Walk ของผู้เล่น และ Idle, Walk, Run, Attack1 และ Scream ของ Killer โดยแยกการแสดงภาพออกจากตรรกะ FSM ในสคริปต์

2.10.5 Unity Animation Parameters

เอกสาร Unity Technologies แบ่งพารามิเตอร์เป็น Float, Integer, Bool และ Trigger การเลือกใช้ขึ้นอยู่กับลักษณะข้อมูล โดย Bool เหมาะกับเงื่อนไขที่คงค่า เช่น กำลังเดิน ส่วน Trigger เหมาะกับเหตุการณ์เริ่มการกระทำ เช่น โจมตี Float และ Int
เป็นทางเลือกสำหรับค่าต่อเนื่องและรหัสสถานะ ซึ่งต้นแบบยังไม่ได้เพิ่มใน Animator

2.10.6 Unity Physics2D Raycast

เอกสาร Physics2D.Raycast
อธิบายการส่งรังสีจากตำแหน่งต้นทางไปตามทิศทางและระยะที่กำหนด พร้อมกรองเลเยอร์ ใช้ประกอบการตรวจสิ่งกีดขวางในแนวสายตาของ Killer การตรวจระยะและมุมเกิดก่อนส่ง Raycast ตามโค้ดของโครงงาน ไม่ใช้ Raycast เป็นหลักฐานว่าไม่มีข้อผิดพลาดในการตรวจจับทุกตำแหน่ง

2.10.7 Improving Mobile Game Performance with Basic Optimization Techniques in Unity

Koulaxidis และ Xinogalos (2022) ศึกษาการปรับประสิทธิภาพเกม Unity บนอุปกรณ์มือถือโดยเปรียบเทียบรุ่นที่ใช้เทคนิคปรับทรัพยากรและการจัดการวัตถุ
ข้อที่นำมาใช้คือควรวัดประสิทธิภาพบนอุปกรณ์จริงก่อนสรุปผล
โครงงานนี้ยังไม่ได้เก็บเฟรมเรตอย่างเป็นระบบ
และไม่ย้ายผลตัวเลขของเกมในบทความมาใช้เป็นผลของต้นแบบ

บทที่ 3 วิธีการดำเนินงาน

การพัฒนาเกมคอมพิวเตอร์ 2 มิติแนว Side-scrolling Hide-and-Seek Horror Game ในโครงงานนี้ดำเนินการด้วย Unity Game Engine และภาษา C# โดยประยุกต์ใช้เทคนิค Finite State Machine หรือ FSM ในการควบคุมสถานะของตัวละครผู้เล่นและพฤติกรรมของศัตรู ระบบถูกแบ่งเป็นโมดูลเพื่อให้ส่วนควบคุมผู้เล่น ระบบศัตรู ระบบไอเท็ม ระบบฉาก ระบบเสียง และส่วนติดต่อผู้ใช้งานทำงานร่วมกันได้อย่างเป็นระบบและสามารถปรับปรุงต่อได้

เกมประกอบด้วย 3 ด่านซึ่งมีระดับความยากและค่าพารามิเตอร์แตกต่างกัน ผู้เล่นต้องสำรวจพื้นที่ เก็บกุญแจให้ครบ หลีกเลี่ยงศัตรู และเดินทางไปยังประตูทางออก โดยสามารถใช้ไฟฉาย จุดซ่อนตัว และก้อนหินสร้างเสียงล่อศัตรูเพื่อช่วยในการหลบหนี ขั้นตอนการดำเนินงานประกอบด้วยการกำหนดปัญหา การรวบรวมข้อมูล การวิเคราะห์ระบบ การออกแบบระบบ การพัฒนา การทดสอบ และการบำรุงรักษา

3.1 การกำหนดปัญหา

3.1.1 ปัญหาด้านการควบคุมตัวละครผู้เล่น

ตัวละครผู้เล่นต้องรองรับการเดิน การวิ่ง การกระโดด การไต่บันได การซ่อนตัว การใช้ไอเท็ม การได้รับความเสียหาย และการเปลี่ยนฉาก การทำงานเหล่านี้อาจเกิดขึ้นต่อเนื่องหรือขัดแย้งกัน หากไม่มีระบบควบคุมสถานะที่ชัดเจนอาจทำให้ตัวละครรับคำสั่งในช่วงที่ไม่ควรควบคุมได้ โครงงานจึงใช้ FSM กำหนดสถานะ Normal, Hidden, Frozen, Hurt, UsingItem, Transitioning และ Dead เพื่อควบคุมสิทธิ์การรับคำสั่งและพฤติกรรมในแต่ละช่วง

3.1.2 ปัญหาด้านพฤติกรรมของศัตรู

ศัตรูต้องตอบสนองต่อผู้เล่นได้หลายลักษณะ ทั้งการลาดตระเวน การหันหาเสียง การตรวจสอบพื้นที่ การค้นหา การไล่ล่า และการโจมตี หากไม่มีลำดับสถานะที่ชัดเจน ศัตรูอาจเปลี่ยนพฤติกรรมอย่างไม่สมเหตุสมผลหรือไม่สามารถกลับไปลาดตระเวนได้หลังจากสูญเสียเป้าหมาย จึงใช้ Enemy FSM ควบคุมลำดับการตัดสินใจจากข้อมูลการมองเห็น การได้ยิน ระดับความสงสัย และตำแหน่งล่าสุดของผู้เล่น

3.1.3 ปัญหาด้านระบบตรวจจับ

ระบบตรวจจับต้องพิจารณาระยะทาง มุมมอง สิ่งกีดขวาง การเปิดไฟฉาย และเสียงที่เกิดจากกิจกรรมต่าง ๆ ของผู้เล่น การวิ่งมีความสำคัญมากกว่าการเดิน ขณะที่ก้อนหินและกับดักกระจกสร้างเสียงจากตำแหน่งอื่นเพื่อเบี่ยงเบนความสนใจของศัตรู ระบบจึงจำแนกประเภทเสียงเป็น Footstep, Sprint, Rock, Glass และ Environmental ก่อนประเมินความดัง ระยะห่าง และอายุของเสียง

3.1.4 ปัญหาด้านตรรกะและเป้าหมายของเกม

ผู้เล่นต้องเก็บกุญแจตามจำนวนที่กำหนดก่อนเปิดประตูทางออก ระบบต้องตรวจสอบจำนวนกุญแจ สถานะประตู ระยะการโต้ตอบ และอันตรายรอบตัวผู้เล่น การปลดล็อกต้องกดปุ่มโต้ตอบค้างไว้และยกเลิกได้เมื่อผู้เล่นปล่อยปุ่ม เดินออกจากระยะ ถูกโจมตี หรือมีศัตรูอยู่ใกล้ เพื่อให้เป้าหมายของเกมชัดเจนและมีความท้าทาย

3.1.5 ปัญหาด้านส่วนติดต่อผู้ใช้งาน

ผู้เล่นต้องรับรู้พลังชีวิต แบตเตอรี่ไฟฉาย พลังงานสำหรับวิ่ง จำนวนก้อนหิน จำนวนกุญแจ ข้อความโต้ตอบ และสถานการณ์อันตรายแบบเรียลไทม์ โดยข้อมูลต้องไม่บดบังพื้นที่เล่น ระบบจึงแสดง HIDDEN เมื่อซ่อนตัว แสดง DANGER เมื่อถูกไล่ล่า และใช้เสียงหัวใจร่วมกับขอบจอสีแดงเมื่อศัตรูอยู่ใกล้

3.1.6 ปัญหาด้านการออกแบบและปรับสมดุลด่าน

แต่ละด่านต้องมีบรรยากาศและระดับความยากต่างกัน ค่าเกี่ยวกับผู้เล่น ศัตรู และไฟฉายจึงต้องปรับได้โดยไม่แก้โค้ดหลัก โครงงานใช้ LevelConfig แยกค่าของแต่ละด่าน และออกแบบ Level 3 เป็นสวนสนุกและพื้นที่ป่ามืดที่มีหมอก ต้นไม้หลายชั้น ทางเดินต่างระดับ กับดักเสียง และตำแหน่งกุญแจที่เปลี่ยนแปลงได้

3.2 การรวบรวมข้อมูล

3.2.1 ข้อมูลด้านการพัฒนาเกม

ผู้พัฒนาได้ศึกษาการพัฒนาเกม 2 มิติด้วย Unity ระบบ Sprite และ Tilemap ระบบ Physics 2D การรับข้อมูลด้วย Unity Input System การจัดการฉาก การสร้าง UI และการใช้ ScriptableObject สำหรับจัดเก็บค่าตั้งต้นของด่าน นอกจากนี้ยังศึกษาการเขียนโปรแกรมเชิงวัตถุเพื่อแยกความรับผิดชอบของแต่ละระบบให้สามารถพัฒนาและทดสอบได้ง่าย

3.2.2 ข้อมูลด้าน Finite State Machine

ศึกษา FSM เพื่อแบ่งสถานะและเงื่อนไขของผู้เล่น ศัตรู ประตูทางออก และระดับความตึงเครียด ส่วน InventoryManager จัดการข้อมูลไอเท็มและเหตุการณ์แจ้ง UI ระบบเสียงและคำเตือนตอบสนองต่อข้อมูลสถานะและระยะห่าง

3.2.3 ข้อมูลด้านเกมแนวซ่อนตัวและสยองขวัญ

ศึกษากลไกการมองเห็นและการได้ยินของศัตรู การใช้เสียงเป็นความเสี่ยงและเครื่องมือล่อ การออกแบบจุดซ่อน การจำกัดทรัพยากรไฟฉาย และการใช้เสียงดนตรี เสียงหัวใจ หมอก แสง และฉากหลังหลายชั้นเพื่อสร้างความกดดันแก่ผู้เล่น

3.3 การวิเคราะห์ระบบ

3.3.1 การวิเคราะห์ระบบควบคุมผู้เล่น

PlayerController รับคำสั่งจาก Unity Input System และควบคุมการเคลื่อนที่แนวนอน การวิ่ง การใช้ Stamina การกระโดด การไต่บันได การโต้ตอบ การเปิดไฟฉาย การซ่อนตัว และการหยุดเกม การวิ่งใช้พลังงานซึ่งลดลงระหว่างใช้งานและฟื้นกลับหลังจากหยุดพัก ส่วนระบบกระโดดมีการตรวจสอบพื้นและ Coyote Time เพื่อช่วยให้การควบคุมตอบสนองเป็นธรรมชาติ

ตารางที่ 3-1 คำสั่งควบคุมเกม

คำสั่ง	แป้นพิมพ์	ผลการทำงาน
เคลื่อนที่	W A S D หรือปุ่มลูกศร	เดินและกำหนดทิศทาง
วิ่ง	Left Shift	เพิ่มความเร็ว ใช้ Stamina และสร้างเสียงมากขึ้น
กระโดด	Space	กระโดดเมื่ออยู่บนพื้นหรือในช่วง Coyote Time
โต้ตอบ	E	โต้ตอบกับวัตถุ หรือกดค้างเพื่อปลดล็อกประตู; กุญแจเก็บเมื่อสัมผัส
ไฟฉาย	F	เปิดหรือปิดไฟฉายและใช้พลังงานแบตเตอรี่
ซ่อนตัว	H	เข้าและออกจากจุดซ่อน
คลังไอเท็ม	C	เปิดหรือปิดหน้าต่างคลังไอเท็ม
ก้อนหิน	Q	ขว้างก้อนหินเพื่อสร้างเสียงล่อศัตรู
หยุดเกม	Escape	เปิดหรือปิด Pause Menu

3.3.2 การวิเคราะห์ Player FSM

ตารางที่ 3-2 สถานะของ Player FSM

สถานะ	หน้าที่
Normal	สถานะปกติที่รับคำสั่งเดิน วิ่ง กระโดด และโต้ตอบ
Hidden	ผู้เล่นอยู่ในจุดซ่อนและถูกจำกัดการเคลื่อนไหว
Frozen	หยุดการเคลื่อนไหวชั่วคราวและต้องทำตามเงื่อนไขเพื่อหลุดออก
Hurt	สถานะตอบสนองเมื่อได้รับความเสียหาย
UsingItem	ล็อกคำสั่งบางส่วนระหว่างเปิดคลังหรือใช้ไอเท็ม

สถานะ	หน้าที่
Transitioning	ล็อกการควบคุมระหว่างการเปลี่ยนด่าน
Dead	หยุดการควบคุมเมื่อพลังชีวิตหมด

3.3.3 การวิเคราะห์ระบบปัญญาประดิษฐ์ของศัตรู

KillerAI ประมวลผลแนวสายตาด้วย Raycasting และประมวลผลเสียงจากคิวเหตุการณ์ โดยให้ค่าน้ำหนักตามประเภทเสียง ความดัง ระยะห่าง และเวลาที่เสียงเกิดขึ้น ศัตรูบันทึกตำแหน่งล่าสุดของผู้เล่นหรือเสียงไว้ชั่วคราว เพื่อนำไปใช้ในสถานะตรวจสอบและค้นหา

การเปิดไฟฉายเพิ่มระยะที่ศัตรูสามารถตรวจพบผู้เล่นด้วยสายตา ส่วนผู้เล่นที่อยู่ในจุดซ่อนจะไม่ถูกตรวจจับตามเงื่อนไขปกติ

ตารางที่ 3-3 สถานะของ Enemy FSM

สถานะ	พฤติกรรม
Patrol	เดินลาดตระเวนตามจุดที่กำหนด
Alert	หันไปยังทิศทางของเหตุการณ์ผิดปกติ
Listening	หยุดเพื่อพิจารณาทิศทางและความสำคัญของเสียง
InvestigateSound	เคลื่อนที่ไปยังต้นกำเนิดเสียง
ConfirmSound	ตรวจสอบบริเวณต้นกำเนิดเสียง
Confused	ลังเลหลังพบการใช้ก้อนหินล่อซ้ำบริเวณเดิม
Investigate	ไปยังตำแหน่งล่าสุดของผู้เล่น
Search	ค้นหาบริเวณรอบตำแหน่งล่าสุด
Chase	ไล่ล่าผู้เล่นที่ถูกตรวจพบ
Attack	โจมตีผู้เล่นเมื่ออยู่ในระยะ
AttackRecover	หยุดชั่วคราวหลังการโจมตี
ReturnToPatrol	กลับไปยังเส้นทางลาดตระเวน
Scream	กรีดร้องเพื่อทำให้ผู้เล่น Stun หรือ Frozen ตามการตั้งค่า

3.3.4 การวิเคราะห์ระบบไอเท็มและภารกิจ

InventoryManager ใช้โครงสร้าง List เก็บไอเท็มที่ผู้เล่นรวบรวมและแยกกุญแจออกจากไอเท็มทั่วไป ระบบส่งเหตุการณ์ไปยัง UI เมื่อจำนวนกุญแจเปลี่ยนแปลง และตรวจสอบว่าผู้เล่นมีกุญแจครบตามค่า RequiredKeyCount หรือไม่ คลังไอเท็มเปิดด้วยปุ่ม C แสดงไอเท็มตามช่องที่กำหนด และอนุญาตให้ใช้ไอเท็มที่รองรับการใช้งาน

3.3.5 การวิเคราะห์ข้อมูลระดับความยาก

ค่าของด่านถูกจัดเก็บใน LevelConfig ซึ่งเป็น ScriptableObject ทำให้ปรับสมดุลเกมได้โดยไม่เปลี่ยนตรรกะหลัก ตารางต่อไปแสดงค่าหลักที่ใช้ในเกมเวอร์ชันปัจจุบัน

ตารางที่ 3-4 พารามิเตอร์ของแต่ละด่าน

พารามิเตอร์	Level 1	Level 2	Level 3
ระดับความยาก	Easy	Normal	Nightmare
กุญแจที่ต้องใช้	5	5	5
พลังชีวิตผู้เล่น	100	100	100
ความเร็วเดิน	4.5	4.5	4.5
ความเร็ววิ่ง	5.7	5.7	5.7
แบตเตอรี่ไฟฉาย	80	60	45
ความเร็วลาดตระเวน	2.0	1.3	2.8
ความเร็วไล่ล่า	4.0	3.0	7.2
ระยะมองเห็น	6.0	4.55	0
ระยะได้ยิน	4.0	3.25	10.0
พลังโจมตี	25	50	35

Level 3 ใช้ชื่อ Echoes of the Blind และกำหนดระยะมองเห็นของศัตรูเป็นศูนย์ แต่เพิ่มระยะได้ยินเป็น 10 หน่วย จึงเน้นการหลบเลี่ยงด้วยการควบคุมเสียงมากกว่าการหลบแนวสายตา

3.4 การออกแบบระบบ

3.4.1 การออกแบบข้อมูลนำเข้า

ข้อมูลนำเข้าประกอบด้วยคำสั่งจากแป้นพิมพ์ ตำแหน่งและทิศทางของผู้เล่น สถานะของ Player FSM ระยะห่างจากศัตรู สถานะไฟฉาย เหตุการณ์เสียง จำนวนกุญแจ ไอเท็มที่เก็บได้ และข้อมูล LevelConfig ของด่านปัจจุบัน ข้อมูลเหล่านี้ถูกส่งไปยังโมดูลที่รับผิดชอบเพื่อประมวลผลตามเงื่อนไข

3.4.2 การออกแบบกระบวนการทำงาน

1) GameManager โหลด LevelConfig และฉากของด่านปัจจุบัน

2) PlayerController รับคำสั่งและตรวจสอบ PlayerState ก่อนดำเนินการ

3) ระบบคำนวณการเคลื่อนที่ Stamina การกระโดด การไต่บันได และการโต้ตอบ

4) FootstepSystem หรือวัตถุในฉากส่งเหตุการณ์เสียงไปยัง KillerAI

5) KillerAI ประเมินการมองเห็น เสียง และระดับความสงสัยก่อนเปลี่ยนสถานะ

6) InventoryManager ตรวจสอบไอเท็มและจำนวนกุญแจ

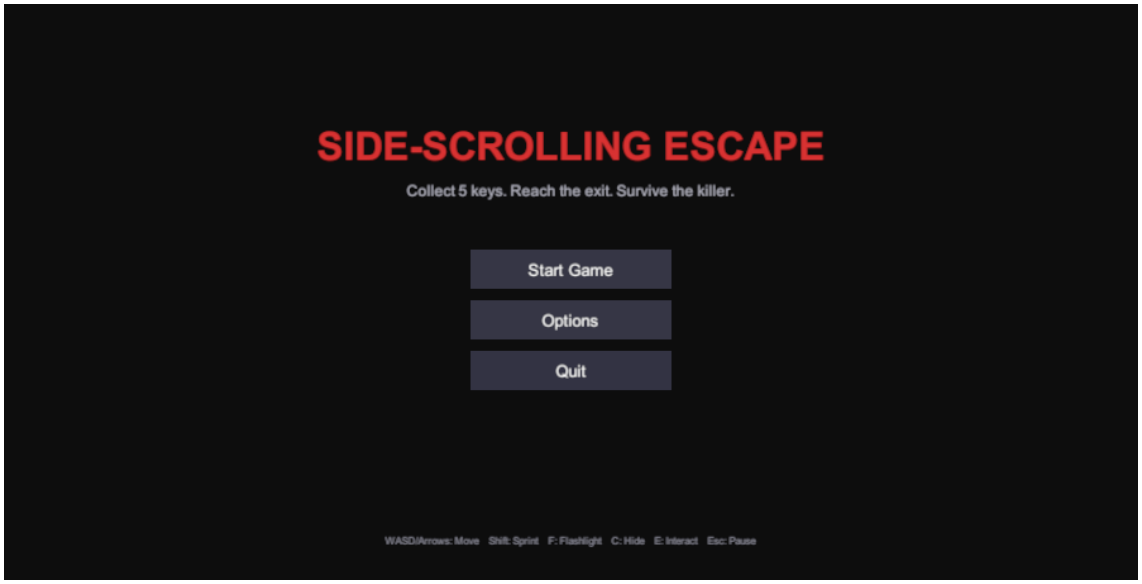
7) ExitDoor ตรวจสอบเงื่อนไขและความคืบหน้าการปลดล็อก

8) UIManager และระบบเสียงอัปเดตผลลัพธ์ให้ผู้เล่นรับรู้

9) GameManager เปลี่ยนด่านหรือแสดงผลการจบเกมเมื่อเงื่อนไขสำเร็จ

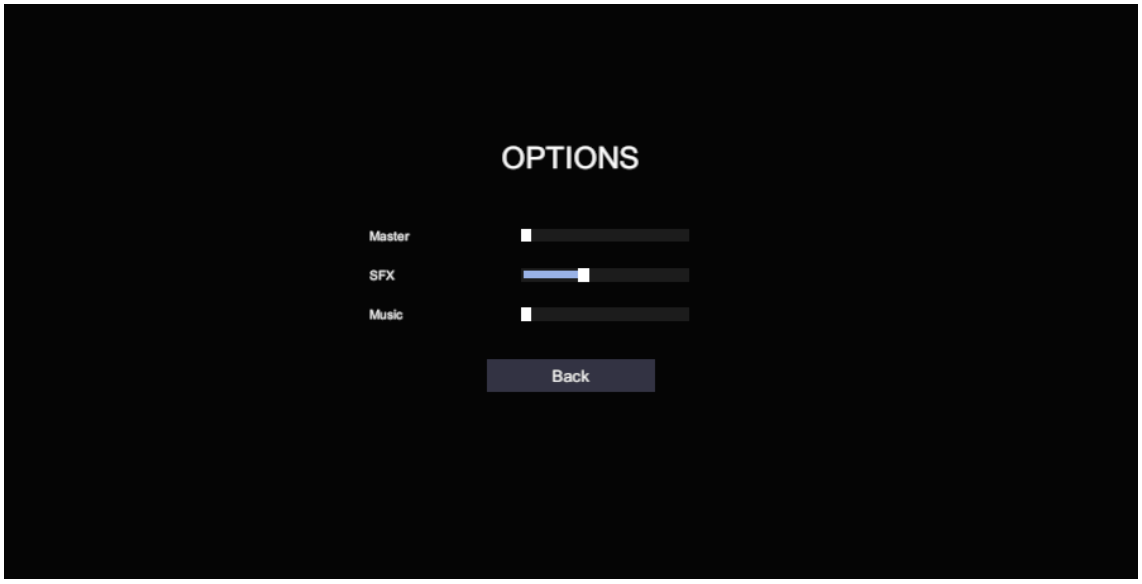
3.4.3 การออกแบบเอาต์พุต

หน้าจอเมนูหลักประกอบด้วยปุ่ม Start สำหรับเริ่มเกม Options สำหรับเปิดหน้าตั้งค่า และ Quit สำหรับออกจากเกม โดยใช้โทนสีเข้มให้สอดคล้องกับบรรยากาศสยองขวัญ



ภาพที่ 3-1 หน้าจอเมนูหลัก (ภาพสาธิตใน Play Mode)

หน้าจอตั้งค่าประกอบด้วย Slider สำหรับ Master Volume, SFX Volume และ Music Volume โดยค่าที่ปรับถูกเก็บด้วย PlayerPrefs เพื่อเรียกใช้ในการเปิดเกมครั้งต่อไป



ภาพที่ 3-2 หน้าจอตั้งค่าเสียง (ภาพสาธิตใน Play Mode)

HUD แสดงชื่อด่าน ระดับความยาก จำนวนกุญแจ พลังชีวิต แบตเตอรี่ไฟฉาย Stamina จำนวนก้อนหิน และข้อความโต้ตอบ รวมทั้ง HIDDEN และ DANGER ตามสถานการณ์



ภาพที่ 3-3 หน้าจอระหว่างเล่น (ภาพสาธิตใน Play Mode)

เมื่อกด Left Shift ระหว่างเคลื่อนที่ ผู้เล่นจะวิ่งเร็วขึ้นและ Stamina ลดลง แถบจะเปลี่ยนสีเมื่อเหลือพลังงานต่ำและฟื้นกลับหลังหยุดใช้



ภาพที่ 3-4 การวิ่งและระบบ Stamina (ภาพสาธิตใน Play Mode)

เมื่อเข้าใกล้วัตถุที่โต้ตอบได้ ระบบแสดง Prompt และใช้ปุ่ม E เพื่อสั่งงาน เช่น ประตูทางออก ส่วน ItemPickup เก็บกุญแจเมื่อผู้เล่นสัมผัส Trigger ของวัตถุ



ภาพที่ 3-5 สาธิตตำแหน่งกุญแจและระบบเก็บไอเท็ม

ผู้เล่นกด *C* เพื่อเปิดและปิดคลังไอเท็มหน้าต่างแสดงช่องไอเท็มที่เก็บได้และรองรับการเลือกใช้ไอเท็มบางประเภท



ภาพที่ 3-6 หน้าจอคลังไอเท็ม (ภาพสาธิตใน Play Mode)

เมื่อผู้เล่นอยู่ใกล้จุดซ่อนและกด H ระบบเปลี่ยนสถานะเป็น Hidden พร้อมแสดง HIDDEN บนหน้าจอ



ภาพที่ 3-7 ระบบซ่อนตัว (ภาพสาธิตใน Play Mode)

ผู้เล่นกด F เพื่อเปิดหรือปิดไฟฉาย แบตเตอรี่ลดลงระหว่างใช้งาน และการเปิดไฟฉายเพิ่มระยะตรวจพบด้วยสายตาของศัตรู



ภาพที่ 3-8 ระบบไฟฉายและแบตเตอรี่ (ภาพสาธิตใน Play Mode)

ก้อนหินใช้สร้างเหตุการณ์เสียงประเภท Rock จากตำแหน่งอื่นเพื่อเบี่ยงเบนศัตรู ภาพสาธิตแสดงการส่งเหตุการณ์ Rock ผ่าน HearNoise แล้ว Killer เข้าสู่ Listening ยังไม่ได้ยืนยันลำดับการขว้าง การตก และการเดินไปตรวจสอบครบวงจร



ภาพที่ 3-9 สาธิต Killer รับเหตุการณ์เสียงประเภท Rock

เมื่อศัตรูตรวจพบผู้เล่น ระบบเปลี่ยนเข้าสู่ Chase HUD แสดง DANGER และระบบเสียงปรับเข้าสู่บรรยากาศการไล่ล่า



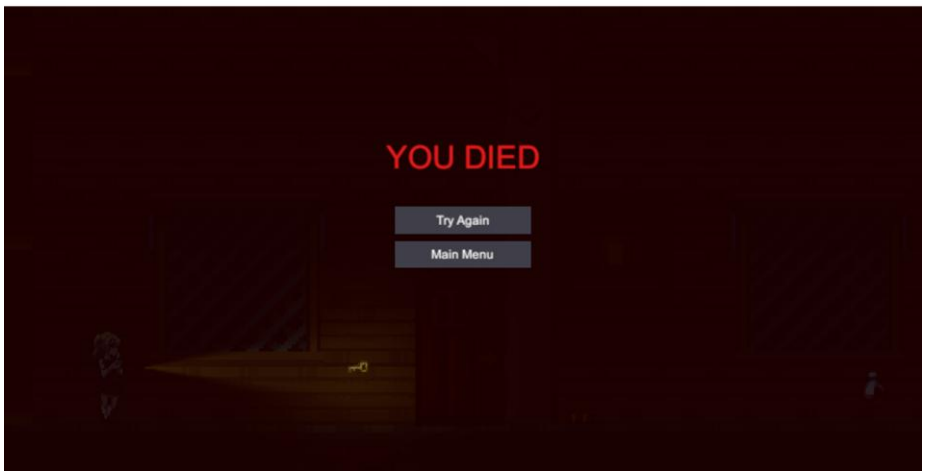
ภาพที่ 3-10 การตรวจพบและไล่ล่าผู้เล่น (ภาพสาธิตใน Play Mode)

เมื่อศัตรูอยู่ในระยะโจมตี ระบบลดพลังชีวิตของผู้เล่น หากพลังชีวิตเป็นศูนย์ Player FSM เปลี่ยนเป็น Dead และแสดงหน้าจอ Game Over

HURT



GAMEOVER



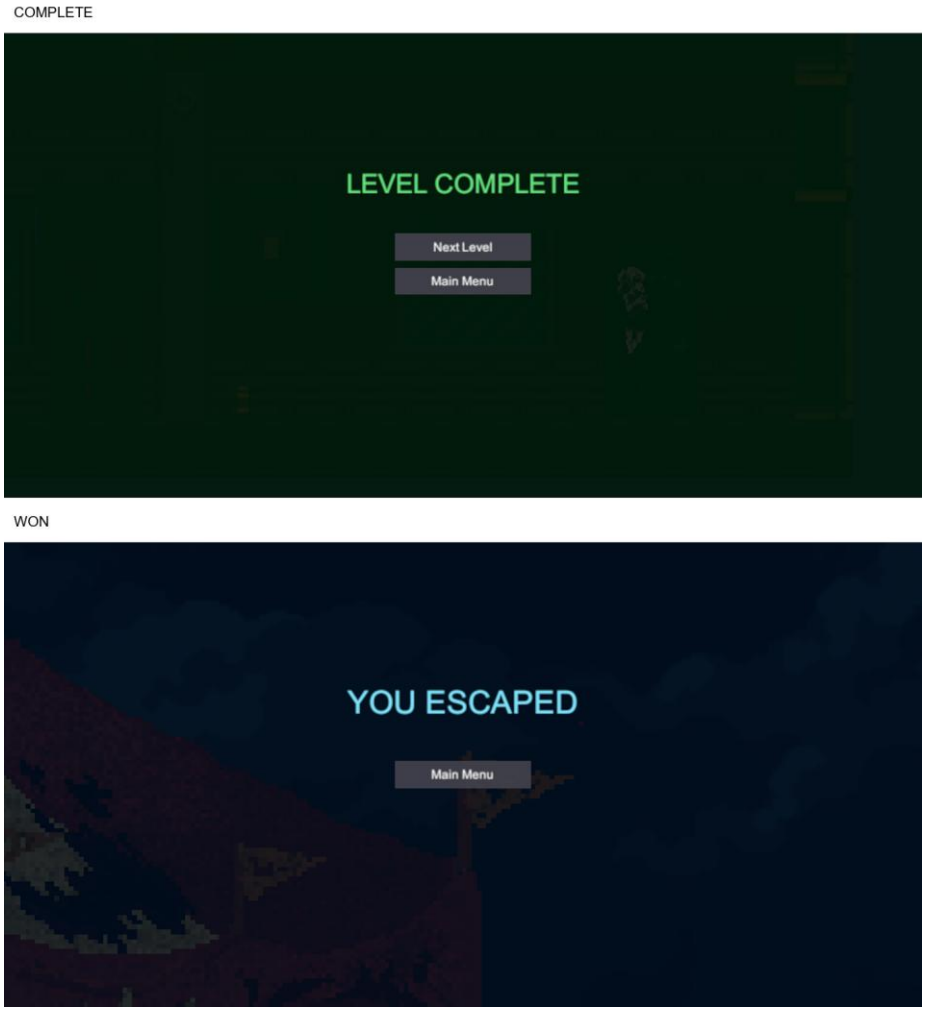
ภาพที่ 3-11 การได้รับความเสียหายและ Game Over (ภาพสาธิตใน Play Mode)

เมื่อกุญแจครบ ผู้เล่นกด E ค้างที่ประตูจนความคืบหน้าครบ การปลดล็อกยกเลิกเมื่อปล่อยปุ่ม ออกจากระยะ ถูกโจมตี หรือมีศัตรูอยู่ใกล้ ภาพนี้เรียกสถานะ Unlocking และกำหนดความคืบหน้าเพื่อแสดง Prompt ของระบบ



ภาพที่ 3-12 สาธิตข้อความและความคืบหน้าการปลดล็อกประตู

เมื่อปลดล็อกประตูสำเร็จ ระบบแสดง Level Complete และโหลดด่านถัดไป หากผ่านด่านสุดท้ายระบบแสดง Game Won



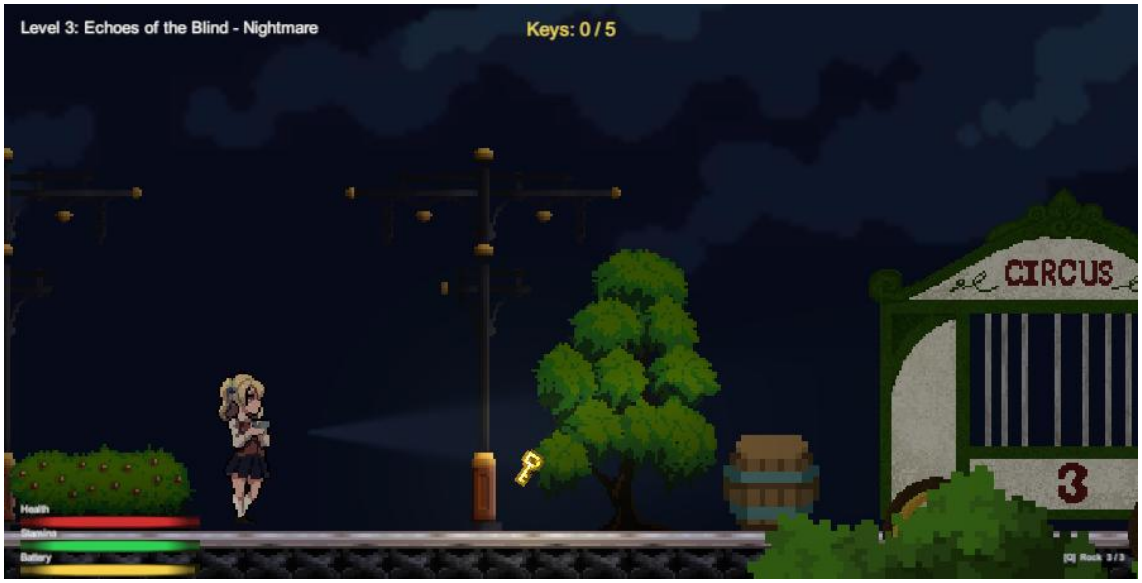
ภาพที่ 3-13 หน้าจอผ่านด่านและจบเกม (ภาพสาธิตใน Play Mode)

Level 2 มีสถานะคำสาป Safe, Exposed, Warning, Critical, Petrified และ Recovering โดย HUD แสดงแถบคำสาป ภาพบดบัง และข้อความเตือนตามระดับอันตราย



ภาพที่ 3-14 ระบบคำสาปใน Level 2 (ภาพสาธิตใน Play Mode)

Level 3 ออกแบบเป็นสวนสนุกและพื้นที่ป่ามืดโดยใช้หมอก ต้นสนระยะไกล ต้นไม้และพุ่มไม้ชั้นกลาง รวมถึงพื้นหญ้าและเส้นทางต่างระดับเพื่อสร้างความลึกของฉาก



ภาพที่ 3-15 สภาพแวดล้อมสวนสนุกและพื้นที่ป่าใน Level 3 (ภาพสาธิตใน Play Mode)

กับดักกระจกทำงานเมื่อผู้เล่นเดินชน โดยเล่นเสียงแตก เปลี่ยนลักษณะวัตถุ และส่งเสียงประเภท Glass ไปยัง KillerAI



ภาพที่ 3-16 สาธิตตำแหน่งกับดักเสียงใน Level 3

เมื่อศัตรูอยู่ในระยะเตือน เกมเล่นเสียงหัวใจและแสดงขอบจอสีแดง ความถี่และความเข้มเพิ่มขึ้นตามระดับอันตราย



ภาพที่ 3-17 ระบบแจ้งเตือนเมื่อศัตรูอยู่ใกล้ (ภาพสาธิตใน Play Mode)

3.5 การพัฒนาระบบ

3.5.1 สถาปัตยกรรมของระบบ

ระบบประกอบด้วย PlayerController, PlayerHealth, FlashlightController, HidingSystem, FootstepSystem, KillerAI, InventoryManager, ExitDoor, GameManager, UIManager, AudioManager, LevelAudioController และ Level3HeartbeatWarning โดย PlayerController รับข้อมูลจากผู้เล่นและส่งเหตุการณ์ให้ระบบที่เกี่ยวข้อง FootstepSystem ส่งเหตุการณ์เสียงให้ KillerAI ขณะที่ InventoryManager ส่งจำนวนกุญแจให้ UIManager และ ExitDoor ส่วน GameManager ควบคุมสถานะเกมและการเปลี่ยนฉาก

3.5.2 การพัฒนา Player FSM

Player FSM ใช้สถานะ Normal, Hidden, Frozen, Hurt, UsingItem, Transitioning และ Dead เพื่อควบคุมการรับคำสั่ง Normal เป็นสถานะที่อนุญาตการควบคุมทั่วไป Hidden ใช้เมื่ออยู่ในจุดซ่อน Frozen และ Hurt ใช้หยุดหรือจำกัดการควบคุมชั่วคราว UsingItem ป้องกันคำสั่งที่ขัดแย้งระหว่างใช้ไอเท็ม Transitioning ใช้ระหว่างเปลี่ยนฉาก และ Dead ใช้เมื่อพลังชีวิตหมด

3.5.3 การพัฒนา Enemy FSM

Enemy FSM รับข้อมูลจากระบบการมองเห็นและการได้ยิน จากนั้นประเมินความสงสัยและเลือกสถานะ เสียง Glass และ Sprint มีความสำคัญมากกว่าเสียง Footstep ทั่วไป ศัตรูเก็บตำแหน่งล่าสุดไว้ชั่วคราวเพื่อใช้ตรวจสอบและค้นหา หากไม่พบเป้าหมายจะกลับไปยังเส้นทางลาดตระเวน

3.5.4 การพัฒนาระบบความตึงเครียด

TensionDirector เป็นคลาสที่ประกาศในไฟล์ LevelAudioController.cs ใช้สถานะ Calm, Suspicious, Hunted, Escaped และ Cooldown
โดยรวมพฤติกรรมศัตรูทุกตัวและส่งเหตุการณ์ให้ระบบเสียง ระหว่าง Hunted ปรับตัวคูณความเร็วไล่ล่า ส่วน UIManager แสดง DANGER จากสถานะ Killer และ Level3HeartbeatWarning
ใช้ระยะห่างควบคุมเสียงหัวใจกับขอบจอสีแดง จึงต้องแยก FSM
ความตึงเครียดออกจากการแสดงคำเตือนโดยตรง

3.5.5 การพัฒนาระบบไอเท็มและประตูทางออก

InventoryManager เก็บไอเท็มใน List และส่งเหตุการณ์เมื่อจำนวนกุญแจเปลี่ยน ExitDoor มีสถานะ Locked, Ready, Unlocking, Open, Transitioning และ Completed การใช้ FSM กับประตูช่วยควบคุมการปลดล็อกแบบกดค้างและรองรับการยกเลิกเมื่อสถานการณ์ไม่ปลอดภัย

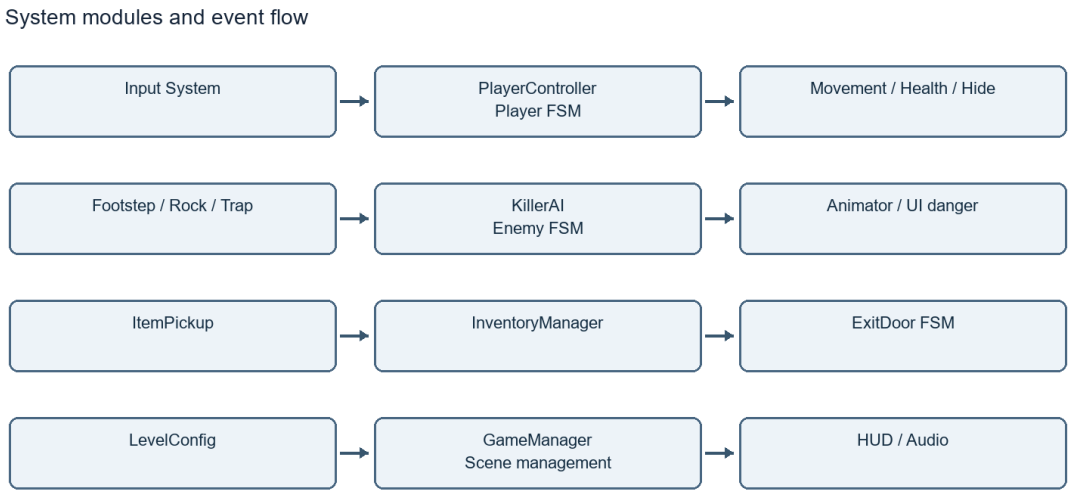
3.5.6 การพัฒนาด่าน Level 3

Level 3 ใช้ Tilemap และ Sprite ที่มีอยู่ในโครงการจัดวางเป็นสวนสนุกที่มีพื้นที่ป่า โดยแบ่งเป็นชั้นหมอกและต้นไม้ระยะไกล ชั้นต้นไม้และพุ่มไม้ระยะกลาง และชั้นพื้นสำหรับการเล่น ระบบ Level3RandomEvents
ค้นหาพื้นที่เดินได้และกระจายกุญแจกับกับดักระหว่างเส้นทางชั้นล่างและชั้นบน เพื่อลดการรวมตัวของวัตถุในตำแหน่งเดียว

3.5.7 การจัดการเงื่อนไขพิเศษ

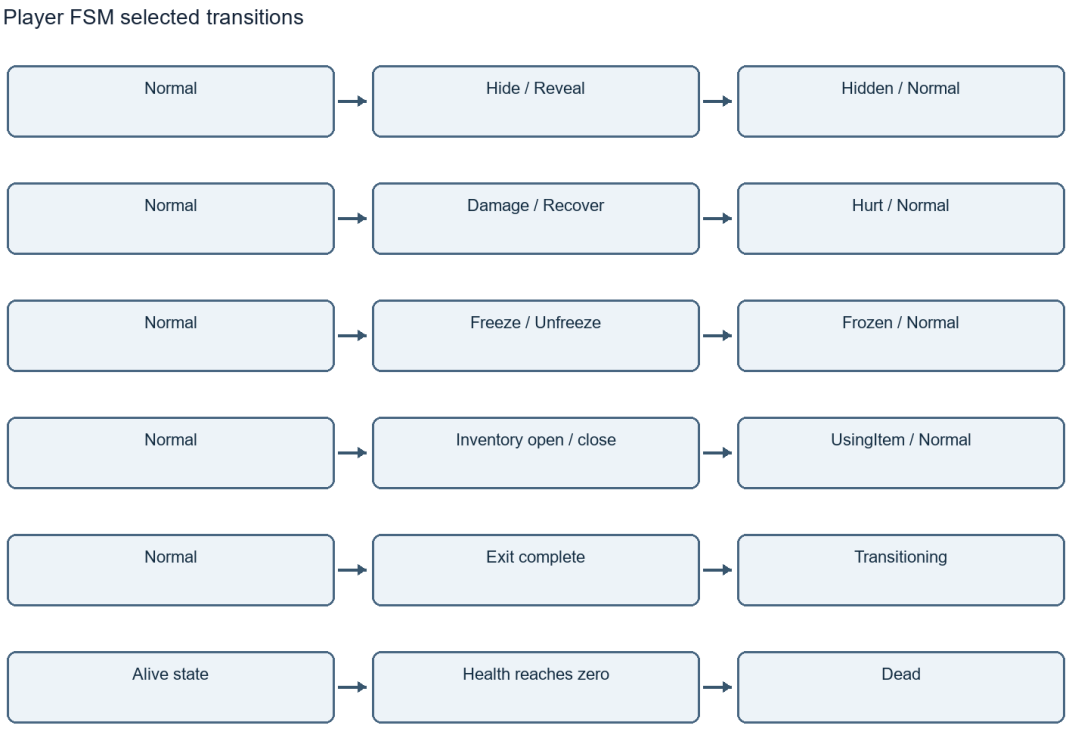
ระบบป้องกันการกระโดดซ้ำกลางอากาศ รองรับการกระโดดออกจากบันได ล็อกการเคลื่อนไหวขณะซ่อน ใช้ไอเท็ม เปลี่ยนฉาก หรือเสียชีวิต ลบเสียงที่หมดอายุจากหน่วยความจำของศัตรู ลดประสิทธิภาพการล่อด้วยก้อนหินซ้ำ และยกเลิกการปลดล็อกประตูเมื่อผู้เล่นอยู่ในอันตราย

แผนภาพแสดงการรับข้อมูลจาก Input System ผ่าน PlayerController การส่งเหตุการณ์ไปยังระบบไอเท็ม ระบบเสียง KillerAI UI และ GameManager



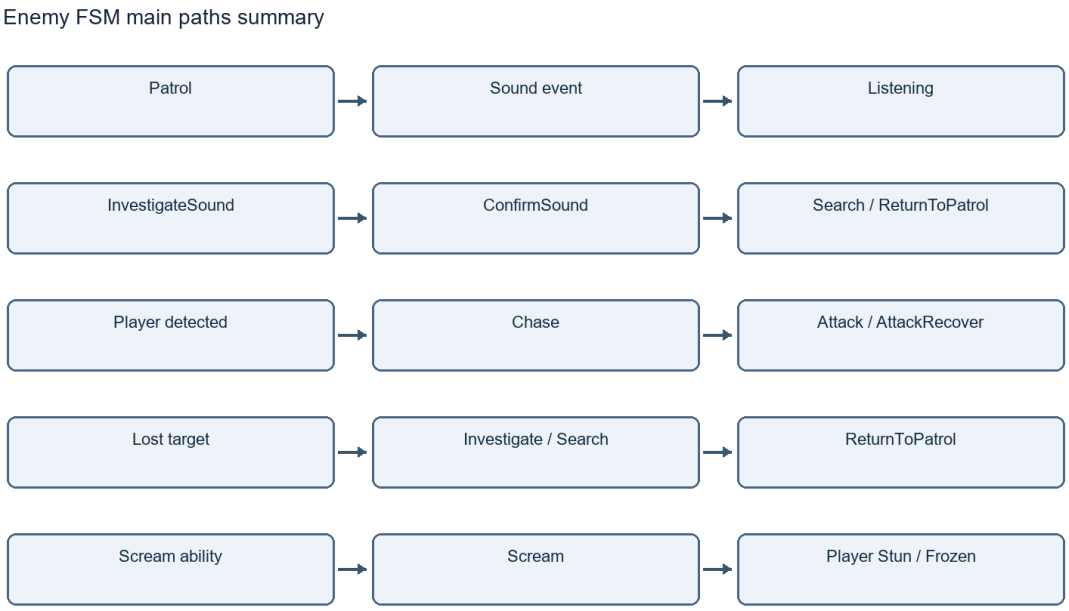
ภาพที่ 3-18 แผนภาพสถาปัตยกรรมของระบบ

แผนภาพแสดงสถานะของผู้เล่นและเหตุการณ์สำคัญที่ทำให้เปลี่ยนระหว่าง Normal, Hidden, Frozen, Hurt, UsingItem, Transitioning และ Dead แผนภาพย่อแสดงคู่เหตุการณ์หลัก ไม่ได้แสดง Transition ทุกเส้น



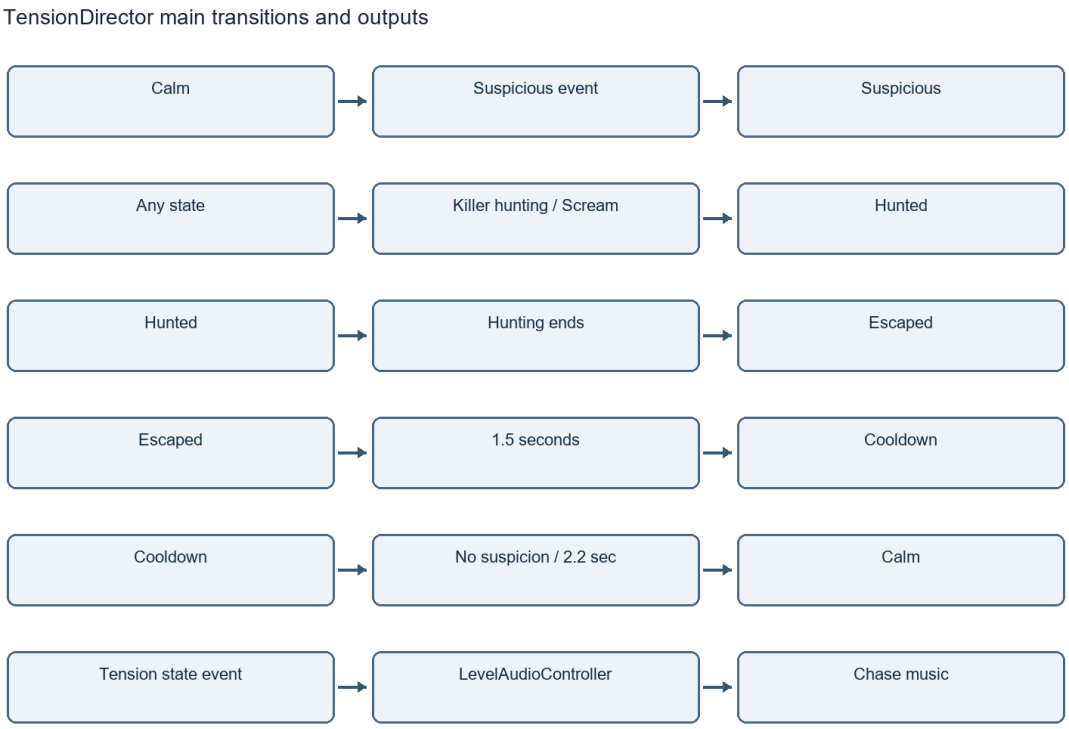
ภาพที่ 3-19 แผนภาพ Player FSM

แผนภาพแสดงกระบวนการตั้งแต่ Patrol การรับรู้เสียงหรือการมองเห็น การตรวจสอบ การค้นหา การไล่ล่า การโจมตี และการกลับไปลาดตระเวน



ภาพที่ 3-20 แผนภาพ Enemy FSM

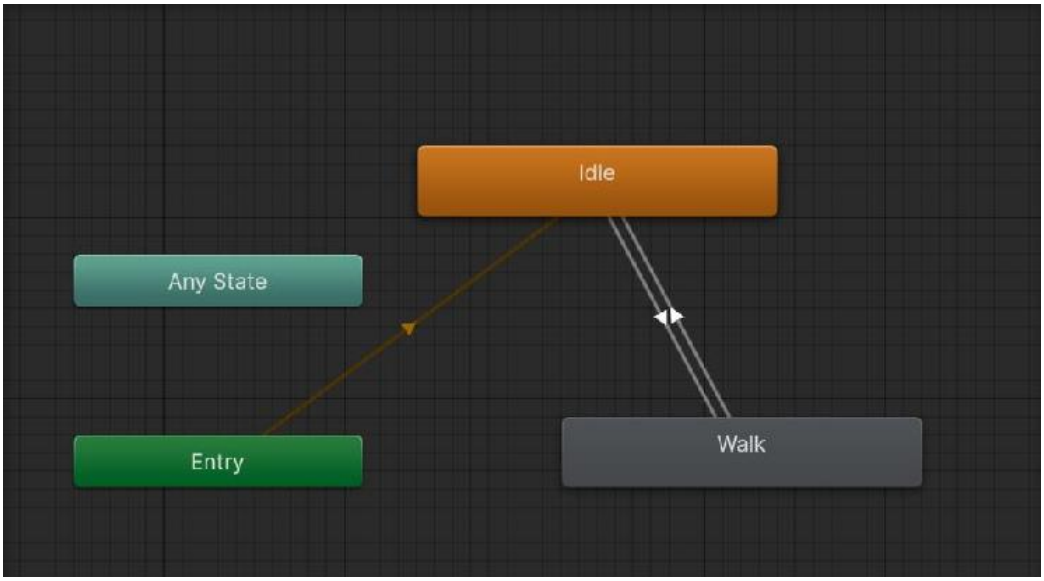
แผนภาพสรุปเส้นทางหลักของ TensionDirector และการส่งข้อมูลให้ระบบเสียง โดย Hunted เกิดเมื่อมีศัตรูล่า ผู้เล่นหลุดจากการล่าจะเข้าสู่ Escaped แล้ว Cooldown ก่อนกลับ Calm หากมีการล่าใหม่จะเข้าสู่ Hunted จากสถานะใดก็ได้



ภาพที่ 3-21 แผนภาพสถานะความตึงเครียดและการส่งข้อมูลเสียง

3.5.8 การพัฒนาระบบแอนิเมชันของตัวละครผู้เล่น

ระบบแอนิเมชันใช้ Animator Controller ของ Unity
เพื่อกำหนดสถานะภาพเคลื่อนไหวและเงื่อนไขเปลี่ยนสถานะ โดย Animation Clip
เก็บลำดับภาพของตัวละคร ส่วน Animator เลือกคลิปที่ต้องแสดงตามพารามิเตอร์ซึ่งได้รับจากสคริปต์
วิธีนี้ทำให้การแสดงผลสัมพันธ์กับการเคลื่อนที่ และตรวจสอบเส้นทางการเปลี่ยนแอนิเมชันได้จากแผนผัง



ภาพที่ 3-22 แผนผัง Animator ของตัวละครผู้เล่นจาก Unity

จากภาพที่ 3-22 PlayerAnimator.controller มีสถานะ Idle และ Walk โดย Idle
เป็นสถานะเริ่มต้น แสดงเป็นกล่องสีส้ม ลูกศรจาก Entry จึงเข้าสู่ Idle ก่อน ส่วน Walk
แสดงภาพการเดิน ลูกศรระหว่างสองสถานะเป็น Transition ที่ใช้เงื่อนไข isWalking ชนิด Bool
เพื่อเลือกแอนิเมชัน

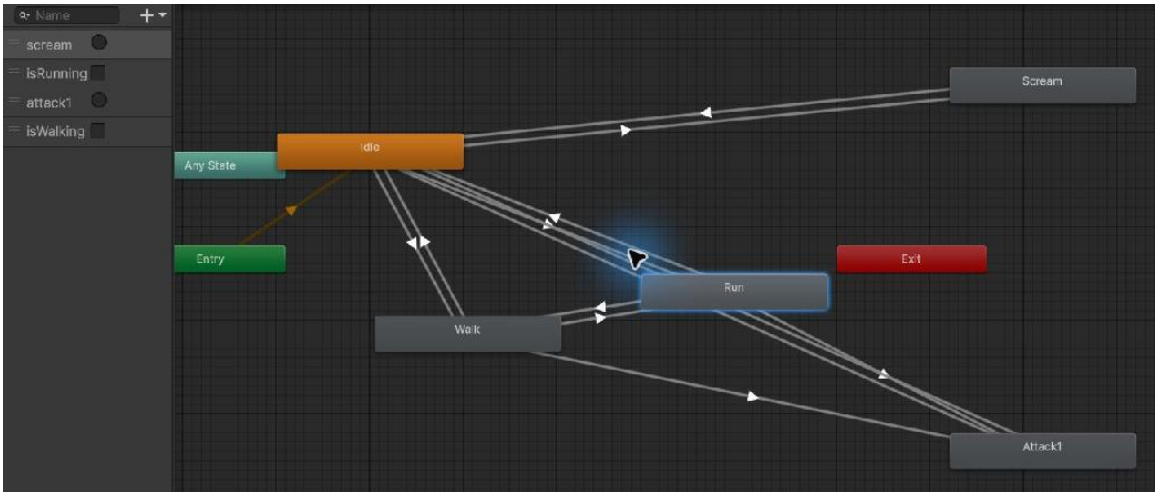
เมื่อ isWalking เป็น true ระบบเปลี่ยนจาก Idle ไป Walk และเมื่อเป็น false จะกลับจาก
Walk ไป Idle ทั้งสอง Transition ปิด Has Exit Time และมี Transition Duration เท่ากับ 0
จึงเปลี่ยนสถานะได้ทันทีเมื่อเงื่อนไขเป็นจริง โดยไม่ต้องรอให้คลิปเดิมเล่นถึงจุดสิ้นสุด

PlayerController ส่งค่าด้วย animator.SetBool(IsWalkingHash, IsMoving)
จึงใช้สถานะการเคลื่อนที่ของผู้เล่นเป็นตัวควบคุมภาพจริง ใน Controller ปัจจุบันยังไม่มีสถานะ Run
แยกต่างหาก การวิ่งจึงไม่ได้หมายความว่ามีคลิป Run ของผู้เล่นอยู่ในแผนผังนี้

Any State เป็นจุดสำหรับกำหนดการเปลี่ยนจากสถานะใดก็ได้ แต่ของผู้เล่นยังไม่มี Transition
ออกจากจุดนี้ ส่วน Entry เป็นจุดเริ่มต้น ไม่ใช่คลิปแอนิเมชัน
การแยกความหมายของแต่ละกล่องช่วยให้ตีความแผนผังได้ถูกต้อง

3.5.9 การพัฒนาระบบแอนิเมชันของ Killer

Killer ใช้ KillerAnimator.controller เชื่อมกับ KillerAI
เพื่อแสดงท่าทางตามพฤติกรรมของศัตรู ประกอบด้วย Idle สำหรับหยุดนิ่ง Walk สำหรับการเดิน Run สำหรับไล่ล่า Attack1 สำหรับโจมตี และ Scream สำหรับกรีดร้อง ภาพที่ 3-23 จับจากหน้าต่าง Animator ของ Unity โดยจัดตำแหน่งกล่องให้เห็นชื่อสถานะและลูกศรชัดเจน



ภาพที่ 3-23 แผนผัง Animator และ Parameters ของ Killer จาก Unity

Entry เริ่มที่ Idle ส่วนการเดินใช้ isWalking ชนิด Bool และการวิ่งใช้ isRunning ชนิด Bool สคริปต์กำหนด isWalking เป็น true เมื่ออยู่ใน Patrol, Investigate, InvestigateSound, Search หรือ ReturnToPatrol และมีค่าสัมบูรณ์ของความเร็วแนวนอนมากกว่า 0.01 ส่วน isRunning เป็น true เมื่ออยู่ใน Chase หรือ Attack

การโจมตีใช้ attack1 ชนิด Trigger โดย KillerAI เรียก SetTrigger เมื่อโจมตีตามรอบคูลดาวน์ หรือเริ่มตรวจค้นจุดซ่อนที่สงสัย จึงเรียกท่า Attack1 เป็นเหตุการณ์แต่ละครั้ง ขณะที่ scream เป็น Trigger ซึ่งเรียกเมื่อระบบสั่งกรีดร้องหรือใช้ความสามารถหยุดผู้เล่นตามเงื่อนไขของศัตรู

Any State เชื่อมไป Scream เพื่อให้การกรีดร้องเริ่มได้จากสถานะใดก็ได้ ส่วน Attack1 รับ Transition จาก Idle, Walk และ Run จึงไม่ได้ใช้ Any State สำหรับการโจมตี เมื่อ Attack1 หรือ Scream เล่นครบตาม Exit Time เท่ากับ 1 จะกลับ Idle
แล้วตรวจพารามิเตอร์การเคลื่อนที่เพื่อเลือกสถานะถัดไป

3.5.10 เงื่อนไขการเปลี่ยนแอนิเมชันและการเชื่อมกับ FSM

ลูกศรในภาพที่ 3-23 แสดงทิศทางที่ Animator เปลี่ยนสถานะได้ เงื่อนไขของ Transition ต้องสอดคล้องกับค่าที่ KillerAI ส่งเข้ามา โดย Controller ปัจจุบันกำหนดเส้นทางดังต่อไปนี้

Idle ไป Walk เมื่อ isWalking เป็น true และ Walk กลับ Idle เมื่อ isWalking เป็น false

Idle หรือ Walk ไป Run เมื่อ isRunning เป็น true และ Run กลับ Idle เมื่อ isRunning เป็น false

Run ไป Walk เมื่อ isRunning เป็น false และ isWalking เป็น true
โดยเงื่อนไขทั้งสองต้องเป็นจริงร่วมกัน

Idle, Walk หรือ Run ไป Attack1 เมื่อได้รับ Trigger attack1 และ Attack1 กลับ Idle ด้วย Has Exit Time ที่ค่า 1

Any State ไป Scream เมื่อได้รับ Trigger scream และ Scream กลับ Idle ด้วย Has Exit Time ที่ค่า 1

Transition ระหว่างการเคลื่อนที่ของ Killer มี Duration เท่ากับ 0.1 วินาที และปิด Has Exit Time เพื่อให้ตอบสนองต่อพฤติกรรมได้โดยไม่รอจบรอบคลิป การเข้าสู่ Attack1 ใช้ Duration 0.05 วินาที ส่วน Any State ไป Scream ใช้ Duration 0 เพื่อเริ่มกรีดร้องทันที การกลับจาก Attack1 และ Scream ใช้ Duration 0.1 วินาที

FSM ใน KillerAI รับหน้าที่ตัดสินใจ เช่น ลาดตระเวน ค้นหา ไล่ล่า หรือโจมตี ส่วน Animator รับหน้าที่แสดงภาพของการตัดสินใจนั้น จึงไม่จำเป็นต้องมีจำนวนสถานะเท่ากัน ตัวอย่างเช่น Patrol และ Search ใช้ภาพ Walk ร่วมกันได้ แต่ยังมีตรรกะเลือกเป้าหมายต่างกันใน FSM

สำหรับผู้เล่น หลักการเดียวกันใช้กับ Player FSM ซึ่งควบคุมการรับคำสั่งและข้อจำกัดในสถานะ Normal, Hidden, Frozen, Hurt, UsingItem, Transitioning และ Dead ส่วน PlayerAnimator ปัจจุบันเลือกภาพระหว่าง Idle กับ Walk ตาม IsMoving การเพิ่มสถานะใน FSM
จึงไม่ทำให้เกิดคลิปแอนิเมชันใหม่โดยอัตโนมัติ

การคำนวณความเสียหายอยู่ใน KillerAI ซึ่งเรียก playerHealth.TakeDamage ก่อนส่ง Trigger attack1 แอนิเมชันโจมตีจึงเป็นภาพประกอบเหตุการณ์ ไม่ใช่ตัวกำหนดความเสียหายด้วย Animation Event ในส่วนโค้ดนี้ หากต้องการให้ความเสียหายตรงกับเฟรมที่อาวุธสัมผัสผู้เล่น จึงค่อยออกแบบ Animation Event เพิ่มเติม

เมื่อ isRunning เป็น false และ isWalking เป็น true เส้นจาก Run ไป Idle กับเส้นไป Walk อาจผ่านเงื่อนไขพร้อมกัน จึงควรตรวจลำดับ Transition
เพื่อให้ผลการเปลี่ยนภาพสอดคล้องกับพฤติกรรมที่ต้องการ

3.5.11 เหตุผลในการเลือกชนิดพารามิเตอร์ของ Animator

Unity มีพารามิเตอร์ Float, Int, Bool และ Trigger เพื่อแทนข้อมูลต่างลักษณะ การเลือกชนิดขึ้นอยู่กับว่าเป็นค่าต่อเนื่อง รหัสสถานะ เงื่อนไขที่คงอยู่ หรือเหตุการณ์เฉพาะครั้ง ไม่จำเป็นต้องใช้ครบทั้งสี่ชนิดใน Controller เดียว โครงการนี้ใช้ Bool และ Trigger จริง ส่วน Float และ Int อธิบายเป็นทางเลือกสำหรับการขยายระบบ

Float สำหรับค่าต่อเนื่อง

Float เก็บตัวเลขที่มีทศนิยม เหมาะกับค่าความเร็ว เช่น 0, 1.5 และ 3.2 หากเพิ่มพารามิเตอร์ Speed จะส่งค่าด้วย SetFloat และใช้เปรียบเทียบกับเกณฑ์หรือควบคุม Blend Tree เพื่อผสมท่ายืน เดิน และวิ่งตามความเร็วได้ เหตุผลที่เลือก Float คือรักษารายละเอียดระหว่างค่าความเร็วซึ่ง Int จะเก็บไม่ได้ ปัจจุบันยังไม่มีพารามิเตอร์ Float ใน Animator ทั้งสองตัว

Int สำหรับรหัสหรือทางเลือกหลายค่า

Int เก็บจำนวนเต็ม เหมาะกับการแทนโหมดที่แยกกัน เช่น MovementMode เท่ากับ 0 สำหรับยืน 1 สำหรับเดิน และ 2 สำหรับวิ่ง แล้วส่งค่าด้วย SetInteger เหตุผลที่เลือก Int คือใช้ค่าเดียวระบุทางเลือกหลายแบบ ลดการใช้ Bool หลายตัวที่อาจเป็น true พร้อมกัน อย่างไรก็ตาม ตัวแปร int ที่เก็บผล Animator.StringToHash เป็นเพียงรหัสอ้างอิงชื่อพารามิเตอร์ ไม่ได้ทำให้พารามิเตอร์นั้นเป็นชนิด Int และ Controller ปัจจุบันยังไม่ได้ใช้ Int

Bool สำหรับเงื่อนไขที่ต้องคงสถานะ

Bool มีค่า true หรือ false เหมาะกับคำถามว่ากำลังเดินหรือกำลังวิ่งหรือไม่ จึงใช้ isWalking ในผู้เล่น และ isWalking กับ isRunning ใน Killer โดยส่งค่าด้วย SetBool เหตุผลที่เลือก Bool คือระบบต้องรักษาค่าขณะพฤติกรรมยังดำเนินอยู่ และเปลี่ยนกลับเมื่อพฤติกรรมหยุด ทำให้การเข้าและออกจาก Walk หรือ Run ตรวจสอบได้ตรงไปตรงมา

Trigger สำหรับเหตุการณ์เฉพาะครั้ง

Trigger เหมาะกับคำสั่งเริ่มการกระทำ เช่น โจมตีหรือกรีดร้อง จึงใช้ attack1 และ scream ผ่าน SetTrigger เมื่อ Transition ใช้ Trigger แล้ว Controller จะรีเซ็ตค่าโดยอัตโนมัติ หากยังไม่มี Transition ใช้ค่านั้น อาจต้องเรียก ResetTrigger เพื่อยกเลิกคำสั่งที่ค้างอยู่ เหตุผลที่เลือก Trigger คือไม่ต้องเปิด Bool ค้างไว้และคอยสั่งปิดเองทุกครั้ง ส่วนการออกจากคลิปยังต้องกำหนด Transition และ Exit Time ให้เหมาะสม

ตรวจสอบจาก PlayerAnimator.controller, KillerAnimator.controller, PlayerController.cs และ KillerAI.cs ในโครงการ SideScrollingEscape ส่วนชนิดพารามิเตอร์อ้างอิง Unity Manual เรื่อง Animation Parameters ตามบรรณานุกรม ตรวจสอบวันที่ 4 ตุลาคม 2569

3.6 การทดสอบระบบ

กำหนด Test Case ด้วยขั้นตอนและผลที่คาดหวัง ตารางบทนี้เป็นแผนทดสอบ สถานะ “กำหนดกรณีแล้ว” หมายถึงมีขั้นตอนพร้อมใช้ ยังไม่ใช่ผลผ่าน
การตรวจโค้ดและสาธิตแบบควบคุมสถานะใน Play Mode แสดงแยกไว้ในบท 4

3.6.1 การทดสอบระบบควบคุมพื้นฐาน

ตารางที่ 3-5 แผนทดสอบการควบคุมพื้นฐาน

รหัส	รายการทดสอบ	ขั้นตอน	ผลที่คาดหวัง	ผลจริง/สถานะ
TC-01	การเดิน	กด A D หรือปุ่มลูกศร	ผู้เล่นเคลื่อนที่และหันถูกทิศ	แผนทดสอบ
TC-02	การวิ่ง	กด Shift พร้อมเคลื่อนที่	ความเร็วเพิ่ม Stamina ลด และเกิดเสียง Sprint	แผนทดสอบ
TC-03	การกระโดด	กด Space บนพื้นและกลางอากาศ	กระโดดได้ครั้งเดียวและไม่กระโดดซ้ำกลางอากาศ	แผนทดสอบ
TC-04	การไต่บันได	กดขึ้นหรือลงเมื่ออยู่บนบันได	ผู้เล่นไต่บันไดและกระโดดออกได้	แผนทดสอบ
TC-05	ไฟฉาย	กด F และเปิดทิ้งไว้	ไฟฉายสลับสถานะและแบตเตอรี่ลดลง	แผนทดสอบ

3.6.2 การทดสอบ Player FSM

ตารางที่ 3-6 แผนทดสอบ Player FSM

รหัส	การเปลี่ยนสถานะ	เงื่อนไขทดสอบ	ผลที่คาดหวัง	ผลจริง/สถานะ
TC-FSM-P01	Normal > Hidden	เข้าใกล้จุดซ่อนและกด H	ผู้เล่นซ่อนและ HUD แสดง HIDDEN	แผนทดสอบ
TC-FSM-P02	Hidden > Normal	กด H เพื่อออกจากจุดซ่อน	กลับมาเคลื่อนที่ได้ตามปกติ	แผนทดสอบ
TC-FSM-P03	Normal > Hurt > Normal	ให้ศัตรูโจมตีโดย HP ยังเหลือ	HP ลดและกลับสู่ Normal หลังช่วง Hurt	แผนทดสอบ
TC-FSM-P04	Normal > Frozen > Normal	รับผล Frozen และทำเงื่อนไขหลุด	หยุดเคลื่อนที่และกลับมาควบคุมได้	แผนทดสอบ

รหัส	การเปลี่ยนสถานะ	เงื่อนไขทดสอบ	ผลที่คาดหวัง	ผลจริง/สถานะ
TC-FSM-P05	Normal > UsingItem > Normal	เปิดและปิด Inventory	คำสั่งการเล่นถูกล็อกและคืนกลับถูกต้อง	แผนทดสอบ
TC-FSM-P06	Normal > Transitioning	ปลดล็อกประตูสำเร็จ	ล็อกการควบคุมระหว่างเปลี่ยนด่าน	แผนทดสอบ
TC-FSM-P07	Hurt > Dead	รับความเสียหายจน HP เป็นศูนย์	เข้าสู่ Dead และแสดง Game Over	แผนทดสอบ

3.6.3 การทดสอบ Enemy FSM

ตารางที่ 3-7 แผนทดสอบ Enemy FSM

รหัส	การเปลี่ยนสถานะ	เงื่อนไขทดสอบ	ผลที่คาดหวัง	ผลจริง/สถานะ
TC-FSM-E01	Patrol > Listening	สร้างเสียงในระยะได้ยิน	ศัตรูหยุดลาดตระเวนและฟังเสียง	แผนทดสอบ
TC-FSM-E02	Listening > InvestigateSound	รอจนศัตรูเลือกเสียง	ศัตรูเคลื่อนไปยังต้นกำเนิดเสียง	แผนทดสอบ
TC-FSM-E03	InvestigateSound > ConfirmSound	ศัตรูถึงตำแหน่งเสียง	ศัตรูตรวจสอบบริเวณดังกล่าว	แผนทดสอบ
TC-FSM-E04	Patrol/Investigate > Chase	ให้ศัตรูตรวจพบผู้เล่น	ศัตรูไล่ล่าและ HUD แสดง DANGER	แผนทดสอบ
TC-FSM-E05	Chase > Attack	ผู้เล่นอยู่ในระยะโจมตี	ศัตรูโจมตีและลด HP	แผนทดสอบ
TC-FSM-E06	Attack > AttackRecover	รอหลังการโจมตี	ศัตรูพักตามช่วง cooldown	แผนทดสอบ
TC-FSM-E07	Chase > Investigate/Search	หลบออกจากการตรวจจับ	ศัตรูตรวจตำแหน่งล่าสุดและค้นหา	แผนทดสอบ
TC-FSM-E08	Search > ReturnToPatrol	รอจนหมดเวลาค้นหา	ศัตรูกลับเส้นทางลาดตระเวน	แผนทดสอบ
TC-FSM-E09	Listening > Confused	ขว้างก้อนหินซ้ำบริเวณเดิม	ศัตรูเข้าสู่ Confused ตามเงื่อนไข	แผนทดสอบ

รหัส	การเปลี่ยนสถานะ	เงื่อนไขทดสอบ	ผลที่คาดหวัง	ผลจริง/สถานะ
TC-FSM-E10	สถานะทั่วไป > Scream	ทำให้ศัตรูที่ตั้งค่า Scream เข้าเงื่อนไข	ผู้เล่นถูก Stun หรือ Frozen	แผนทดสอบ

3.6.4 การทดสอบระบบเกมและ Level 3

ตารางที่ 3-8 แผนทดสอบไอเท็มและ Level 3

รหัส	รายการทดสอบ	ขั้นตอน	ผลที่คาดหวัง	ผลจริง/สถานะ
TC-06	เก็บกุญแจ	เก็บกุญแจหนึ่งชิ้น	จำนวนกุญแจและ Inventory อัปเดต	แผนทดสอบ
TC-07	ประตูยังล็อก	ใช้ประตูก่อนกุญแจครบ	แสดงจำนวนกุญแจที่ยังขาด	แผนทดสอบ
TC-08	ปลดล็อกประตู	เก็บครบและกด E ค้าง	ความคืบหน้าครบและผ่านด่าน	แผนทดสอบ
TC-09	ยกเลิกการปลดล็อก	ปล่อย E เดินออก หรือถูกโจมตี	ความคืบหน้าถูกยกเลิก	แผนทดสอบ
TC-10	กับดักกระจก	เหยียบกับดักใน Level 3	เกิดเสียง Glass และศัตรูตรวจสอบ	แผนทดสอบ
TC-11	การสุ่มวัตถุ	เข้า Level 3 หลายรอบ	กุญแจและกับดักกระจายตามจุดที่เดินได้	แผนทดสอบ
TC-12	เสียงหัวใจ	เข้าใกล้ศัตรู	ความถี่เสียงและขอบจอแดงเพิ่มตามอันตราย	แผนทดสอบ
TC-13	จบเกม	ผ่าน Level 3	ระบบแสดง Game Won	แผนทดสอบ

3.6.5 การทดสอบโดยผู้ใช้งาน

การทดสอบโดยผู้ใช้งานมีวัตถุประสงค์เพื่อประเมินความถูกต้อง ความเสถียร ความเข้าใจง่ายของการควบคุม ความชัดเจนของ UI ความเหมาะสมของระดับความยาก และบรรยากาศของเกม ผู้ทดสอบให้คะแนนด้วยมาตราส่วน 5 ระดับ ได้แก่ 5 ดีมาก 4 ดี 3 ปานกลาง 2 น้อย และ 1 น้อยมาก

ผลการประเมินจะวิเคราะห์ด้วยค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐาน ทั้งนี้ต้องระบุจำนวนผู้ทดสอบ คุณสมบัติของกลุ่มตัวอย่าง วันที่หรือช่วงเวลาทดสอบ และผลคะแนนจริงก่อนนำเสนอข้อสรุป

3.7 การบำรุงรักษาระบบ

3.7.1 การบำรุงรักษาเพื่อแก้ไขข้อผิดพลาด

ตรวจสอบและแก้ไขปัญหาการเปลี่ยนสถานะของผู้เล่นและศัตรู การติดขอบพื้น การเดินบนสะพาน การตรวจจับจุดซ่อน การปลดล็อกประตู และการติดตามกล้องในพื้นที่ต่างระดับ

3.7.2 การบำรุงรักษาเพื่อปรับตัว

ปรับ Input System ความละเอียด อัตราส่วนภาพ ขอบเขตกล้อง และค่าประสิทธิภาพ เพื่อให้รองรับอุปกรณ์หรือสภาพแวดล้อมที่แตกต่างกัน

3.7.3 การบำรุงรักษาเพื่อพัฒนาเพิ่มเติม

เพิ่มชนิดศัตรู ไอเท็ม กับดัก ด่าน และสถานะ FSM ใหม่ได้ผ่านโครงสร้างโมดูล พร้อมปรับความยากของด่านผ่าน LevelConfig โดยไม่เปลี่ยนตรรกะหลัก

3.7.4 การบำรุงรักษาเชิงป้องกัน

ทดสอบทุกด่านหลังแก้ไขระบบส่วนกลาง ตรวจสอบ Unity Console ไม่ให้มีข้อผิดพลาด สำรอง Scene และ Prefab ก่อนแก้ไข และอัปเดตแผนภาพ ตาราง รวมถึงภาพหน้าจอในรายงานให้ตรงกับเกมเวอร์ชันล่าสุด

บทที่ 4 ผลการดำเนินงาน

บทนี้นำเสนอผลการตรวจโครงสร้างและการสาธิตต้นแบบใน Unity Play Mode เมื่อวันที่ 4 ตุลาคม 2569 ภาพเกมจับจากการทำงานจริง
ส่วนภาพเฟรมแอนิเมชันและแผนภาพระบุแหล่งที่มาตามคำบรรยาย
การสาธิตใช้สคริปต์ชั่วคราวควบคุมตำแหน่ง เรียกเมธอดและบางสถานะ หยุดการอัปเดตศัตรูเพื่อจัดภาพ และปิดโอกาสสุ่ม Jumpscare ในช่วงเก็บภาพ จึงไม่ใช่การทดลองเล่นครบวงจรและไม่ยืนยันผลผ่านทุก Test Case ยังไม่มีข้อมูลแบบสอบถามจากผู้ทดลองเล่น

4.1 ผลการพัฒนาระบบควบคุมตัวละครผู้เล่น

ผู้พัฒนาได้สร้างระบบควบคุมตัวละครผู้เล่นให้รองรับการเคลื่อนที่ภายในเกมแบบ Side-scrolling ผู้เล่นสามารถเดิน วิ่ง กระโดด ไต่บันได ใช้ไฟฉาย เก็บไอเท็ม ซ่อนตัว เปิดกระเป๋าไอเท็ม และโต้ตอบกับประตูทางออกได้ตามปุ่มควบคุมที่กำหนด การเคลื่อนที่แบบเดินเป็นรูปแบบการเคลื่อนที่พื้นฐาน ส่วนการวิ่งช่วยให้ผู้เล่นเคลื่อนที่เร็วขึ้น แต่ส่งผลให้ค่า Stamina ลดลงและสร้างเสียงที่ศัตรูสามารถตรวจจับได้จากระยะไกลขึ้น การกระโดดถูกควบคุมไม่ให้เกิดการกระโดดซ้ำกลางอากาศ ขณะที่ระบบไต่บันไดช่วยให้ผู้เล่นเดินทางระหว่างพื้นที่ต่างระดับภายในด่านได้ ระบบไฟฉายสามารถเปิดและปิดได้ระหว่างเล่น พร้อมแสดงปริมาณแบตเตอรี่คงเหลือบนหน้าจอ เมื่อเปิดไฟฉายอย่างต่อเนื่อง ปริมาณแบตเตอรี่จะลดลงตามเวลาที่ใช้งาน ทำให้ผู้เล่นต้องพิจารณาการใช้ไฟฉายให้เหมาะสมกับสถานการณ์



ภาพที่ 4-1 ผลการทำงานของระบบควบคุมตัวละครผู้เล่น (ภาพสาธิตใน Play Mode)

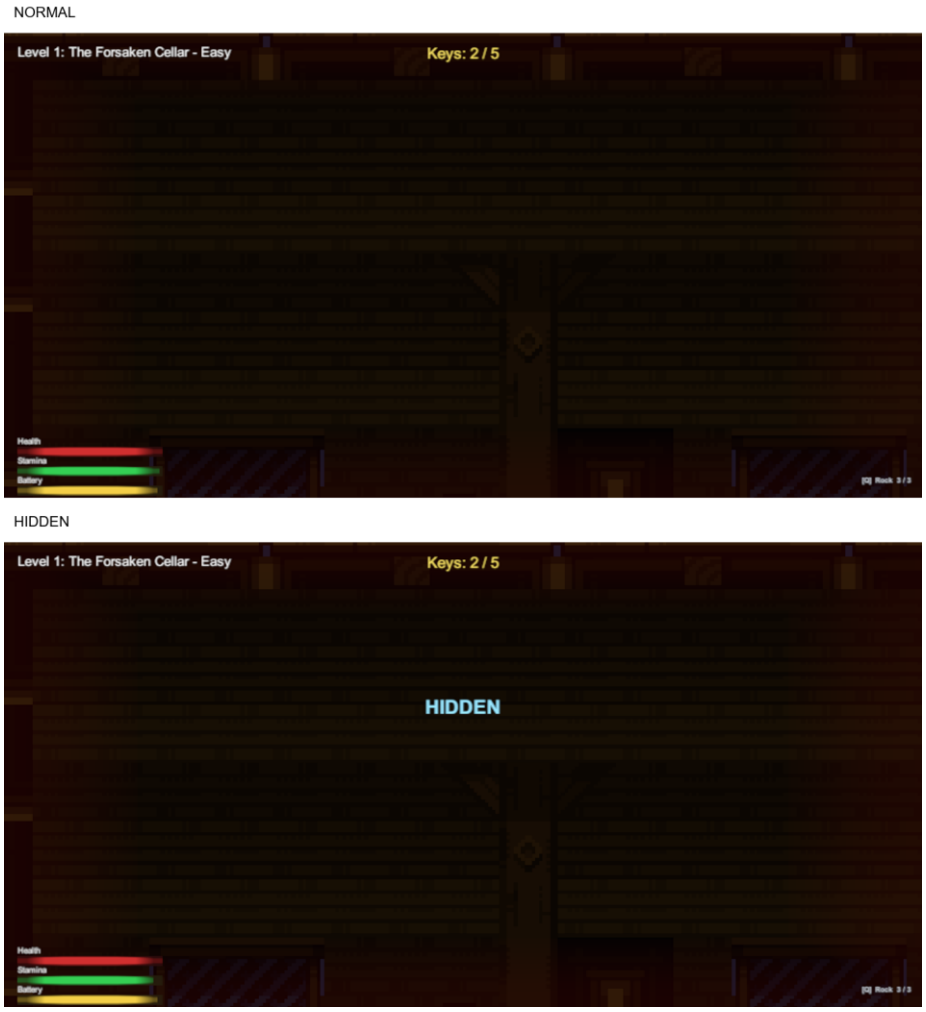


ภาพที่ 4-2 ผลการทำงานของระบบไฟฉาย (ภาพสาธิตใน Play Mode)

4.2 ผลการพัฒนา Finite State Machine ของผู้เล่น

ระบบควบคุมผู้เล่นใช้เทคนิค Finite State Machine เพื่อแบ่งพฤติกรรมออกเป็นสถานะที่ชัดเจน ประกอบด้วย Normal, Hidden, Frozen, Hurt, UsingItem, Transitioning และ Dead สถานะ Normal เป็นสถานะหลักที่ผู้เล่นสามารถเดิน วิ่ง กระโดด และรับคำสั่งควบคุมทั่วไปได้ เมื่อผู้เล่นเข้าใช้จุดซ่อน ระบบจะเปลี่ยนเป็นสถานะ Hidden และจำกัดการเคลื่อนไหวบางส่วน หากผู้เล่นได้รับผลที่ทำให้หยุดการเคลื่อนไหว ระบบจะเข้าสู่สถานะ Frozen ชั่วคราว และกลับสู่ Normal เมื่อพ้นระยะเวลาที่กำหนดเมื่อได้รับความเสียหาย ผู้เล่นจะเปลี่ยนเข้าสู่สถานะ Hurt เพื่อป้องกันการรับความเสียหายซ้ำในช่วงเวลาสั้น ๆ หากพลังชีวิตลดลงจนหมด ระบบจะเปลี่ยนเป็นสถานะ Dead และแสดงหน้าจอ Game Over ส่วนสถานะ UsingItem ใช้ระหว่างเปิดกระเป๋าหรือใช้ไอเท็ม และสถานะ Transitioning ใช้ขณะระบบกำลังเปลี่ยนฉาก

บันทึกการสาธิตพบสถานะ Normal, UsingItem, Hidden, Frozen, Hurt และ Dead ตามเมธอดที่เรียก ภาพแสดงผล HIDDEN และ Game Over สอดคล้องกับสถานะที่บันทึก การสาธิตนี้ยืนยันการแสดงผลภายใต้เงื่อนไขที่จัดไว้
ยังไม่ยืนยันว่าทุกการเปลี่ยนสถานะจะถูกต้องเมื่อหลายเหตุการณ์เกิดพร้อมกัน



ภาพที่ 4-3 สาธิตหน้าจอ Normal และ Hidden ของผู้เล่น

4.3 ผลการพัฒนาระบบปัญญาประดิษฐ์ของศัตรู

ระบบปัญญาประดิษฐ์ของศัตรูได้รับการพัฒนาให้ตอบสนองต่อผู้เล่นผ่านการมองเห็นและการได้ยิน โดยมี KillerAI เป็นส่วนควบคุมพฤติกรรมหลัก ศัตรูสามารถลาดตระเวน รับรู้สิ่งผิดปกติ ตรวจสอบตำแหน่งเสียง ค้นหาผู้เล่น ไล่ล่า โจมตี และกลับไปยังเส้นทางลาดตระเวนได้ Enemy FSM ประกอบด้วยสถานะ Patrol, Alert, Listening, InvestigateSound, ConfirmSound, Confused, Investigate, Search, Chase, Attack, AttackRecover, ReturnToPatrol และ Scream สถานะเหล่านี้ทำให้ศัตรูสามารถตอบสนองต่อสถานการณ์ได้ละเอียดกว่าการแบ่งพฤติกรรมเพียงการลาดตระเวนและไล่ล่า ในสถานะ Patrol ศัตรูจะเดินตามเส้นทางลาดตระเวน เมื่อได้ยินเสียงจะเข้าสู่กระบวนการ Listening และ InvestigateSound เพื่อเลือกและเดินทางไปยังตำแหน่งกำเนิดเสียง หากตรวจสอบแล้วยังไม่พบผู้เล่น ศัตรูอาจเข้าสู่สถานะ Confused, Investigate หรือ Search ก่อนกลับไปลาดตระเวน เมื่อศัตรูตรวจพบผู้

เล่น ระบบจะเปลี่ยนเข้าสู่สถานะ Chase และเคลื่อนที่ไล่ตามตำแหน่งของผู้เล่น หากผู้เล่นอยู่ในระยะโจมตี ศัตรูจะเข้าสู่สถานะ Attack หลังจากโจมตีแล้วจะพักตามระยะเวลาที่กำหนดในสถานะ AttackRecover ก่อนตัดสินใจไล่ตามหรือตรวจสอบพื้นที่ต่อไป



ภาพที่ 4-4 Killer ในสถานะ Patrol ที่จัดไว้เพื่อสาธิต



ภาพที่ 4-5 สาธิตการรับเสียง Rock และสถานะ Listening



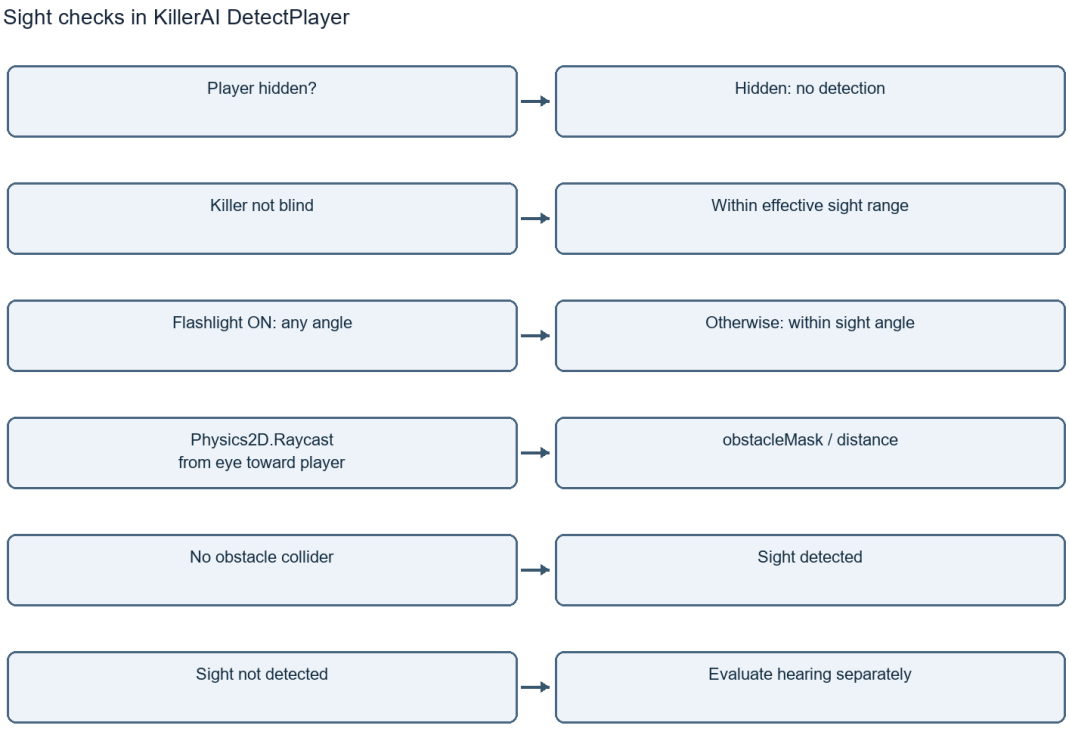
ภาพที่ 4-6 สาธิตสถานะ Chase และข้อความ DANGER

4.4 ผลการพัฒนาระบบตรวจจับผู้เล่น

ระบบตรวจจับแบ่งออกเป็นการตรวจจับด้วยการมองเห็นและการตรวจจับด้วยเสียง การมองเห็นใช้ระยะตรวจจับ ทิศทาง และสิ่งกีดขวางภายในฉากเป็นเงื่อนไข ศัตรูจึงไม่สามารถตรวจพบผู้เล่นผ่านวัตถุที่บดบังการมองเห็นได้

ระบบเสียงรับเหตุการณ์จาก FootstepSystem และกิจกรรมที่สร้างเสียงภายในด่าน เช่น การวิ่งและการเหยียบกับดักกระจก เสียงแต่ละชนิดมีความสำคัญแตกต่างกัน โดยเสียง Sprint และ Glass มีผลต่อการรับรู้ของศัตรูมากกว่าเสียง Footstep ทั่วไป

เมื่อศัตรูได้รับเหตุการณ์เสียง ระบบจะบันทึกตำแหน่งกำเนิดเสียงและเปลี่ยนไปยังสถานะที่เกี่ยวข้องเพื่อทำการตรวจสอบ ส่งผลให้ผู้เล่นต้องตัดสินใจว่าจะเคลื่อนที่อย่างรวดเร็วหรือหลีกเลี่ยงการสร้างเสียงเพื่อไม่ให้ศัตรูทราบตำแหน่ง



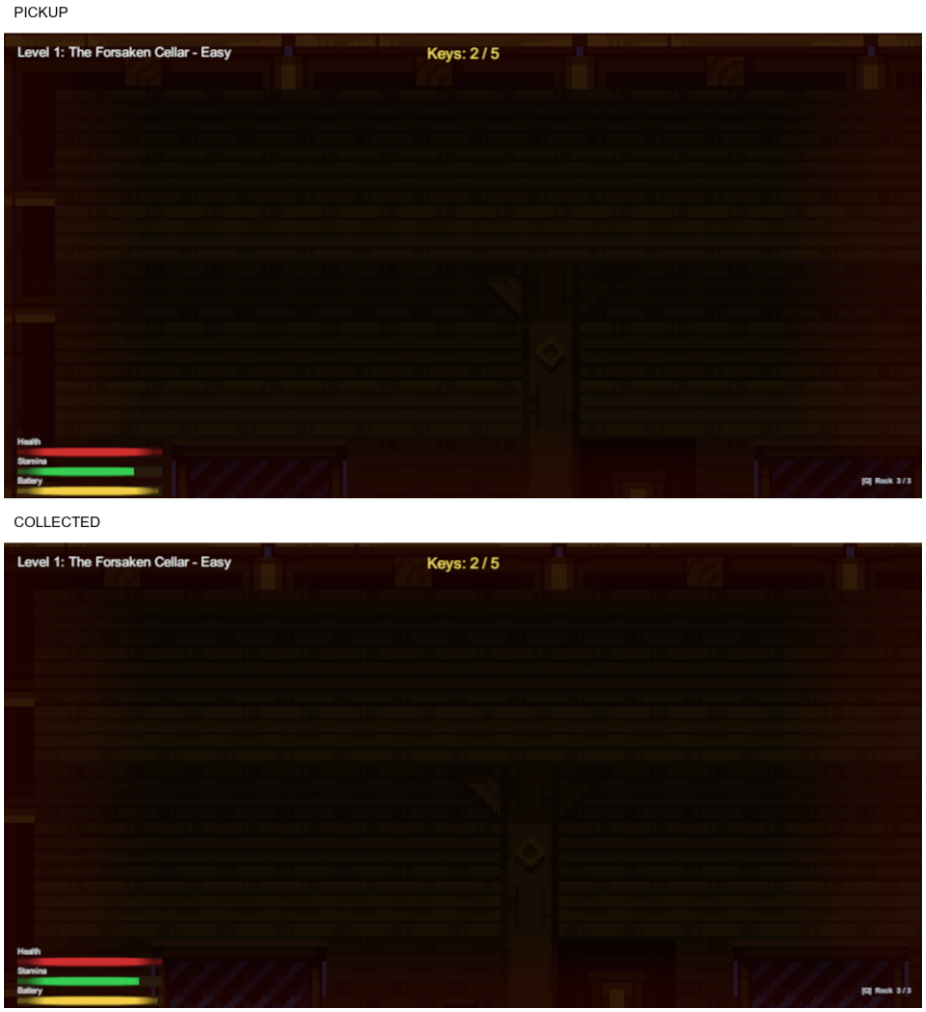
ภาพที่ 4-7 แผนภาพเงื่อนไขการตรวจแนวสายตาจาก KillerAI



ภาพที่ 4-8 สาธิตพื้นที่กับดักเสียงของ Level 3

4.5 ผลการพัฒนาระบบไอเท็มและประตูทางออก

ระบบไอเท็มใช้ InventoryManager สำหรับจัดเก็บและนับจำนวนกุญแจที่ผู้เล่นเก็บได้ เมื่อผู้เล่นเก็บกุญแจ จำนวนกุญแจบน HUD และข้อมูลในกระเป๋าไอเท็มจะเปลี่ยนแปลงทันที ประตูทางออกใช้ FSM ควบคุมสถานะ ได้แก่ Locked, Ready, Unlocking, Open, Transitioning และ Completed หากผู้เล่นยังเก็บกุญแจไม่ครบ ประตูจะแสดงจำนวนกุญแจที่ยังขาด เมื่อผู้เล่นเก็บกุญแจครบและกดปุ่ม E ค้าง ระบบจะแสดงความคืบหน้าการปลดล็อก เมื่อดำเนินการสำเร็จ ประตูจะเปิดและโหลดด่านถัดไป หากเป็นด่านสุดท้าย ระบบจะแสดงผลการจบเกม การแยกสถานะของประตูช่วยป้องกันการปลดล็อกซ้ำ และรองรับการยกเลิกเมื่อผู้เล่นปล่อยปุ่ม เดินออกจากพื้นที่ หรือถูกศัตรูโจมตีระหว่างปลดล็อก



ภาพที่ 4-9 ผลการทำงานของระบบเก็บกุญแจ (ภาพสาธิตใน Play Mode)



ภาพที่ 4-10 ผลการทำงานของระบบประตูทางออก (ภาพสาธิตใน Play Mode)

4.6 ผลการพัฒนาระบบส่วนติดต่อผู้ใช้งาน

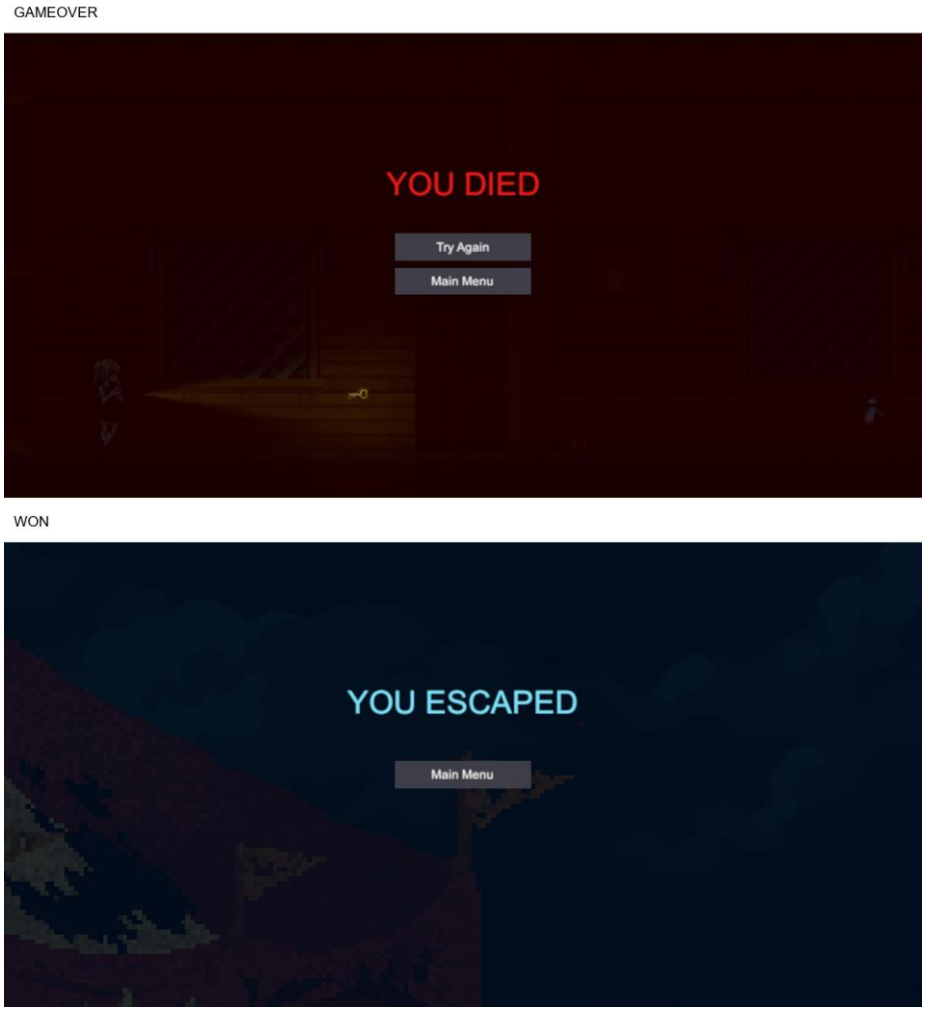
ระบบส่วนติดต่อผู้ใช้งานได้รับการออกแบบให้แสดงข้อมูลที่จำเป็นระหว่างเล่น เช่น พลังชีวิต จำนวนกุญแจ แบตเตอรี่ไฟฉาย Stamina ข้อความโต้ตอบ สถานะการซ่อน และคำเตือนเมื่อศัตรูอยู่ใกล้

เมื่อผู้เล่นเข้าใกล้วัตถุที่สามารถโต้ตอบได้ ระบบจะแสดงข้อความแนะนำปุ่มควบคุม เช่น การเก็บไอเท็ม การเข้าจุดซ่อน และการปลดล็อกประตู ข้อความจะเปลี่ยนไปตามสถานการณ์และหายไปเมื่อผู้เล่นออกจากพื้นที่โต้ตอบ

เมื่อศัตรูตรวจพบหรือไล่ล่าผู้เล่น HUD จะแสดงคำเตือนและเอฟเฟกต์ภาพตามระดับอันตราย หากผู้เล่นเสียชีวิต ระบบจะแสดงหน้า Game Over แต่หากผ่านด่านสำเร็จจะแสดง Level Complete หรือ Game Won ตามสถานะของเกม



ภาพที่ 4-11 ผลการแสดงข้อมูลบนหน้าจอระหว่างเล่น (ภาพสาธิตใน Play Mode)

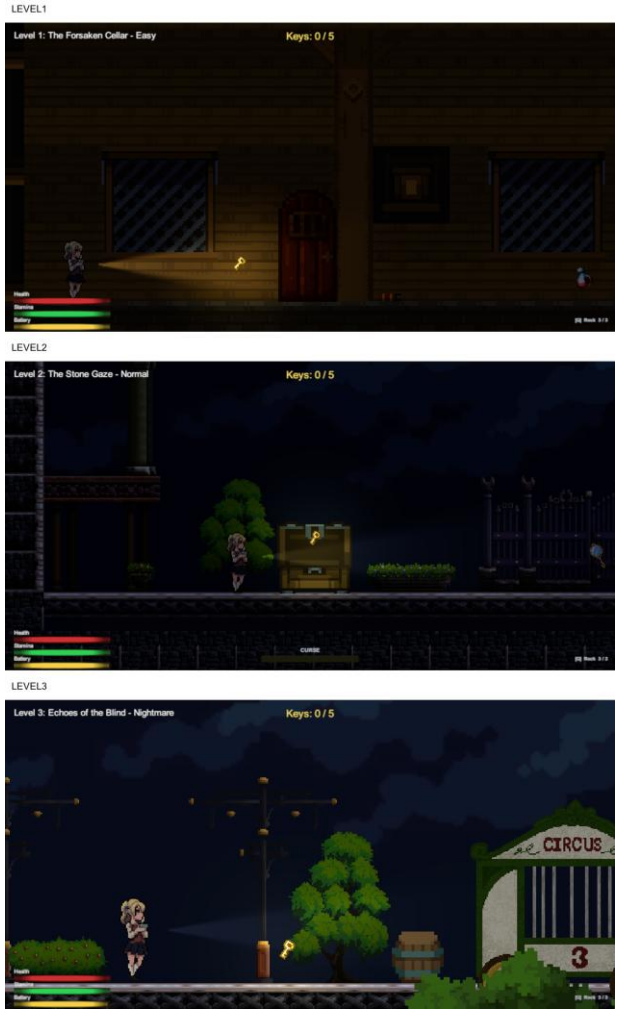


ภาพที่ 4-12 ผลการแสดงหน้าจอสรุปสถานะเกม (ภาพสาธิตใน Play Mode)

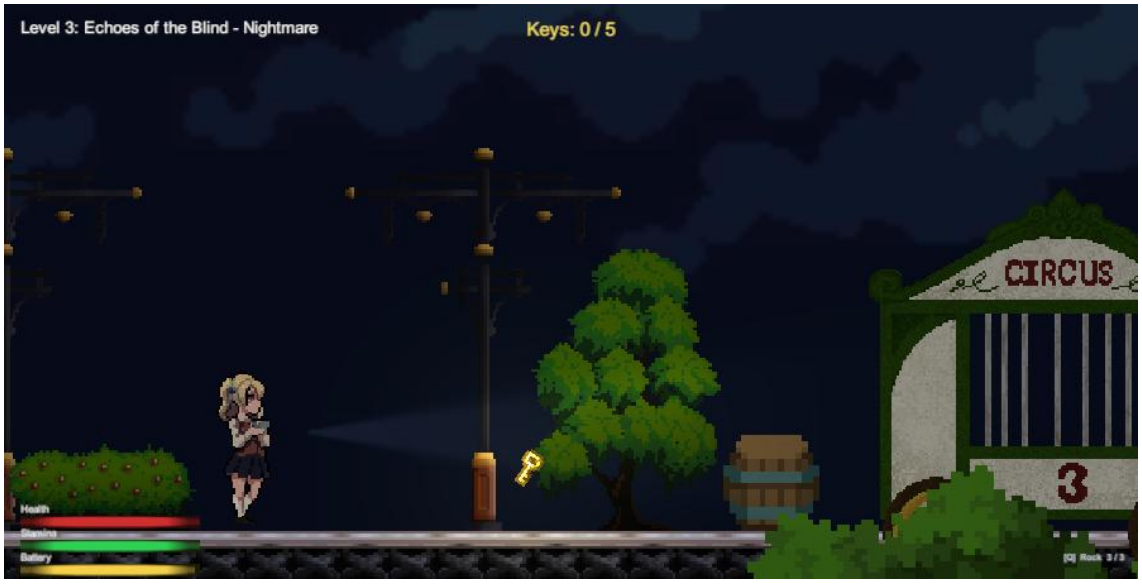
4.7 ผลการพัฒนาด่านภายในเกม

เกมประกอบด้วยด่านที่มีการกำหนดระดับความยากแตกต่างกันผ่าน LevelConfig โดยแต่ละด่านสามารถปรับค่าการทำงาน เช่น ระยะมองเห็น ระยะได้ยิน ความเร็วของศัตรู ความเสียหาย และปริมาณแบตเตอรี่เริ่มต้น Level 1 เป็นด่านเริ่มต้นที่ช่วยให้ผู้เล่นเรียนรู้ระบบพื้นฐาน ศัตรูมีความเร็วและความสามารถในการตรวจจับต่ำกว่าด่านอื่น Level 2 เพิ่มความกดดันจากระบบคำสาปและลดพื้นที่การตรวจจับของศัตรูบางส่วนเพื่อให้เหมาะกับโครงสร้างของด่านLevel 3 ได้รับการออกแบบเป็นสวนสนุกเพื่อสร้างบรรยากาศน่ากลัว ภายในด่านมีเส้นทางต่างระดับ กุญแจ กับดักกระจก และจุดซ่อนที่กระจายตามตำแหน่งที่สามารถเดินได้ ระบบ Level3RandomEvents ทำให้ตำแหน่งวัตถุบางส่วนเปลี่ยนแปลงเมื่อเริ่มเล่นใหม่ จึงช่วยลดการจดจำตำแหน่งจากการเล่นรอบก่อนLevel 3 กำหนดให้ศัตรูเน้นการตรวจจับด้วยเสียง โดยมีระยะได้ยินและความเร็วไล่ล่าสูง เมื่อผู้เล่นสร้างเสียงหรือเข้าใกล้ศัตรู ระบบจะ

แสดงขอบจอสีแดงและเล่นเสียงหัวใจตามระดับอันตราย ส่งผลให้ด่านมีบรรยากาศกดดันและแตกต่างจากด่านก่อนหน้า



ภาพที่ 4-13 ผลการพัฒนาด่านภายในเกม (ภาพสาธิตใน Play Mode)



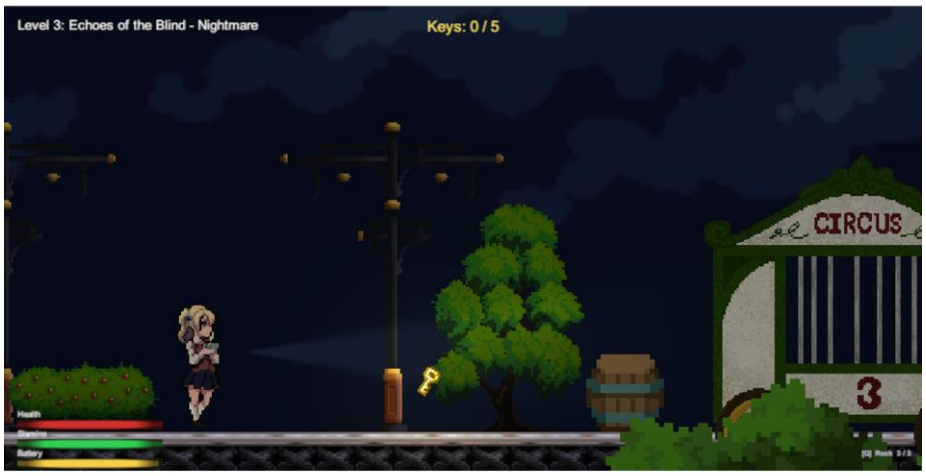
ภาพที่ 4-14 สวนสนุกและพื้นที่ป่าใน Level 3 (ภาพสาธิตใน Play Mode)

4.8 ผลการพัฒนาระบบความตึงเครียด

TensionDirector รวมสถานะของศัตรูเป็น Calm, Suspicious, Hunted, Escaped และ Cooldown แล้วส่งเหตุการณ์ให้ LevelAudioController ปรับดนตรี ส่วน UIManager แสดง DANGER และ Level3HeartbeatWarning ควบคุมคำเตือนตามระยะห่าง
ภาพเปรียบเทียบแสดงหน้าจอปกติกับหน้าจอที่เรียก Chase เพื่อสาธิต ไม่ได้จับหน้าต่าง Debug ของ TensionDirector จึงยังไม่ใช้ภาพยืนยันลำดับของทั้งห้าสถานะครบถ้วน

สิ่งที่เห็นจากภาพคือข้อความ DANGER และขอบจอสีแดงตามเงื่อนไขที่จัดไว้ ส่วนความชัดเจนของเสียงและความเหมาะสมของระดับความกดดันต้องตรวจด้วยการฟังและทดลองเล่นเพิ่มเติม

LEVEL3



WARNING



ภาพที่ 4-15 เปรียบเทียบหน้าจอปกติกับคำเตือนเมื่อเรียก Chase (ภาพสาธิตใน Play Mode)

4.9 ผลการพัฒนาระบบแอนิเมชันของตัวละคร

ผลการพัฒนาในส่วนนี้ต่อเนื่องจากการออกแบบ Animator ในหัวข้อ 3.5.8–3.5.11 โดยตรวจสอบไฟล์ Animation Clip, Animator Controller และสคริปต์ที่เชื่อมต่อกัน ภาพประกอบเป็นเฟรมจาก Sprite Sheet ที่ใช้ในโครงการ มิใช่ภาพหน้าจอขณะทดลองเล่น จึงใช้แสดงลักษณะของท่าทางและผลการจัดเตรียมคลิป ส่วนผลการตอบสนองระหว่างเล่นต้องยืนยันด้วยการทดสอบเพิ่มเติม

4.9.1 ผลการจัดทำแอนิเมชันของผู้เล่น



ภาพที่ 4-16 ตัวอย่างสี่เฟรมแรกของแอนิเมชัน Idle ของผู้เล่น



ภาพที่ 4-17 ตัวอย่างสี่เฟรมแรกของแอนิเมชัน Walk ของผู้เล่น

ภาพที่ 4-16 และ 4-17 แสดงท่ายืนและท่าเดินที่นำไปใช้กับ PlayerIdle.anim และ PlayerWalk.anim ตามลำดับ ทั้งสองคลิปกำหนด Sample Rate เท่ากับ 12 และเปิด Loop Time เพื่อเล่นวนขณะตัวละครคงอยู่ในสถานะเดิม โดยคลิป Idle มีช่วงเวลา 0.75 วินาที ส่วน Walk มีช่วงเวลา 1 วินาที

จากการตรวจสอบโครงสร้าง PlayerAnimator พบสถานะ Idle และ Walk เชื่อมด้วย isWalking ชนิด Bool และ PlayerController มีคำสั่ง SetBool(IsWalkingHash, IsMoving) ตรงกับชื่อพารามิเตอร์ ผลที่ยืนยันได้คือมีคลิป สถานะ และจุดส่งค่าจากสคริปต์ครบสำหรับการเลือกท่ายืนและเดิน ส่วนความถูกต้องของภาพที่แสดงเมื่อกดหรือปล่อยปุ่มยังต้องตรวจใน Play Mode

Controller ของผู้เล่นในรุ่นที่ตรวจสอบยังไม่มีสถานะ Run แยกต่างหาก จึงรายงานเฉพาะผลการจัดทำ Idle และ Walk การเพิ่มความเร็ววิ่งในระบบควบคุมไม่ได้เป็นหลักฐานว่ามีแอนิเมชัน Run เพิ่มขึ้นด้วย

4.9.2 ผลการจัดทำแอนิเมชันของ Killer

KillerAnimator มีสถานะ Idle, Walk, Run, Attack1 และ Scream โดยเชื่อมกับไฟล์ KillerIdle.anim, KillerWalk.anim, KillerRun.anim, KillerAttack1.anim และ KillerScream.anim ภาพต่อไปนี้แสดงเฟรมต้นคลิปจากไฟล์ภาพที่ใช้กับศัตรู เพื่อให้เห็นความแตกต่างระหว่างการเคลื่อนที่ การโจมตี และการกรีดร้อง



ภาพที่ 4-18 ตัวอย่างสี่เฟรมแรกของแอนิเมชัน Walk ของ Killer



ภาพที่ 4-19 ตัวอย่างสี่เฟรมแรกของแอนิเมชัน Run ของ Killer



ภาพที่ 4-20 ตัวอย่างสี่เฟรมแรกของแอนิเมชัน Attack1 ของ Killer



ภาพที่ 4-21 ตัวอย่างสี่เฟรมแรกของแอนิเมชัน Scream ของ Killer

ผลการตรวจไฟล์คลิปพบว่า Idle, Walk และ Run เปิด Loop Time เพื่อแสดงท่าทางต่อเนื่อง ส่วน Attack1 และ Scream ปิด Loop Time เพื่อให้เล่นเป็นครั้งตามเหตุการณ์ การกลับสู่ Idle หลังสองท่านี้กำหนดด้วย Transition ที่เปิด Has Exit Time และตั้ง Exit Time เท่ากับ 1 ตามแผนผังในภาพที่ 3-23

4.9.3 ผลการตรวจสอบการเชื่อมต่อและกรณีทดสอบแอนิเมชัน

การตรวจสอบไฟล์ Controller และสคริปต์พบว่า isWalking และ isRunning ของ Killer เป็น Bool และมีคำสั่ง SetBool ใน KillerAI ส่วน attack1 และ scream เป็น Trigger และมีคำสั่ง SetTrigger ที่ใช้เรียกเหตุการณ์โจมตีและกรีดร้อง ชื่อพารามิเตอร์ที่สคริปต์เรียกตรงกับรายการใน Controller จึงยืนยันการเชื่อมต่อได้ในระดับโครงสร้างไฟล์

Bool ทำหน้าที่คงสถานะการเคลื่อนที่ ขณะที่ Trigger ใช้เริ่มการกระทำเฉพาะครั้ง ผลการตรวจสอบนี้สอดคล้องกับเหตุผลการเลือกชนิดพารามิเตอร์ในหัวข้อ 3.5.11 ไม่พบ Float หรือ Int ใน Animator ทั้งสองตัว จึงไม่มีผลทดสอบการทำงานของพารามิเตอร์สองชนิดนี้ในรุ่นปัจจุบัน

เพื่อยืนยันผลขณะทำงานจริง กำหนดกรณีทดสอบต่อไปนี้ โดยทุกกรณียังมีสถานะรอทดสอบใน Play Mode และต้องบันทึกภาพสถานะ Animator พร้อมผลที่สังเกตได้ก่อนสรุปว่าผ่าน

AN-P01 การเริ่มต้นและหยุดเคลื่อนที่ของผู้เล่น

เริ่มฉากโดยไม่กดปุ่มเคลื่อนที่ แล้วตรวจว่า Idle ทำงานและ isWalking เป็น false จากนั้นเดินและปล่อยปุ่ม ผลที่คาดหวังคือเปลี่ยนไป Walk เมื่อเป็น true และกลับ Idle เมื่อเป็น false โดยภาพไม่ค้าง

AN-K01 การเดินลาดตระเวนและการไล่ล่า

ให้ Killer เดินลาดตระเวนแล้วตรวจ isWalking จากนั้นกระตุ้นให้เข้าสู่ Chase ผลที่คาดหวังคือใช้ Walk ขณะเดิน และเปลี่ยนเป็น Run เมื่อ isRunning เป็น true

AN-K02 การโจมตีและกลับสู่สถานะเคลื่อนที่

ให้ผู้เล่นอยู่ในระยะโจมตีจน KillerAI ส่ง attack1 ผลที่คาดหวังคือเล่น Attack1 ตามเหตุการณ์ แล้วกลับ Idle ตาม Exit Time ก่อนเลือกสถานะเคลื่อนที่ตามค่าพารามิเตอร์ โดยบันทึกทั้งภาพและความเสียหายที่เกิดขึ้น

AN-K03 การกรีดร้องจากสถานะอื่น

ใช้ศัตรูหรือเงื่อนไขที่เปิดความสามารถกรีดร้อง แล้วกระตุ้นการส่ง scream ผลที่คาดหวังคือเปลี่ยนจากสถานะปัจจุบันผ่าน Any State ไป Scream และกลับ Idle หลังคลิปถึง Exit Time

AN-K04 การหยุดไล่ล่าและกลับไปเดิน

ทำให้ Killer ออกจาก Chase และกลับไปเดิน ผลที่คาดหวังคือหยุด Run และแสดงท่าที่สัมพันธ์กับพฤติกรรม โดยตรวจลำดับ Transition เมื่อ isRunning เป็น false และ isWalking เป็น true พร้อมกัน

4.10 ผลการทดสอบการทำงานของระบบ

ตารางต่อไปแยกการตรวจโค้ดและการสาธิตออกจากการทดสอบครบวงจร “สาธิตบางส่วน” หมายถึงสังเกตผลบางส่วนใน Play Mode ภายใต้การควบคุมของสคริปต์ ส่วน “ยังไม่ทดสอบ” หมายถึงยังไม่มีหลักฐานเพียงพอสำหรับผลที่คาดหวังทั้งแถว จึงยังไม่มีแถวใดสรุปว่าผ่านครบทุกเงื่อนไข

ตารางที่ 4-1 ผลการทดสอบระบบควบคุมพื้นฐาน

รหัส	รายการทดสอบ	ผลที่คาดหวัง	ผลจริง	สถานะ
TC-01	การเดิน	ผู้เล่นเคลื่อนที่และหันถูกทิศ	มีภาพเดินจากการส่งปุ่ม D; ยังไม่ตรวจครบทุกทิศ	สาธิตบางส่วน
TC-02	การวิ่ง	ความเร็วเพิ่ม Stamina ลด และเกิดเสียง Sprint	มีภาพจากการส่ง D กับ Shift; ยังไม่วัดความเร็วและเสียง	สาธิตบางส่วน
TC-03	การกระโดด	กระโดดได้ครั้งเดียวและไม่กระโดดซ้ำกลางอากาศ	มีภาพจากปุ่ม Space; ยังไม่ทดสอบกระโดดซ้ำ	สาธิตบางส่วน
TC-04	การไต่บันได	ผู้เล่นขึ้นลงบันไดและกระโดดออกได้	ตรวจโค้ดพบเงื่อนไข; ยังไม่มีผลทดลองครบกรณี	ยังไม่ทดสอบ
TC-05	ไฟฉาย	ไฟฉายเปิดปิดได้และแบตเตอรี่ลดลง	มีภาพไฟฉายหลังเรียก SetOn; ยังไม่ทดสอบการหมดแบตเตอรี่	สาธิตบางส่วน

ตารางที่ 4-2 ผลการทดสอบ Player FSM

รหัส	การเปลี่ยนสถานะ	ผลที่คาดหวัง	ผลจริง	สถานะ
TC-FSM-P01	Normal ⟶ Hidden	ผู้เล่นซ่อนและ HUD แสดง HIDDEN	เรียก ToggleHide หลังจัดเงื่อนไข; log เป็น Hidden และเห็น HIDDEN	สาธิตบางส่วน
TC-FSM-P02	Hidden ⟶ Normal	ผู้เล่นออกจากจุดซ่อนและเคลื่อนที่ได้	เรียก ForceReveal; log กลับ Normal	สาธิตบางส่วน
TC-FSM-P03	Normal ⟶ Hurt ⟶ Normal	HP ลดและกลับสู่ Normal หลังช่วง Hurt	เรียก TakeDamage(10); log เป็น Hurt	สาธิตบางส่วน
TC-FSM-P04	Normal ⟶ Frozen ⟶ Normal	ผู้เล่นหยุดเคลื่อนไหวชั่วคราวและกลับมาควบคุมได้	เรียก Freeze และ Unfreeze; log เป็น Frozen และกลับ Normal	สาธิตบางส่วน
TC-FSM-P05	Normal ⟶ UsingItem ⟶ Normal	การควบคุมถูกล็อกระหว่างใช้ไอเท็มและคืนค่าถูกต้อง	เรียก OpenRoutine; log เป็น UsingItem และมีภาพคลัง	สาธิตบางส่วน
TC-FSM-P06	Normal ⟶ Transitioning	ล็อกการควบคุมระหว่างเปลี่ยนฉาก	เรียก CompleteUnlock; log เป็น Transitioning และ LevelComplete	สาธิตบางส่วน

รหัส	การเปลี่ยนสถานะ	ผลที่คาดหวัง	ผลจริง	สถานะ
TC-FSM-P07	Hurt ⟶ Dead	แสดงสถานะ Dead และหน้า Game Over	เรียก Kill; log เป็น Dead และมี Game Over; ไม่ใช่โจมตีต่อเนื่อง	สาธิตบางส่วน

ตารางที่ 4-3 ผลการทดสอบ Enemy FSM

รหัส	การเปลี่ยนสถานะ	ผลที่คาดหวัง	ผลจริง	สถานะ
TC-FSM-E01	Patrol ⟶ Listening	ศัตรูหยุดลาดตระเวนและรับฟังเสียง	ส่ง HearNoise ประเภท Rock; log จาก Patrol เป็น Listening	สาธิตบางส่วน
TC-FSM-E02	Listening ⟶ InvestigateSound	ศัตรูเคลื่อนที่ไปยังต้นกำเนิดเสียง	ตรวจโค้ดพบเงื่อนไข; ยังไม่มีผลทดลองครบกรณี	ยังไม่ทดสอบ
TC-FSM-E03	InvestigateSound ⟶ ConfirmSound	ศัตรูตรวจสอบบริเวณที่เกิดเสียง	ตรวจโค้ดพบเงื่อนไข; ยังไม่มีผลทดลองครบกรณี	ยังไม่ทดสอบ
TC-FSM-E04	Patrol/Investigate ⟶ Chase	ศัตรูเริ่มไล่ล่าและ HUD แสดง DANGER	เรียก SetState(Chase); มี DANGER; ยังไม่ทดสอบการตรวจพบตามธรรมชาติ	สาธิตบางส่วน
TC-FSM-E05	Chase ⟶ Attack	ศัตรูโจมตีและลด HP ของผู้เล่น	ตรวจโค้ดพบเงื่อนไข; ยังไม่มีผลทดลองครบกรณี	ยังไม่ทดสอบ
TC-FSM-E06	Attack ⟶ AttackRecover	ศัตรูหยุดพักตามเวลาหลังการโจมตี	ตรวจโค้ดพบเงื่อนไข; ยังไม่มีผลทดลองครบกรณี	ยังไม่ทดสอบ
TC-FSM-E07	Chase ⟶ Investigate/Search	ศัตรูค้นหาตำแหน่งล่าสุดของผู้เล่น	ตรวจโค้ดพบเงื่อนไข; ยังไม่มีผลทดลองครบกรณี	ยังไม่ทดสอบ
TC-FSM-E08	Search ⟶ ReturnToPatrol	ศัตรูกลับสู่เส้นทางลาดตระเวน	ตรวจโค้ดพบเงื่อนไข; ยังไม่มีผลทดลองครบกรณี	ยังไม่ทดสอบ
TC-FSM-E09	Listening ⟶ Confused	ศัตรูเข้าสู่ Confused ตามเงื่อนไข	ตรวจโค้ดพบเงื่อนไข; ยังไม่มีผลทดลองครบกรณี	ยังไม่ทดสอบ
TC-FSM-E10	สถานะทั่วไป ⟶ Scream	ผู้เล่นได้รับผล Stun หรือ Frozen	ตรวจโค้ดพบเงื่อนไข; ยังไม่มีผลทดลองครบกรณี	ยังไม่ทดสอบ

ตารางที่ 4-4 ผลการทดสอบระบบเกมและ Level 3

รหัส	รายการทดสอบ	ผลที่คาดหวัง	ผลจริง	สถานะ
TC-06	เก็บกุญแจ	จำนวนกุญแจและ Inventory เปลี่ยนแปลง	เรียก Collect; จำนวนกุญแจและภาพคลังเปลี่ยน	สาธิตบางส่วน
TC-07	ใช้ประตูก่อนกุญแจครบ	แสดงจำนวนกุญแจที่ยังขาด	มีภาพประตูก่อนเก็บครบ; ยังไม่ยืนยัน Prompt ทุกเงื่อนไข	สาธิตบางส่วน
TC-08	ปลดล็อกประตู	ความคืบหน้าครบและผ่านด่าน	กำหนด Unlocking และความคืบหน้า; มี Prompt และเรียก CompleteUnlock	สาธิตบางส่วน
TC-09	ยกเลิกการปลดล็อก	ความคืบหน้าถูกยกเลิก	ตรวจโค้ดพบเงื่อนไข; ยังไม่มีผลทดลองครบกรณี	ยังไม่ทดสอบ
TC-10	กับดักกระจก	เกิดเสียง Glass และศัตรูเข้าตรวจสอบ	จัดผู้เล่นที่กับดัก; log Killer เป็น Listening; ยังไม่ฟังเสียงครบลำดับ	สาธิตบางส่วน
TC-11	การสุ่มวัตถุ	กุญแจและกับดักกระจายตามจุดที่เดินได้	ตรวจโค้ดพบเงื่อนไข; ยังไม่มีผลทดลองครบกรณี	ยังไม่ทดสอบ
TC-12	เสียงหัวใจ	ความถี่เสียงและขอบจอเพิ่มตามระดับอันตราย	ภาพมีขอบจอสีแดงเมื่อจัดระยะใกล้; ยังไม่วัดเสียงหัวใจ	สาธิตบางส่วน
TC-13	ผ่าน Level 3	ระบบแสดง Game Won	เรียก NotifyLevelCompleted และ ContinueToNextLevel; เห็น YOU ESCAPED	สาธิตบางส่วน

4.11 ผลการประเมินโดยผู้ใช้งาน

กำหนดการประเมิน 6 ด้านด้วยคะแนน 1–5 โดย 5 หมายถึงดีมาก 4 ดี 3 ปานกลาง 2 น้อย และ 1 น้อยมาก ณ วันที่จัดทำรายงานยังไม่ได้เก็บแบบสอบถาม จึงไม่มีจำนวนผู้ตอบจริง ค่าเฉลี่ย ส่วนเบี่ยงเบนมาตรฐาน หรือข้อเสนอแนะจากผู้ใช้ ตารางนี้รายงานสถานะของข้อมูลจริง ส่วนตัวอย่างคะแนนสมมติแสดงในภาคผนวก ข

ตารางที่ 4-5 สถานะข้อมูลการประเมินจากผู้ใช้งานจริง

รายการประเมิน	ค่าเฉลี่ย	ระดับผลการประเมิน
ความสะดวกในการควบคุมตัวละคร	ยังไม่มีข้อมูล	ยังไม่ประเมิน
ความชัดเจนของ UI และข้อความแนะนำ	ยังไม่มีข้อมูล	ยังไม่ประเมิน
ความเหมาะสมของพฤติกรรมศัตรู	ยังไม่มีข้อมูล	ยังไม่ประเมิน

รายการประเมิน	ค่าเฉลี่ย	ระดับผลการประเมิน
ความสมดุลของระดับความยาก	ยังไม่มีข้อมูล	ยังไม่ประเมิน
ความน่ากลัวและบรรยากาศของเกม	ยังไม่มีข้อมูล	ยังไม่ประเมิน
ความพึงพอใจโดยรวม	ยังไม่มีข้อมูล	ยังไม่ประเมิน

ยังไม่สามารถจัดอันดับด้านที่ได้รับคะแนนสูงสุดหรือสรุประดับความพึงพอใจจริงได้ หลังเก็บข้อมูลควรคำนวณค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐานรายด้าน ระบุจำนวนผู้ตอบและช่วงเวลาทดสอบ แล้วปรับบทอภิปรายให้สอดคล้องกับข้อมูลที่ได้ ตัวเลขในภาคผนวก ข เป็นตัวอย่างสมมติและไม่ใช้แทนผลส่วนนี้

4.12 สรุปผลการดำเนินงาน

มีต้นแบบเกมสามด่าน พร้อมสคริปต์ควบคุมผู้เล่น Killer ไอเท็ม ประตู และส่วนติดต่อผู้ใช้ การสาธิตแสดงหน้าจอเมนู การเคลื่อนที่ คลังไอเท็ม สถานะซ่อน คำเตือน และหน้าจอผลลัพธ์ภายใต้การจัดเงื่อนไข การตรวจไฟล์ยืนยันโครงสร้าง FSM และการเชื่อม Animator ส่วนความถูกต้องของการเล่นตั้งแต่เริ่มจนชนะทุกด่านและความพึงพอใจยังต้องทดสอบเพิ่มเติม

ผลการตรวจโครงสร้างแอนิเมชันพบว่ามีคลิป Idle และ Walk สำหรับผู้เล่น และ Idle, Walk, Run, Attack1 และ Scream สำหรับ Killer พร้อมการส่งพารามิเตอร์ Bool และ Trigger จากสคริปต์ตามหัวข้อ 4.9 ทั้งนี้ผลการเปลี่ยนภาพระหว่างเล่นยังต้องยืนยันจากกรณีทดสอบ AN-P01 และ AN-K01–AN-K04

ข้อจำกัดของหลักฐานคือการควบคุมสถานะระหว่างถ่ายภาพและยังไม่มีผู้ทดลองเล่น รายงานจึงสรุปผลเฉพาะโครงสร้างกับการแสดงผลที่ตรวจพบ ไม่รายงานอัตราผ่าน เวลาเล่น หรือคะแนนความพึงพอใจเป็นผลจริง

บทที่ 5 สรุปผลโครงงานและข้อเสนอแนะ

บทนี้นำเสนอสรุปผลการพัฒนาโครงงานเกมคอมพิวเตอร์ 2 มิติแนว Side-scrolling Hide-and-Seek Horror Game โดยสรุปตามวัตถุประสงค์และขอบเขตของโครงงาน อธิบายวิธีดำเนินงานโดยย่อ ผลที่ได้จากการพัฒนาระบบ ปัญหาและอุปสรรคที่พบ ตลอดจนข้อเสนอแนะสำหรับการนำผลงานไปใช้และการพัฒนาต่อยอดในอนาคต

5.1 สรุปผลโครงงาน

โครงงานนี้มีวัตถุประสงค์เพื่อพัฒนาเกมคอมพิวเตอร์ 2 มิติแนว Side-scrolling Hide-and-Seek Horror Game ด้วย Unity และภาษา C# โดยนำเทคนิค Finite State Machine หรือ FSM มาใช้ควบคุมสถานะของตัวละครผู้เล่นและพฤติกรรมของศัตรู เพื่อให้แต่ละระบบเปลี่ยนพฤติกรรมตามเงื่อนไขที่กำหนดไว้อย่างเป็นลำดับ นอกจากนี้ยังมีวัตถุประสงค์เพื่อพัฒนาระบบตรวจจับผู้เล่น ระบบไอเท็ม ระบบประตูทางออก ส่วนติดต่อผู้ใช้งาน และด่านที่มีระดับความยากแตกต่างกัน

การดำเนินโครงงานเริ่มจากการศึกษาหลักการพัฒนาเกม 2 มิติ การใช้งาน Unity การเขียนโปรแกรมด้วยภาษา C# และแนวคิด Finite State Machine จากนั้นจึงวิเคราะห์ความต้องการ ออกแบบโครงสร้างระบบ พัฒนาส่วนประกอบต่าง ๆ และทดสอบการทำงานภายในเกม โดยแบ่งระบบออกเป็นส่วนย่อย เพื่อให้สามารถพัฒนาและแก้ไขแต่ละระบบได้อย่างเป็นอิสระ

ผลการดำเนินโครงงานสามารถสรุปตามวัตถุประสงค์ได้ดังนี้

5.1.1 ผลการพัฒนาระบบควบคุมตัวละครผู้เล่น

สามารถพัฒนาระบบควบคุมตัวละครผู้เล่นให้รองรับการเดิน วิ่ง กระโดด ใช้ไฟฉาย เก็บไอเท็ม ซ่อนตัว เปิดกระเป๋าไอเท็ม และโต้ตอบกับประตูทางออกได้ การวิ่งส่งผลให้ผู้เล่นเคลื่อนที่เร็วขึ้น แต่ทำให้ Stamina ลดลงและสร้างเสียงที่ศัตรูสามารถตรวจจับได้ ส่วนไฟฉายมีปริมาณแบตเตอรี่จำกัด ทำให้ผู้เล่นต้องวางแผนการใช้งานระหว่างสำรวจด่าน

ระบบควบคุมผู้เล่นใช้ FSM แบ่งสถานะออกเป็น Normal, Hidden, Frozen, Hurt, UsingItem, Transitioning และ Dead การแบ่งสถานะดังกล่าวช่วยควบคุมเงื่อนไขการรับคำสั่งและป้องกันพฤติกรรมที่ขัดแย้งกัน เช่น การควบคุมตัวละครระหว่างเสียชีวิต การเคลื่อนที่ขณะถูกหยุด หรือการรับคำสั่งระหว่างเปลี่ยนฉาก

5.1.2 ผลการพัฒนาระบบปัญญาประดิษฐ์ของศัตรู

สามารถพัฒนาระบบปัญญาประดิษฐ์ของศัตรูให้ตอบสนองต่อผู้เล่นผ่านการมองเห็นและการได้ยิน ศัตรูสามารถลาดตระเวน รับฟังเสียง ตรวจสอบตำแหน่งที่เกิดเสียง ค้นหาผู้เล่น ไล่ล่า โจมตี และกลับไปยังเส้นทางลาดตระเวนได้ Enemy FSM ประกอบด้วยสถานะ Patrol, Alert, Listening, InvestigateSound, ConfirmSound, Confused, Investigate, Search, Chase, Attack,

AttackRecover, ReturnToPatrol และ Scream สถานะเหล่านี้ช่วยให้ศัตรูมีพฤติกรรมหลายขั้นตอนและตอบสนองต่อสถานการณ์ได้เหมาะสมกว่าการกำหนดเงื่อนไขทั้งหมดไว้ในกระบวนการเดียว

เมื่อศัตรูได้ยินเสียงจากการเดิน วิ่ง หรือกับดัก ศัตรูจะจดจำตำแหน่งกำเนิดเสียงไว้ชั่วคราวและเคลื่อนที่ไปตรวจสอบ หากมองเห็นผู้เล่นจะเปลี่ยนเข้าสู่สถานะ Chase และโจมตีเมื่อผู้เล่นอยู่ในระยะที่กำหนด หากสูญเสียการตรวจจับ ศัตรูจะตรวจสอบตำแหน่งล่าสุดหรือค้นหาในบริเวณใกล้เคียงก่อนกลับไปลาดตระเวน

5.1.3 ผลการพัฒนาระบบตรวจจับ

สามารถพัฒนาระบบตรวจจับด้วยการมองเห็นและการได้ยินให้ทำงานร่วมกับ Enemy FSM ได้ ระบบการมองเห็นพิจารณาระยะตรวจจับ ทิศทาง และสิ่งกีดขวางภายในฉาก ส่วนระบบการได้ยินรับเหตุการณ์เสียงจากการกระทำของผู้เล่นและวัตถุภายในด่าน

เสียงแต่ละประเภทมีความสำคัญแตกต่างกัน โดยเสียงจากการวิ่งและเสียงจากกับดักกระจกสามารถดึงดูดความสนใจของศัตรูได้มากกว่าเสียงเดินทั่วไป ระบบดังกล่าวทำให้เสียงเป็นส่วนหนึ่งของการเล่น ผู้เล่นจึงต้องเลือกระหว่างการเคลื่อนที่อย่างรวดเร็วกับการลดโอกาสที่ศัตรูจะตรวจพบตำแหน่ง

5.1.4 ผลการพัฒนาระบบไอเท็มและประตูทางออก

สามารถพัฒนาระบบเก็บกุญแจและกระเป๋าไอเท็มโดยใช้ InventoryManager สำหรับจัดเก็บและตรวจสอบจำนวนไอเท็ม เมื่อผู้เล่นเก็บกุญแจ ข้อมูลบน HUD และในกระเป๋าไอเท็มจะได้รับการปรับปรุงตามจำนวนที่เก็บได้

ระบบประตูทางออกใช้ FSM แบ่งสถานะเป็น Locked, Ready, Unlocking, Open, Transitioning และ Completed ประตูจะยังไม่สามารถเปิดได้หากผู้เล่นเก็บกุญแจไม่ครบ เมื่อผู้เล่นมีจำนวนกุญแจตามที่กำหนดและกดปุ่มโต้ตอบค้าง ระบบจะแสดงความคืบหน้าการปลดล็อก ก่อนเปิดประตูและเปลี่ยนไปยังด่านถัดไปหรือแสดงผลการจบเกม

5.1.5 ผลการพัฒนาด่านและบรรยากาศของเกม

สามารถพัฒนาด่านภายในเกมให้มีระดับความยากแตกต่างกันผ่าน LevelConfig ซึ่งกำหนดค่าต่าง ๆ เช่น ความเร็วของศัตรู ระยะการมองเห็น ระยะการได้ยิน ความเสียหาย และปริมาณแบตเตอรี่เริ่มต้น

Level 3 ได้รับการออกแบบให้เป็นสวนสนุก โดยต้นไม้ พุ่มไม้ พื้นหญ้า และเส้นทางต่างระดับเพื่อสร้างบรรยากาศน่ากลัว ภายในด่านมีการสุ่มตำแหน่งกุญแจและกับดักในพื้นที่ที่ผู้เล่นสามารถเดินถึง ทำให้การเล่นแต่ละรอบมีความแตกต่างกัน

ศัตรูใน Level 3 เน้นการตรวจจับด้วยเสียง มีความเร็วในการไล่ล่าและระยะการได้ยินสูงกว่าด่านอื่น เมื่อผู้เล่นเข้าใกล้ศัตรูหรือถูกไล่ล่า ระบบจะเล่นเสียงหัวใจ แสดงขอบจอเตือน และปรับระดับดนตรีตามสถานการณ์ ส่งผลให้ผู้เล่นรับรู้ถึงอันตรายและความกดดันได้อย่างชัดเจน

5.1.6 ผลการพัฒนาส่วนติดต่อผู้ใช้งาน

สามารถพัฒนาส่วนติดต่อผู้ใช้งานให้แสดงข้อมูลสำคัญระหว่างเล่น ได้แก่ พลังชีวิต Stamina ปริมาณแบตเตอรี่ จำนวนกุญแจ ข้อความโต้ตอบ สถานะการซ่อน และคำเตือนเมื่อศัตรูอยู่ใกล้

ระบบแสดงผลจะเปลี่ยนตามสถานการณ์ เช่น แสดงข้อความแนะนำเมื่อผู้เล่นเข้าใกล้วัตถุ แสดง DANGER เมื่อศัตรูไล่ล่า แสดง Game Over เมื่อผู้เล่นเสียชีวิต และแสดง Level Complete หรือ Game Won เมื่อผู้เล่นผ่านเงื่อนไขของด่าน

5.1.7 อภิปรายผลการพัฒนา

ผลการพัฒนาแสดงให้เห็นว่า Finite State Machine เหมาะสำหรับระบบที่ประกอบด้วยพฤติกรรมหลายสถานะและมีเงื่อนไขการเปลี่ยนสถานะที่ชัดเจน เนื่องจากช่วยแยกหน้าที่ของแต่ละสถานะ ลดความซับซ้อนของการตรวจสอบเงื่อนไข และช่วยให้สามารถติดตามพฤติกรรมของระบบได้ง่ายขึ้น

ผลดังกล่าวสอดคล้องกับหลักการของ FSM ที่กำหนดให้ระบบอยู่ในสถานะหนึ่ง ณ ช่วงเวลาหนึ่ง และเปลี่ยนไปยังสถานะใหม่เมื่อเกิดเหตุการณ์หรือเป็นไปตามเงื่อนไขที่กำหนด การนำแนวคิดนี้มาใช้กับผู้เล่นช่วยป้องกันคำสั่งที่ขัดแย้งกัน ส่วนการใช้กับศัตรูช่วยให้กระบวนการลาดตระเวน ตรวจจับ ค้นหา ไล่ล่า และโจมตีมีลำดับที่ชัดเจน

การใช้ระบบตรวจจับด้วยเสียงร่วมกับ FSM ช่วยเพิ่มความหลากหลายของพฤติกรรมศัตรู เนื่องจากศัตรูไม่จำเป็นต้องมองเห็นผู้เล่นโดยตรงจึงจะตอบสนองได้ อย่างไรก็ตาม ความสมจริงของศัตรูยังขึ้นอยู่กับการกำหนดค่าความเร็ว ระยะตรวจจับ ระยะเวลาของแต่ละสถานะ และรูปแบบพื้นที่ภายในด่าน หากกำหนดค่าไม่เหมาะสม เกมอาจง่ายหรือยากเกินไป

การแบ่งระบบออกเป็นส่วนย่อย เช่น PlayerController, KillerAI, InventoryManager, ExitDoor, GameManager, UIManager และระบบเสียงของด่าน
ช่วยให้สามารถปรับปรุงแต่ละระบบได้โดยไม่กระทบส่วนอื่นมากเกินไป
และช่วยให้ตรวจสอบข้อผิดพลาดได้สะดวกกว่าการรวมการทำงานทั้งหมดไว้ในสคริปต์เดียว
ระบบความตึงเครียดยังใช้ TensionDirector ที่ประกาศในไฟล์ LevelAudioController.cs
เพื่อรวมสถานะศัตรูและแจ้งระบบเสียง

5.1.8 สถานะการบรรลุวัตถุประสงค์

ด้านการพัฒนาซอฟต์แวร์ มีต้นแบบและโครงสร้าง FSM ของผู้เล่นและศัตรู พร้อม Animator ที่อธิบายในบท 3 และผลตรวจในบท 4 ด้าน Gameplay Loop มีเมธอดและหน้าจอผ่านด่าน แต่ยังต้องเล่นทดสอบต่อเนื่องทุกด่านเพื่อยืนยัน ด้านการประเมินความพึงพอใจยังไม่บรรลุ เนื่องจากยังไม่ได้เก็บแบบสอบถาม ตัวอย่างข้อมูลสมมติในภาคผนวกใช้สอนวิธีคำนวณเท่านั้น

5.2 ปัญหาที่พบในโครงงาน

5.2.1 ปัญหาการเปลี่ยนสถานะของ FSM

ในระหว่างการพัฒนา
มีความเสี่ยงว่าบางสถานการณ์ทำให้ระบบพยายามเปลี่ยนสถานะมากกว่าหนึ่งครั้งในเวลาใกล้เคียงกัน เช่น ผู้เล่นได้รับความเสียหายขณะเปิดกระเป๋า ถูกศัตรูโจมตีระหว่างปลดล็อกประตู
หรือศัตรูได้ยินเสียงพร้อมกับมองเห็นผู้เล่น
ปัญหานี้อาจทำให้คำสั่งบางอย่างทำงานซ้อนกันหรือระบบเปลี่ยนไปยังสถานะที่ไม่เหมาะสม

แนวทางแก้ไขคือกำหนดลำดับความสำคัญของสถานะ ตรวจสอบสถานะปัจจุบันก่อนรับคำสั่ง และจำกัดให้แต่ละระบบเปลี่ยนสถานะผ่านฟังก์ชันส่วนกลาง เช่น SetPlayerState() หรือ SetState() เพื่อให้สามารถควบคุมการเปลี่ยนสถานะได้จากตำแหน่งเดียว

5.2.2 ปัญหาการตรวจจับของศัตรู

ระบบการมองเห็นของศัตรูอาจตรวจจับผิดพลาดเมื่อผู้เล่นอยู่ใกล้ขอบสิ่งกีดขวาง หรือเมื่อ Collider ของวัตถุไม่ตรงกับภาพที่แสดงในฉาก
ส่วนระบบการได้ยินอาจทำให้ศัตรูตอบสนองต่อเสียงหลายเหตุการณ์พร้อมกัน
จนเกิดการเปลี่ยนเป้าหมายบ่อยเกินไป

แนวทางแก้ไขคือปรับ LayerMask และ Collider ให้ตรงกับวัตถุ ตรวจสอบแนว Raycast ที่ใช้ในการมองเห็น และกำหนดระดับความสำคัญของเสียง ศัตรูจะเลือกตรวจสอบเสียงที่มีความสำคัญมากกว่าหรือเสียงที่เกิดขึ้นล่าสุด พร้อมเก็บตำแหน่งเสียงไว้เพียงช่วงเวลาที่กำหนด

5.2.3 ปัญหาการเคลื่อนที่ของศัตรูในฉากสองมิติ

ศัตรูอาจติดขอบพื้น สิ่งกีดขวาง หรือพื้นที่ต่างระดับ โดยเฉพาะฉากที่ประกอบจาก Tilemap หลายชั้น หากจุดลาดตระเวนอยู่ในตำแหน่งที่ศัตรูเดินไม่ถึง
ระบบอาจพยายามเคลื่อนที่ไปยังเป้าหมายเดิมอย่างต่อเนื่อง

แนวทางแก้ไขคือกำหนดเส้นทางลาดตระเวนให้สอดคล้องกับพื้นที่ที่เดินได้ ตรวจสอบพื้นและขอบทางเดินก่อนเปลี่ยนทิศทาง รวมถึงกำหนดเงื่อนไขให้ศัตรูกลับไปยังจุดลาดตระเวนที่เหมาะสมเมื่อไม่สามารถเดินทางไปยังเป้าหมายเดิมได้

5.2.4 ปัญหาการสุ่มตำแหน่งวัตถุใน Level 3

การสุ่มตำแหน่งกุญแจและกับดักอาจทำให้วัตถุปรากฏซ้อนกัน อยู่ใกล้กันเกินไป หรืออยู่ในตำแหน่งที่ผู้เล่นไม่สามารถเข้าถึงได้ หากสุ่มโดยใช้พิกัดทั่วไปโดยไม่ตรวจสอบพื้นภายในฉาก

แนวทางแก้ไขคือให้ Level3RandomEvents
เลือกตำแหน่งจากพื้นที่ที่ตรวจสอบแล้วว่าสามารถเดินได้ กำหนดระยะห่างขั้นต่ำระหว่างวัตถุ และแยกจุดสุ่มของกุญแจกับกับดัก เพื่อลดโอกาสที่วัตถุหลายชิ้นจะรวมอยู่ในบริเวณเดียวกัน

5.2.5 ปัญหาการปรับสมดุลระดับความยาก

ค่าความเร็ว ระยะการมองเห็น ระยะการได้ยิน และความเสียหายของศัตรูมีผลต่อความยากอย่างมาก หากเพิ่มหลายค่าพร้อมกัน ผู้เล่นอาจไม่มีเวลาเพียงพอสำหรับหลบหนีหรือหาจุดซ่อน โดยเฉพาะ Level 3 ซึ่งกำหนดให้ศัตรูมีความเร็วและระยะการได้ยินสูง

แนวทางแก้ไขคือแยกค่าความยากของแต่ละด่านไว้ใน LevelConfig
และปรับค่าทีละส่วนจากผลการทดลองเล่น เช่น ปรับความเร็วไล่ล่า ระยะได้ยิน เวลาค้นหา และจำนวนจุดซ่อน เพื่อหาค่าที่สร้างความกดดันโดยยังเปิดโอกาสให้ผู้เล่นสามารถผ่านด่านได้

5.2.6 ปัญหาการจัดวางกล้องและองค์ประกอบของด่าน

กล้องอาจอยู่ต่ำหรือสูงเกินไป
ทำให้ไม่สามารถมองเห็นเส้นทางและองค์ประกอบสำคัญของด่านได้ครบถ้วน นอกจากนี้ การใช้ขอบเขตกล้องร่วมกันกับด่านที่มีขนาดแตกต่างกันอาจทำให้เกิดพื้นที่ว่างนอกฉากหรือทำให้วัตถุบางส่วนอยู่นอกหน้าจอ

แนวทางแก้ไขคือกำหนดตำแหน่งและขอบเขตกล้องแยกตามแต่ละด่าน ปรับค่าแกนตั้งของกล้องให้เหมาะกับระดับพื้น และทดสอบบริเวณขอบซ้าย ขอบขวา พื้นที่ชั้นบน และพื้นที่ชั้นล่างก่อนสรุปการจัดวางด่าน

5.2.7 ปัญหาการทำงานร่วมกันของหลายระบบ

ระบบผู้เล่น ศัตรู UI เสียง ไอเท็ม และการเปลี่ยนฉากมีการส่งข้อมูลถึงกันหลายส่วน หากวัตถุหรือคอมโพเนนต์บางรายการไม่ได้กำหนดใน Inspector
อาจเกิดข้อผิดพลาดหรือทำให้ฟังก์ชันบางอย่างไม่ทำงาน

แนวทางแก้ไขคือเพิ่มการตรวจสอบค่าว่างก่อนเรียกใช้งาน แยกความรับผิดชอบของแต่ละสคริปต์ ใช้ Event สำหรับส่งข้อมูลระหว่างระบบที่ไม่จำเป็นต้องอ้างอิงกันโดยตรง และตรวจสอบ Unity Console ทุกครั้งหลังแก้ไข Scene, Prefab หรือสคริปต์ส่วนกลาง

5.2.8 สิ่งที่ได้เรียนรู้จากการพัฒนา

ผู้พัฒนาได้เรียนรู้ว่าการออกแบบโครงสร้างระบบก่อนเขียนโปรแกรมช่วยลดการแก้ไขโค้ดในภายหลัง การแบ่งระบบออกเป็นโมดูลและกำหนดสถานะด้วย FSM ทำให้สามารถวิเคราะห์สาเหตุของข้อผิดพลาดได้ชัดเจนขึ้น

นอกจากนี้ การทดสอบจากการเล่นจริงมีความสำคัญต่อการพัฒนาเกม เนื่องจากระบบที่ทำงานถูกต้องตามโค้ดอาจยังไม่ให้ประสบการณ์ที่เหมาะสม เช่น ศัตรูอาจเร็วเกินไป คำเตือนอาจไม่ชัดเจน หรือการจัดวางวัตถุอาจทำให้ผู้เล่นไม่เข้าใจเส้นทาง ดังนั้นผลจากผู้ทดลองเล่นจึงควรถูกนำมาใช้ร่วมกับการทดสอบทางเทคนิค

5.3 ข้อเสนอแนะ

5.3.1 ข้อเสนอแนะสำหรับการนำระบบไปใช้

ก่อนนำเกมไปใช้งานหรือเผยแพร่ ควรทดสอบกรณีทดสอบทั้งหมดและบันทึกผลที่เกิดขึ้นจริง โดยเฉพาะการเปลี่ยนสถานะของผู้เล่นและศัตรู การตรวจจับเสียง การปลดล็อกประตู และการผ่าน Level 3 ไม่ควรสรุปว่าระบบสมบูรณ์จากการตรวจสอบโค้ดเพียงอย่างเดียว

ควรทดสอบกับอุปกรณ์ที่มีประสิทธิภาพแตกต่างกัน ตรวจสอบอัตราเฟรม ความละเอียดหน้าจอ ตำแหน่ง UI และการตอบสนองของปุ่มควบคุม เนื่องจากผลการทำงานอาจแตกต่างกันตามขนาดหน้าจอและความสามารถของอุปกรณ์

การปรับค่าความยากผ่าน LevelConfig ควรดำเนินการอย่างระมัดระวัง เพราะความเร็ว ระยะตรวจจับ ความเสียหาย และทรัพยากรเริ่มต้นมีผลต่อกัน หากต้องการปรับความยากควรเปลี่ยนค่าทีละส่วนและทดลองเล่นซ้ำก่อนนำไปใช้จริง

5.3.2 ข้อเสนอแนะสำหรับการพัฒนาระบบเพิ่มเติม

ควรเพิ่มรูปแบบพฤติกรรมของศัตรูให้หลากหลาย เช่น การตรวจสอบจุดซ่อน การคาดเดาเส้นทางของผู้เล่น การตอบสนองร่วมกันของศัตรูหลายตัว และการจดจำพื้นที่ที่พบผู้เล่นบ่อย เพื่อให้การเล่นแต่ละรอบไม่สามารถคาดเดาได้ง่าย

ควรเพิ่มความหลากหลายของไอเท็ม เช่น ไอเท็มฟื้นฟูพลังชีวิต แบตเตอรี่สำรอง อุปกรณ์สร้างเสียงล่อศัตรู หรือไอเท็มป้องกันการโจมตีชั่วคราว โดยต้องกำหนดข้อจำกัดเพื่อไม่ให้ไอเท็มลดความน่ากลัวของเกมมากเกินไป

ควรพัฒนาระบบตั้งค่าให้รองรับการปรับระดับเสียง ความสว่าง ความละเอียดหน้าจอ และการกำหนดปุ่มควบคุมใหม่ รวมถึงเพิ่มตัวเลือกด้านการเข้าถึง เช่น ขนาดข้อความ คำบรรยายเสียง และการปรับความรุนแรงของเอฟเฟกต์ภาพ

ควรเพิ่มระบบบันทึกความคืบหน้าในอนาคต โดยต้องกำหนดข้อมูลที่จำเป็น เช่น ด่านปัจจุบัน จำนวนไอเท็ม สถานะผู้เล่น และค่าตั้งค่า พร้อมตรวจสอบความถูกต้องของข้อมูลก่อนโหลดกลับเข้าสู่เกม

5.3.3 ข้อเสนอแนะสำหรับการศึกษาและวิจัยครั้งต่อไป

การศึกษาครั้งต่อไปควรเปรียบเทียบประสิทธิภาพของ FSM กับเทคนิคควบคุมพฤติกรรมแบบอื่น เช่น Behavior Tree หรือ Utility AI โดยพิจารณาความซับซ้อนของโค้ด เวลาในการประมวลผล ความสะดวกในการเพิ่มพฤติกรรม และความสมจริงที่ผู้เล่นรับรู้

ควรศึกษาผลของเสียงต่อพฤติกรรมของผู้เล่น โดยทดลองปรับระดับความดัง ความถี่ของเสียง หัวใจ และระยะการได้ยินของศัตรู แล้วประเมินว่าค่าใดสามารถสร้างความกดดันได้เหมาะสมโดยไม่ทำให้ผู้เล่นสับสนหรือรู้สึกว่าเกมไม่ยุติธรรม

ควรศึกษาการสุ่มตำแหน่งวัตถุและเหตุการณ์ภายในด่านว่ามีผลต่อความน่ากลัวและความต้องการเล่นซ้ำหรือไม่ โดยเปรียบเทียบระหว่างด่านที่มีตำแหน่งคงที่กับด่านที่มีการสุ่ม พร้อมเก็บข้อมูลระยะเวลาเล่น จำนวนครั้งที่ถูกจับ และความคิดเห็นของผู้เล่น

ควรเริ่มเก็บข้อมูลจากผู้ทดลองเล่นที่มีประสบการณ์ต่างกัน ระบุวิธีคัดเลือก จำนวนผู้ตอบ วันที่และอุปกรณ์ทดสอบ พร้อมเก็บข้อมูลดิบเพื่อทวนสอบ
เนื่องจากรุ่นรายงานนี้ยังไม่มีข้อมูลจากกลุ่มตัวอย่างจริง

บรรณานุกรม

Jagdale, D. (2021). Finite State Machine in Game Development. International Journal of Advanced Research in Science, Communication and Technology, 10(1), 384–390. https://doi.org/10.48175/IJARSCT-2062

Kusuma, A. F. A. A., & Handayani, R. D. (2024). Implementation of Inventory Checking System Using Finite State Machine in Role Playing Game. International Journal of Computer and Information System, 5(4). https://doi.org/10.29040/ijcis.v5i4.197

Thyrun, V. (2026). An Analysis of Finite State Machine Based Enemy Artificial Intelligence in Kirby: Nightmare in Dream Land. Preprints.org [บทความก่อนประเมินโดยผู้ทรงคุณวุฒิ]. https://doi.org/10.20944/preprints202601.1473.v1

Koulaxidis, G., & Xinogalos, S. (2022). Improving Mobile Game Performance with Basic Optimization Techniques in Unity. Modelling, 3, 201–223. https://doi.org/10.3390/modelling3020014

Unity Technologies. (2026). Animator Controller. Unity Manual 6.4. สืบค้น 4 ตุลาคม 2569 จาก https://docs.unity3d.com/6000.4/Documentation/Manual/class-AnimatorController.html

Unity Technologies. (2026). Animation Parameters. Unity Manual 6.4. สืบค้น 4 ตุลาคม 2569 จาก https://docs.unity3d.com/6000.4/Documentation/Manual/AnimationParameters.html

Unity Technologies. (2026). Physics2D.Raycast. Unity Scripting API 6.4. สืบค้น 4 ตุลาคม 2569 จาก https://docs.unity3d.com/6000.4/Documentation/ScriptReference/Physics2D.Raycast.html

ผู้จัดทำ. (2569). SideScrollingEscape [โค้ดต้นแบบและทรัพยากรเกม Unity]. PlayerController.cs, KillerAI.cs, PlayerAnimator.controller, KillerAnimator.controller, InventoryManager.cs, ExitDoor.cs, UIManager.cs และ LevelConfig. ตรวจสอบ 4 ตุลาคม 2569.

ภาคผนวก ก หลักฐานและวิธีการสาธิต

ก.1 แหล่งภาพและข้อจำกัด

ภาพเกมในบท 3 และ 4 มาจาก Unity Play Mode ของโครงการ ภาพ Animator ที่ 3-22 และ 3-23 มาจากหน้าต่าง Animator ของ Unity ส่วน 4-16–4-21 เป็นเฟรมจาก Sprite Sheet จริง แผนภาพที่ 3-18–3-21 และ 4-7 วาดสรุปจากโค้ดเพื่ออธิบายโครงสร้าง ไม่ใช่ภาพจับจากการเล่น

สคริปต์สาธิตชั่วคราวใช้ส่งปุ่ม D, Shift และ Space เรียก SetOn, Collect, OpenRoutine, ToggleHide, ForceReveal, Freeze, Unfreeze, TakeDamage, HearNoise, SetState, CompleteUnlock, Kill และ ApplyStatueCurse รวมถึงคำสั่งเปลี่ยนด่าน บางภาพจัดตำแหน่งผู้เล่นและ Killer ปิดการอัปเดตศัตรู ปิดโอกาสสุ่ม Jumpscare หรือคงสถานะประตูเพื่อแสดงผล ภาพดังกล่าวไม่ใช้ยืนยันว่าผู้เล่นทำสำเร็จผ่านการเล่นตามปกติ

ก.2 บันทึกสถานะที่สังเกตได้

ตารางที่ ก-1 ตัวอย่างสถานะจากบันทึกการสาธิต

เหตุการณ์สาธิต	สถานะผู้เล่น	ผลแสดงหรือสถานะเกม
เปิดคลัง	UsingItem	หน้าต่าง Inventory
เข้าและออกจากการซ่อน	Hidden ⟶ Normal	HIDDEN แสดงและหาย
หยุดผู้เล่น	Frozen	จัดสถานะหยุดชั่วคราว
ความเสียหาย 10 หน่วย	Hurt	แถบ Health เปลี่ยน
ส่งเสียง Rock	Normal	Killer: Listening
เรียก Chase	Normal	Killer: Chase / DANGER
เรียก CompleteUnlock	Transitioning	LevelComplete
เรียก Kill	Dead	Dead / Game Over
ผ่านเงื่อนไขด่านสุดท้ายโดยคำสั่ง	Normal	GameWon / YOU ESCAPED

ก.3 วิธีทดสอบต่อ

คืนการทำงานตามปกติของศัตรูและการสุ่มเหตุการณ์ เล่นแต่ละ Test Case ด้วยปุ่มควบคุมจริง บันทึกผลเทียบเกณฑ์ทีละข้อ แล้วเล่นจากเมนูจนผ่านทุกด่านอย่างต่อเนื่อง ควรตรวจ Console ปัญหาการชน แสง กล้อง เสียง และการยกเลิกปลดล็อก จากนั้นจึงเก็บแบบสอบถามกับผู้ทดลองเล่น

ภาคผนวก ข ตัวอย่างคะแนนสมมติและแบบประเมิน

ข.1 ข้อมูลสมมติสำหรับแสดงวิธีคำนวณ

ข้อมูลทั้งหมดในหัวข้อนี้สร้างขึ้นเพื่อเป็นตัวอย่าง ไม่ได้มาจากผู้ทดลองเล่นจริง รหัส S1–S5 หมายถึงแถวข้อมูลสมมติ 5 แถว ไม่ใช่จำนวนกลุ่มตัวอย่างจริง
จึงห้ามนำค่าเฉลี่ยหรือส่วนเบี่ยงเบนมาตรฐานนี้ไปอ้างว่าเป็นผลความพึงพอใจของเกม

ตารางที่ ข-1 คะแนนสมมติสำหรับตัวอย่างการคำนวณเท่านั้น

รหัสสมมติ	ควบคุม	UI	ศัตรู	ความยาก	บรรยากาศ	โดยรวม
S1	4	4	4	3	5	4
S2	5	4	4	4	4	4
S3	4	5	3	4	5	5
S4	4	4	4	3	4	4
S5	5	4	5	4	4	5

ค่าเฉลี่ย $\bar{x} = \sum x_i / n$ และส่วนเบี่ยงเบนมาตรฐานของตัวอย่าง $s = \sqrt{[\sum (x_i - \bar{x})^2 / (n - 1)]}$ โดย n ในตัวอย่างนี้คือจำนวนแถวสมมติ สำหรับด้านควบคุม ผลรวมเท่ากับ 22 จึงได้ค่าเฉลี่ย 22/5 = 4.40 และ s = 0.55 เมื่อปัดสองตำแหน่ง

ตารางที่ ข-2 ผลคำนวณจากคะแนนสมมติเท่านั้น

ด้านสมมติ	ค่าเฉลี่ย	s	ระดับตัวอย่าง
การควบคุม	4.40	0.55	ดีมาก
UI	4.20	0.45	ดี
พฤติกรรมศัตรู	4.00	0.71	ดี
ความยาก	3.60	0.55	ดี
บรรยากาศ	4.40	0.55	ดีมาก
ความพึงพอใจโดยรวม	4.40	0.55	ดีมาก

เกณฑ์ตัวอย่างแบ่งค่าเฉลี่ยเป็น 4.21–5.00 ดีมาก, 3.41–4.20 ดี, 2.61–3.40 ปานกลาง, 1.81–2.60 น้อย และ 1.00–1.80 น้อยมาก ในข้อมูลสมมตินี้บรรยากาศกับความพึงพอใจโดยรวมมีค่าเฉลี่ย 4.40 และด้านความยาก 3.60 เป็นเพียงการฝึกแปลผล ยังไม่ใช่ข้อค้นพบจากเกมหรือผู้ใช้งานจริง

ข.2 แบบประเมินสำหรับเก็บข้อมูลจริง

ให้ผู้ตอบทดลองเล่นด้วยระบบตามปกติ แล้วเลือกคะแนนแต่ละข้อจาก 1 ถึง 5 โดย 1 น้อยมาก 2 น้อย 3 ปานกลาง 4 ดี และ 5 ดีมาก แบบฟอร์มนี้ยังไม่ได้รับคำตอบจริง

วันที่ทดลอง _____ อุปกรณ์ที่ใช้ _____

ประสบการณ์เล่นเกมสยองขวัญ: ไม่เคย / บางครั้ง / เป็นประจำ

ตารางที่ ข-3 แบบประเมินสำหรับผู้ทดลองเล่นจริง

รายการประเมิน	1	2	3	4	5
ควบคุมตัวละครได้สะดวก	○	○	○	○	○
UI และข้อความแนะนำชัดเจน	○	○	○	○	○
พฤติกรรมศัตรูเหมาะสม	○	○	○	○	○
ระดับความยากมีความสมดุล	○	○	○	○	○
บรรยากาศสอดคล้องกับเกมสยองขวัญ	○	○	○	○	○
ความพึงพอใจโดยรวม	○	○	○	○	○

คำถามปลายเปิด: ส่วนใดเข้าใจยากที่สุด เหตุการณ์ใดทำให้รู้สึกไม่ยุติธรรม พบปัญหาหรือข้อผิดพลาดใด และต้องการให้ปรับปรุงอะไร ควรเก็บคำตอบดิบไว้ตรวจสอบและแยกข้อเสนอแนะจริงออกจากความคิดเห็นของผู้พัฒนา

ข้อเสนอแนะ _____

_____

_____